

Frequency Fitness Assignment

频率适应度分配

Thomas Weise (汤卫思)

tweise@hfu.edu.cn · <https://thomasweise.github.io>

School of Artificial Intelligence and Big Data
Hefei University
Hefei, Anhui, China

人工智能与大数据学院
合肥大学
中国安徽省合肥市

Outline



1. Introduction
2. Metaheuristic Optimization
3. Invariance Properties
4. Frequency Fitness Assignment
5. Where did we try FFA?
6. Summary
7. Advertisement





Introduction



Introduction to Optimization



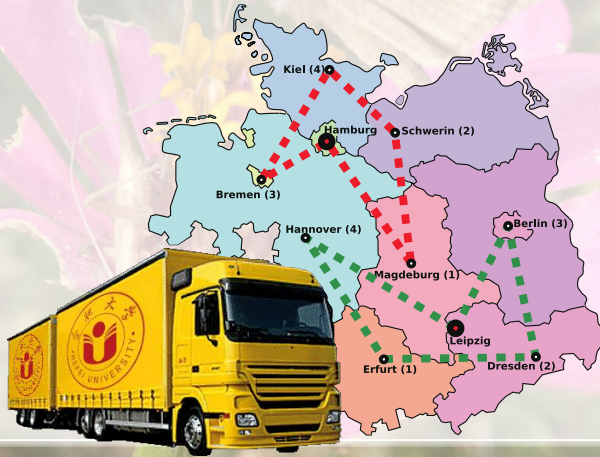
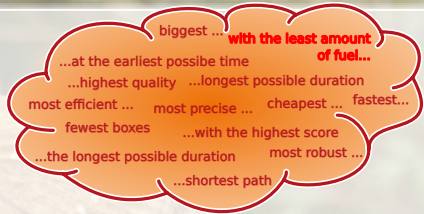
- Optimization means finding superlatives.

biggest ... with the least amount
of fuel...
...at the earliest possible time
...highest quality ...longest possible duration
most efficient ... most precise ... cheapest ... fastest...
fewest boxes ...with the highest score
...the longest possible duration most robust ...
...shortest path

The *superlative* form of an adjective is used to show that something has a quality to the greatest or least degree.

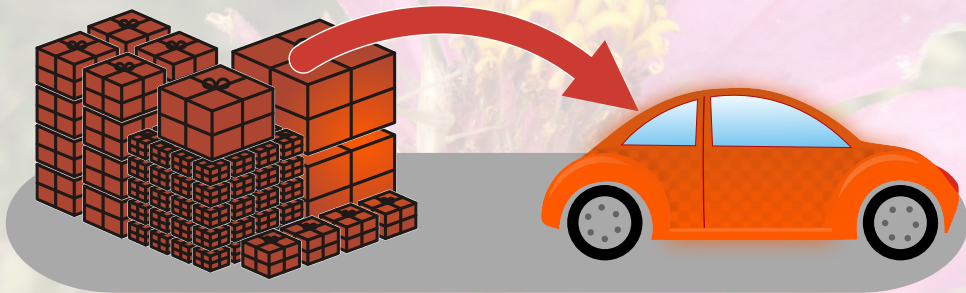
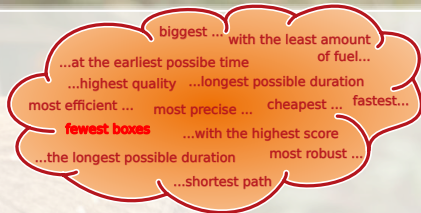
Introduction to Optimization

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{75,101–103}.



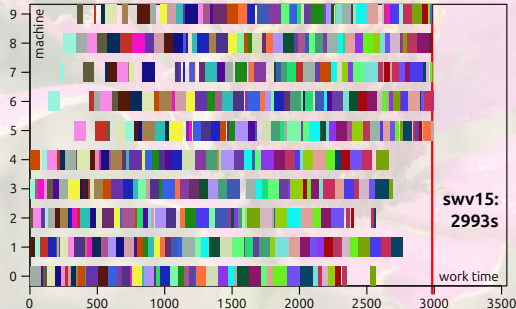
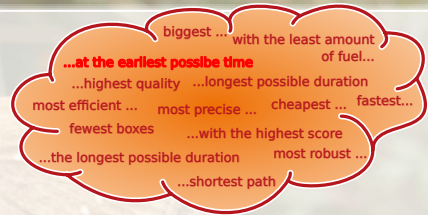
Introduction to Optimization

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{75,101–103}.
- Pack a set of things into the **fewest** boxes^{28,87,117,118}.



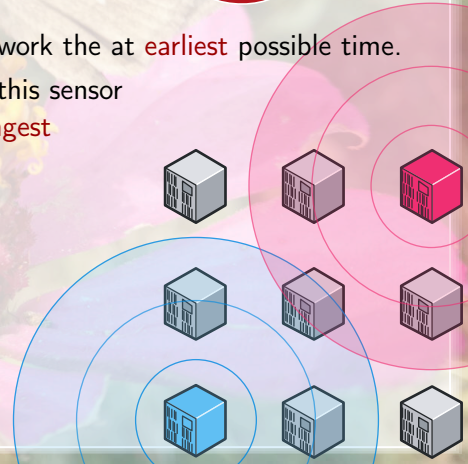
Introduction to Optimization

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{75,101–103}.
- Pack a set of things into the **fewest** boxes^{28,87,117,118}.
- Assign tasks to machines such that we can finish our work the at **earliest** possible time.



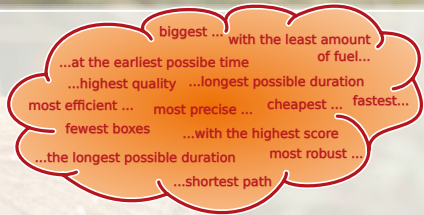
Introduction to Optimization

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{75,101-103}.
- Pack a set of things into the **fewest** boxes^{28,87,117,118}.
- Assign tasks to machines such that we can finish our work the at **earliest** possible time.
- Find a strategy to manage the power of the nodes in this sensor network so that full coverage is guaranteed for the **longest** possible duration.



Introduction to Optimization

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{75,101-103}.
- Pack a set of things into the **fewest** boxes^{28,87,117,118}.
- Assign tasks to machines such that we can finish our work the at **earliest** possible time.
- Find a strategy to manage the power of the nodes in this sensor network so that full coverage is guaranteed for the **longest** possible duration.
- And so on.



What are Optimization Problems?

- There are two ways to define optimization problems.



What are Optimization Problems?



- The economic view.

Optimization Problem

An optimization problem is a situation which requires deciding for one choice from a set of possible alternatives in order to reach a predefined or required benefit at minimal costs.

What are Optimization Problems?



- The mathematical view.

Optimization Problem

An optimization problem is a situation which requires deciding for one choice from a set of possible alternatives in order to reach a predefined or required benefit at minimal costs.

Solving an optimization problem requires finding an input element x^* within a set $\mathbb{X}$ of allowed elements for which a mathematical function $f: \mathbb{X} \mapsto \mathbb{R}$ takes on the smallest possible value.

Example: Traveling Salesperson Problem

- In the Traveling Salesperson Problem (TSP)^{3,40,57,59,98,110}, the goal is to find the shortest round-trip tour through a set of n cities.



Example: Traveling Salesperson Problem

- In the Traveling Salesperson Problem (TSP)^{3,40,57,59,98,110}, the goal is to find the shortest round-trip tour through a set of n cities.
- The search space $\mathbb{X}$ thus is the set of all possible round-trip tours through these n cities, usually specified as permutations of the first n natural numbers.



Example: Traveling Salesperson Problem

- In the Traveling Salesperson Problem (TSP)^{3,40,57,59,98,110}, the goal is to find the shortest round-trip tour through a set of n cities.
- The search space $\mathbb{X}$ thus is the set of all possible round-trip tours through these n cities, usually specified as permutations of the first n natural numbers.
- The objective function $f : \mathbb{X} \mapsto \mathbb{R}$, subject to minimization, is the length of the tour.



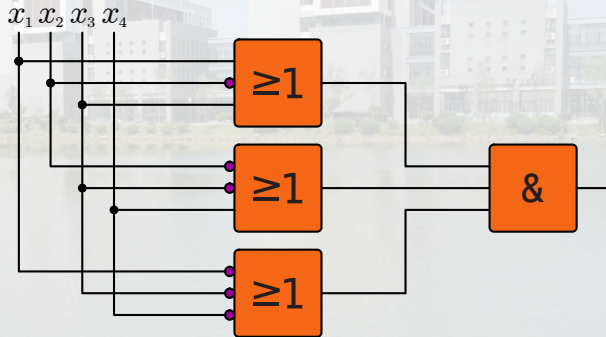
Example: Traveling Salesperson Problem

- In the Traveling Salesperson Problem (TSP)^{3,40,57,59,98,110}, the goal is to find the shortest round-trip tour through a set of n cities.
- The search space $\mathbb{X}$ thus is the set of all possible round-trip tours through these n cities, usually specified as permutations of the first n natural numbers.
- The objective function $f : \mathbb{X} \mapsto \mathbb{R}$, subject to minimization, is the length of the tour.
- The optimal solution $x^* \in \mathbb{X}$ is the shortest possible tour.



Example: Maximum Satisfiability Problem

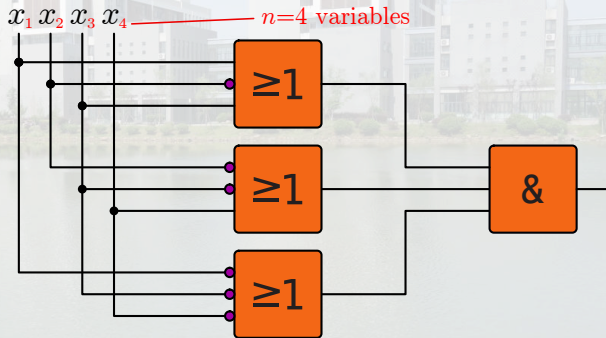
- The goal of the Maximum Satisfiability (MaxSAT)^{17,42} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into one AND-clause.





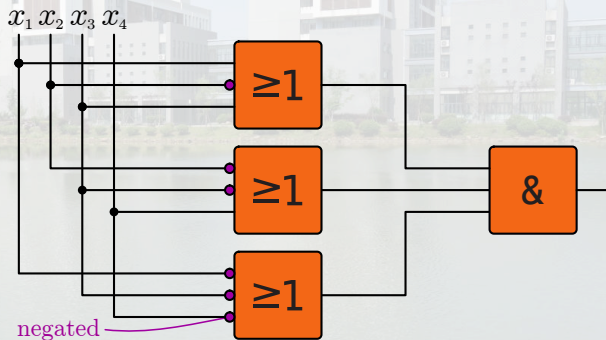
Example: Maximum Satisfiability Problem

- The goal of the Maximum Satisfiability (MaxSAT)^{17,42} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into one AND-clause.



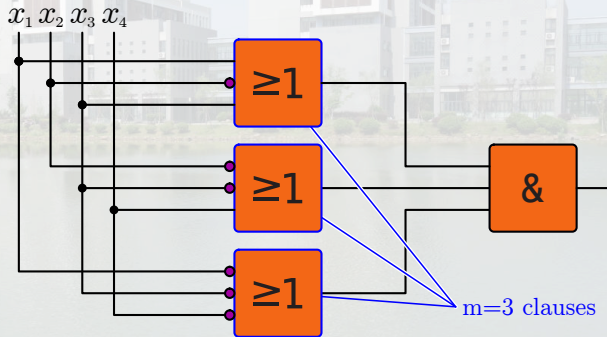
Example: Maximum Satisfiability Problem

- The goal of the Maximum Satisfiability (MaxSAT)^{17,42} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or **negated** in m OR-clauses, whose results flow into one AND-clause.



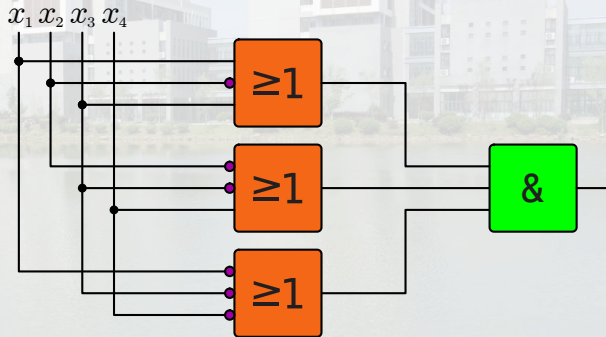
Example: Maximum Satisfiability Problem

- The goal of the Maximum Satisfiability (MaxSAT)^{17,42} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into one AND-clause.



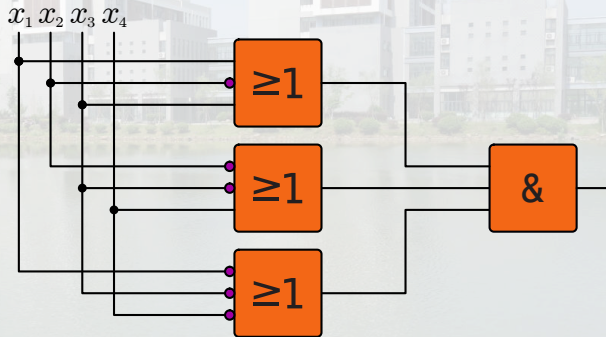
Example: Maximum Satisfiability Problem

- The goal of the Maximum Satisfiability (MaxSAT)^{17,42} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into **one AND-clause**.



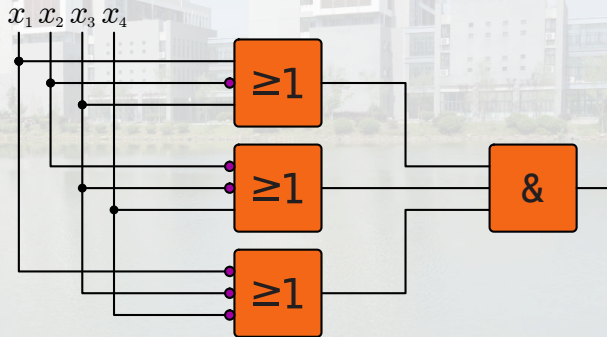
Example: Maximum Satisfiability Problem

- The goal of the Maximum Satisfiability (MaxSAT)^{17,42} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into one AND-clause.
- $\mathbb{X}$ is the set of all possible bit strings of length n .



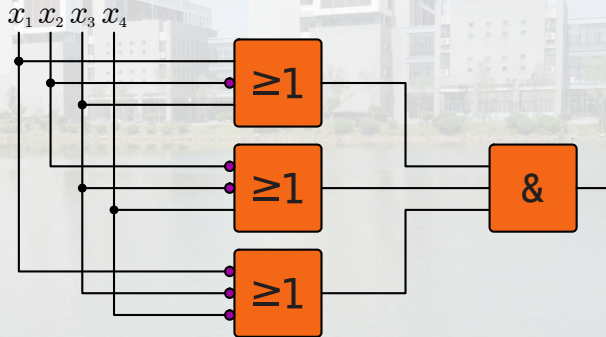
Example: Maximum Satisfiability Problem

- The goal of the Maximum Satisfiability (MaxSAT)^{17,42} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into one AND-clause.
- $\mathbb{X}$ is the set of all possible bit strings of length n .
- The objective function $f : \mathbb{X} \mapsto \mathbb{R}$ is the number of unsatisfied OR-clauses.



Example: Maximum Satisfiability Problem

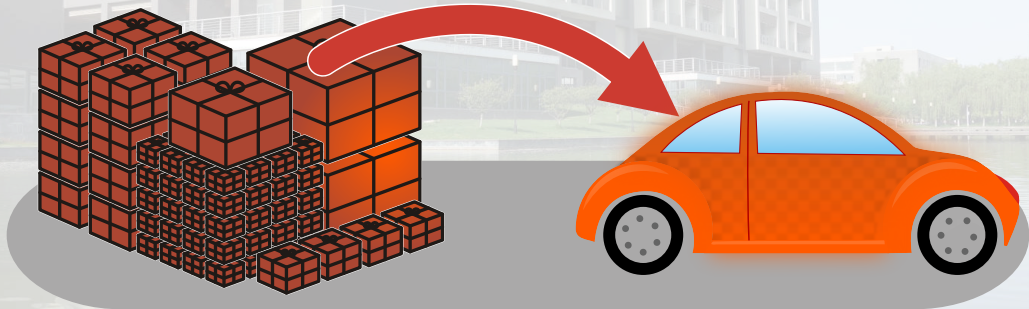
- The goal of the Maximum Satisfiability (MaxSAT)^{17,42} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into one AND-clause.
- $\mathbb{X}$ is the set of all possible bit strings of length n .
- The objective function $f : \mathbb{X} \mapsto \mathbb{R}$ is the number of unsatisfied OR-clauses.
- If the problem can be solved, then the optimum $x^* \in \mathbb{X}$ has $f(x^*) = 0$, i.e., all clauses satisfied, i.e., $F(x^*) = \text{True}$.



Example: Bin Packing Problem



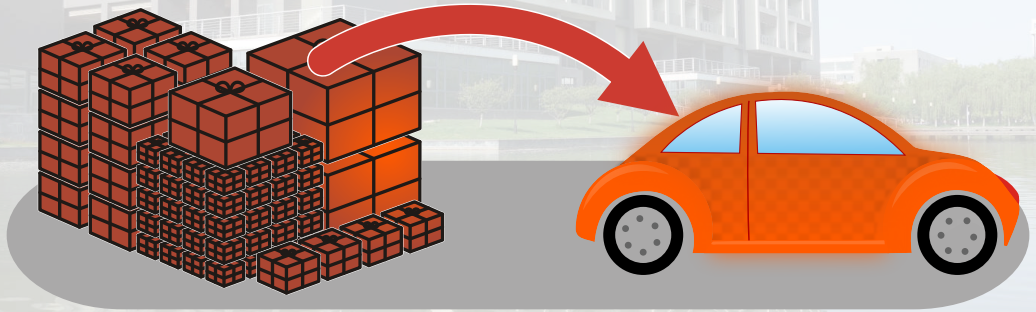
- The goal of the Bin Packing Problem is to pack n objects, each having a specific size, into as few bins (also of a given size) as possible^{117,118}.



Example: Bin Packing Problem



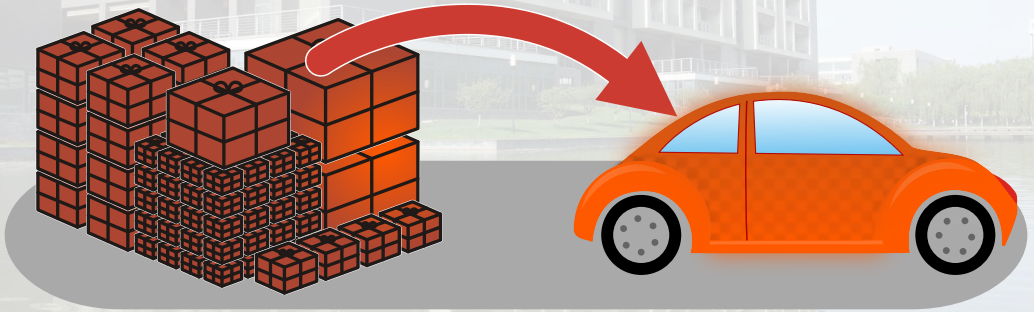
- The goal of the Bin Packing Problem is to pack n objects, each having a specific size, into as few bins (also of a given size) as possible^{117,118}.
- The $\mathbb{X}$ comprises all possible packing orders of the n objects.



Example: Bin Packing Problem



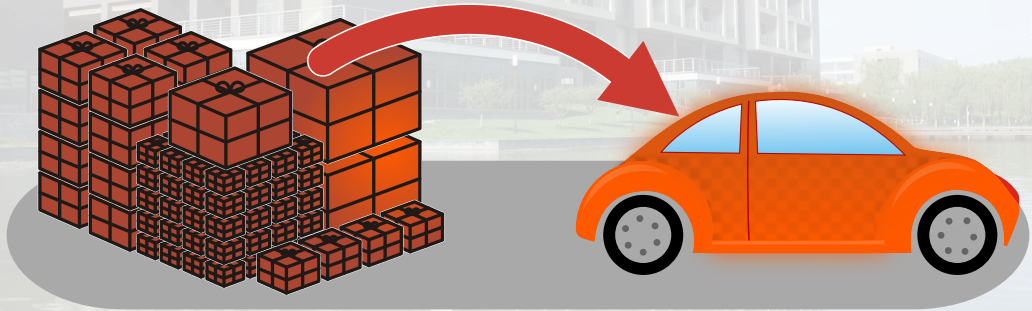
- The goal of the Bin Packing Problem is to pack n objects, each having a specific size, into as few bins (also of a given size) as possible^{117,118}.
- The $\mathbb{X}$ comprises all possible packing orders of the n objects.
- The objective function f is the number of bins needed by a given packing order.



Example: Bin Packing Problem



- The goal of the Bin Packing Problem is to pack n objects, each having a specific size, into as few bins (also of a given size) as possible^{117,118}.
- The $\mathbb{X}$ comprises all possible packing orders of the n objects.
- The objective function f is the number of bins needed by a given packing order.
- The optimum x^* is the packing order requiring the fewest bins.



Optimization is Hard



- Finding the globally optimal solution x^* from the set of all possible solutions $\mathbb{X}$ is often an $\mathcal{NP}$ -hard problem.



Optimization is Hard



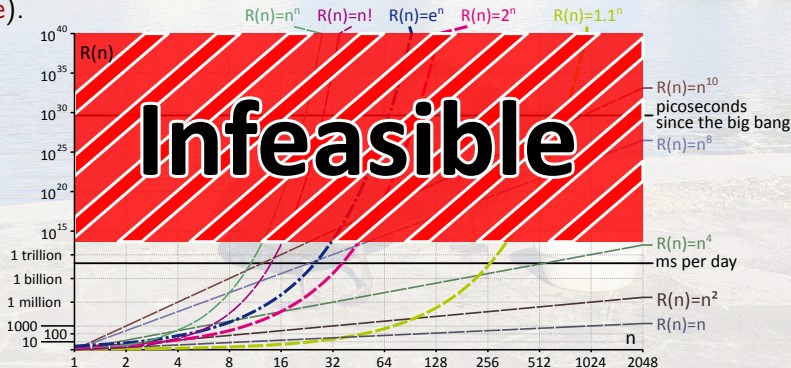
- Finding the globally optimal solution x^* from the set of all possible solutions $\mathbb{X}$ is often an $\mathcal{NP}$ -hard problem.
- Currently, there is no algorithm that can **guarantee** to find the optimal solution of **every instance** of a given $\mathcal{NP}$ -hard problem in a runtime that is not longer than polynomial in the size of the problem (i.e., existing algorithms may need exponential runtime in the **worst case**).



Optimization is Hard



- Finding the globally optimal solution x^* from the set of all possible solutions $\mathbb{X}$ is often an $\mathcal{NP}$ -hard problem.
- Currently, there is no algorithm that can **guarantee** to find the optimal solution of **every instance** of a given $\mathcal{NP}$ -hard problem in a runtime that is not longer than polynomial in the size of the problem (i.e., existing algorithms may need exponential runtime in the **worst case**).



Optimization is Hard



- Finding the globally optimal solution x^* from the set of all possible solutions $\mathbb{X}$ is often an $\mathcal{NP}$ -hard problem.
- Currently, there is no algorithm that can **guarantee** to find the optimal solution of **every instance** of a given $\mathcal{NP}$ -hard problem in a runtime that is not longer than polynomial in the size of the problem (i.e., existing algorithms may need exponential runtime in the **worst case**).
- In other words, if we want to guarantee to find the best possible solution x^* for all possible instances of a problem, we often cannot really be much faster than testing all possible candidate solutions $x \in \mathbb{X}$ in the **worst case**.



Metaheuristic Optimization



Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.

Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.

Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.

Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.

Begin with a set $S_0 \subset \mathbb{X}$ of one or multiple randomly sampled solutions

Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



Begin with a set $S_0 \subset \mathbb{X}$ of one or multiple randomly sampled solutions

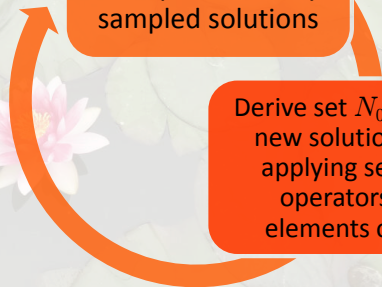
Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.

Begin with a set $S_0 \subset \mathbb{X}$ of one or multiple randomly sampled solutions

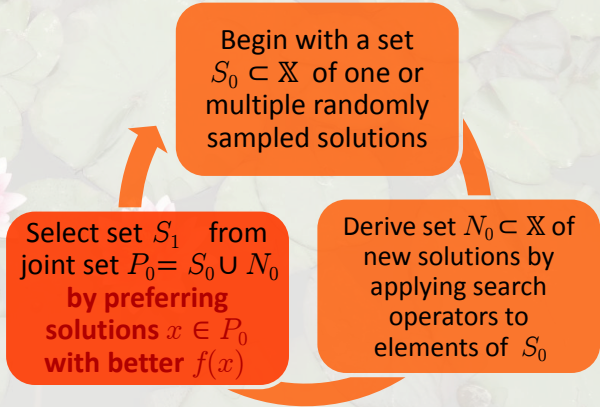
Derive set $N_0 \subset \mathbb{X}$ of new solutions by applying search operators to elements of S_0



Metaheuristics



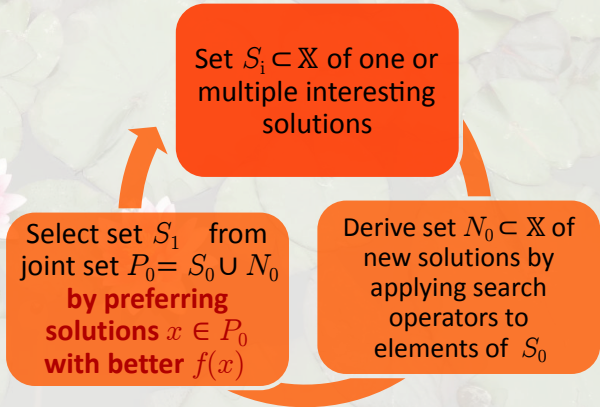
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



Metaheuristics



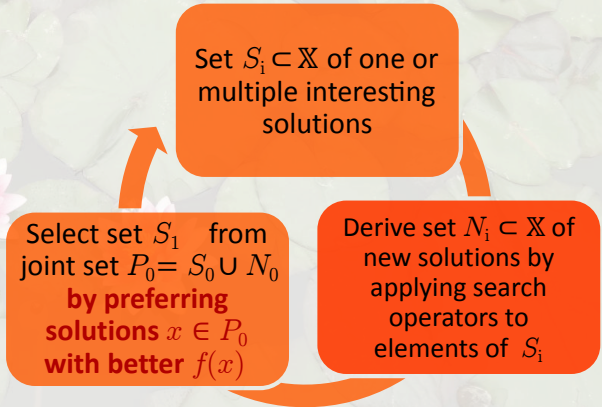
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



Metaheuristics



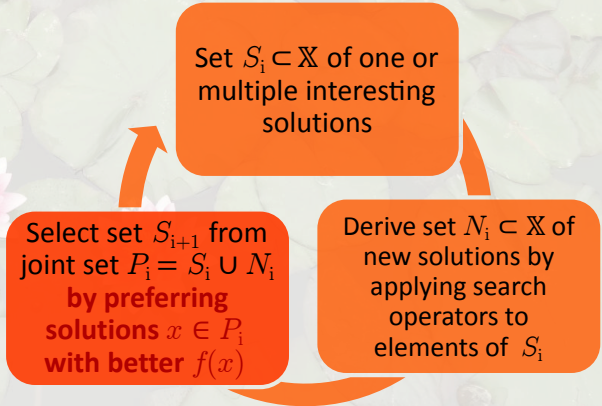
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



RLS and the $(1 + 1)$ EA

- stochastic local search (SLS) with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.



RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{12,25}.

procedure $(1 + 1)$ EA($f : \mathbb{X} \mapsto \mathbb{R}$)

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{12,25}.

procedure $(1 + 1)$ EA($f : \mathbb{X} \mapsto \mathbb{R}$)
 randomly sample x_c from $\mathbb{X}$;

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{12,25}.

procedure $(1 + 1)$ EA($f : \mathbb{X} \mapsto \mathbb{R}$)
 randomly sample x_c from $\mathbb{X}$; $y_c \leftarrow f(x_c)$;

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{12,25}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do
```

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{12,25}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;
```

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{12,25}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;
```

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{12,25}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then
```

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{12,25}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;
```

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{12,25}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{12,25}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

Simulated Annealing

- Simulated Annealing (SA)⁵⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.



Simulated Annealing

- Simulated Annealing (SA)⁵⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- Let's start with RLS/(1 + 1) EA and modify it to become SA.



```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$   
  
    if  $y_n \leq y_c$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ 
```

Simulated Annealing

- Simulated Annealing (SA)⁵⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathcal{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$   
  
    if  $\mathcal{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ 
```



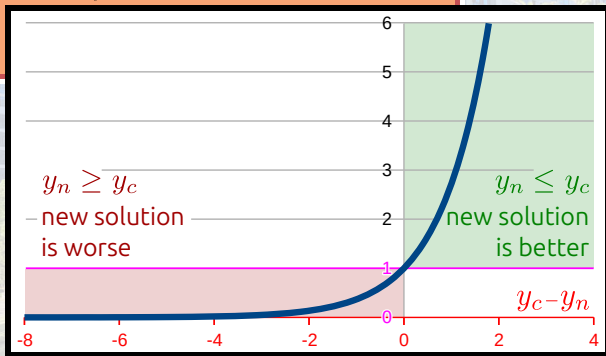
Simulated Annealing

- Simulated Annealing (SA)⁵⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;
```

```
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;
```

```
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then
```



Simulated Annealing

- Simulated Annealing (SA)⁵⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.

procedure SA($f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$)

randomly sample x_c from $\mathbb{X}$; $y_c \leftarrow f(x_c)$;

while $\neg$ terminate **do**

$x_n \leftarrow \text{move}(x_c)$; $y_n \leftarrow f(x_n)$;

if $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$ **then** $\triangleright$ always true if $y_n \leq y_c$

$x_c \leftarrow x_n$; $y_c \leftarrow y_n$;



Simulated Annealing



- Simulated Annealing (SA)⁵⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathcal{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.

procedure SA($f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$)

randomly sample x_c from $\mathbb{X}$; $y_c \leftarrow f(x_c)$;

while $\neg$ terminate **do**

$x_n \leftarrow \text{move}(x_c)$; $y_n \leftarrow f(x_n)$;

if $\mathcal{R}_0^1 < e^{\frac{y_c - y_n}{T}}$ **then** $\triangleright$ always true if $y_n \leq y_c$
 $x_c \leftarrow x_n$; $y_c \leftarrow y_n$;

Simulated Annealing



- Simulated Annealing (SA)⁵⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameter T_0 .

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$   
  
     $T \leftarrow T_0$  ;  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then  $\triangleright$  always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ 
```

Simulated Annealing



- Simulated Annealing (SA)⁵⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathcal{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameter T_0 .

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$   
  
     $T \leftarrow T_0$  ;  
    if  $\mathcal{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ 
```

Simulated Annealing



- Simulated Annealing (SA)⁵⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameter T_0 .

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $\tau \leftarrow \tau + 1$ ;  
     $T \leftarrow T_0$  ;  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```

Simulated Annealing



- Simulated Annealing (SA)⁵⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameters T_0 and ε .

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $\tau \leftarrow \tau + 1$ ;  
     $T \leftarrow T_0(1 - \varepsilon)^{\tau-1}$ ; ▷  $T$  decreases over time  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```

Simulated Annealing



- Simulated Annealing (SA)⁵⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameters T_0 and ε .
- SA also remembers best-so-far solution x_B and its objective value y_B , because it could get lost.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ; ▷ preserve best!  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $\tau \leftarrow \tau + 1$ ;  
     $T \leftarrow T_0(1 - \varepsilon)^{\tau-1}$ ; ▷  $T$  decreases over time  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```

Simulated Annealing



- Simulated Annealing (SA)⁵⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameters T_0 and ε .
- SA also remembers best-so-far solution x_B and its objective value y_B , because it could get lost.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ; ▷ preserve best!
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter
  while  $\neg$  terminate do
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;
     $\tau \leftarrow \tau + 1$ ;
     $T \leftarrow T_0(1 - \varepsilon)^{\tau-1}$ ; ▷  $T$  decreases over time
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$ 
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
      if  $y_c < y_B$  then  $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;
```

Simulated Annealing



- Simulated Annealing (SA)⁵⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameters T_0 and ε .
- SA also remembers best-so-far solution x_B and its objective value y_B , because it could get lost.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ; ▷ preserve best!  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $\tau \leftarrow \tau + 1$ ;  
     $T \leftarrow T_0(1 - \varepsilon)^{\tau-1}$ ; ▷  $T$  decreases over time  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
      if  $y_c < y_B$  then  $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;  
  return  $x_B, y_B$ 
```

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary operator.

procedure SGA($f : \mathbb{X} \mapsto \mathbb{R}^+$, ps , cr)

▷ for maximization!

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary operator.

procedure SGA($f : \mathbb{X} \mapsto \mathbb{R}^+$, ps , cr)

▷ for maximization!

for $j \in 1 \dots ps$ do

▷ random initial population

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary operator.

procedure SGA($f : \mathbb{X} \mapsto \mathbb{R}^+$, ps , cr)

▷ for maximization!

for $j \in 1 \dots ps$ do ▷ random initial population
randomly sample $S_0[j].x$ from $\mathbb{X}$;

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary operator.

procedure SGA($f : \mathbb{X} \mapsto \mathbb{R}^+$, ps , cr)

▷ for maximization!

for $j \in 1 \dots ps$ do ▷ random initial population
randomly sample $S_0[j].x$ from $\mathbb{X}$; $S_0[j].y \leftarrow f(S_0[j].x)$;

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization**^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary operator.

procedure SGA($f : \mathbb{X} \mapsto \mathbb{R}^+$, ps , cr)

▷ for maximization!

for $j \in 1 \dots ps$ do ▷ random initial population

randomly sample $S_0[j].x$ from $\mathbb{X}$; $S_0[j].y \leftarrow f(S_0[j].x)$;

for $i \in 0 \dots \infty$ do

▷ iterate “generations”

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary operator.

```
procedure SGA( $f : \mathbb{X} \mapsto \mathbb{R}^+$ ,  $ps$ ,  $cr$ ) ▷ for maximization!  
  
  for  $j \in 1 \dots ps$  do ▷ random initial population  
    randomly sample  $S_0[j].x$  from  $\mathbb{X}$ ;  $S_0[j].y \leftarrow f(S_0[j].x)$ ;  
  
  for  $i \in 0 \dots \infty$  do ▷ iterate "generations"  
    for  $j \in 1 \dots ps$  do ▷ new pop
```

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary operator.

```
procedure SGA( $f : \mathbb{X} \mapsto \mathbb{R}^+$ ,  $ps$ ,  $cr$ ) ▷ for maximization!  
  
  for  $j \in 1 \dots ps$  do ▷ random initial population  
    randomly sample  $S_0[j].x$  from  $\mathbb{X}$ ;  $S_0[j].y \leftarrow f(S_0[j].x)$ ;  
  
  for  $i \in 0 \dots \infty$  do ▷ iterate "generations"  
    for  $j \in 1 \dots ps$  do ▷ new pop. via mutation  
       $N_i[j].x \leftarrow \text{move}(S_i[[\mathfrak{R}_i^{ps}]].x)$ ;
```

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary and binary operator (with crossover rate cr).

```
procedure SGA( $f : \mathbb{X} \mapsto \mathbb{R}^+$ ,  $ps$ ,  $cr$ ) ▷ for maximization!  
  
  for  $j \in 1 \dots ps$  do ▷ random initial population  
    randomly sample  $S_0[j].x$  from  $\mathbb{X}$ ;  $S_0[j].y \leftarrow f(S_0[j].x)$ ;  
  
  for  $i \in 0 \dots \infty$  do ▷ iterate "generations"  
    for  $j \in 1 \dots ps$  do ▷ new pop. via mutation and crossover  
      if  $\mathfrak{R}_0^1 < cr$  then  $N_i[j].x \leftarrow \text{binary}(S_i[[\mathfrak{R}_i^{ps}]].x, S_i[[\mathfrak{R}_i^{ps}]].x)$ ;  
      else  $N_i[j].x \leftarrow \text{move}(S_i[[\mathfrak{R}_i^{ps}]].x)$ ;
```

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary and binary operator (with crossover rate cr).

```
procedure SGA( $f : \mathbb{X} \mapsto \mathbb{R}^+$ ,  $ps$ ,  $cr$ ) ▷ for maximization!  
  
  for  $j \in 1 \dots ps$  do ▷ random initial population  
    randomly sample  $S_0[j].x$  from  $\mathbb{X}$ ;  $S_0[j].y \leftarrow f(S_0[j].x)$ ;  
  
  for  $i \in 0 \dots \infty$  do ▷ iterate "generations"  
    for  $j \in 1 \dots ps$  do ▷ new pop. via mutation and crossover  
      if  $\mathfrak{R}_0^1 < cr$  then  $N_i[j].x \leftarrow \text{binary}(S_i[\lceil \mathfrak{R}_i^{ps} \rceil].x, S_i[\lceil \mathfrak{R}_i^{ps} \rceil].x)$ ;  
      else  $N_i[j].x \leftarrow \text{move}(S_i[\lceil \mathfrak{R}_i^{ps} \rceil].x)$ ;  
       $N_i[j].y \leftarrow f(N_i[j].x)$ ;
```

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary and binary operator (with crossover rate cr).

```
procedure SGA( $f : \mathbb{X} \mapsto \mathbb{R}^+$ ,  $ps$ ,  $cr$ ) ▷ for maximization!  
  
  for  $j \in 1 \dots ps$  do ▷ random initial population  
    randomly sample  $S_0[j].x$  from  $\mathbb{X}$ ;  $S_0[j].y \leftarrow f(S_0[j].x)$ ;  
  
  for  $i \in 0 \dots \infty$  do ▷ iterate "generations"  
    for  $j \in 1 \dots ps$  do ▷ new pop. via mutation and crossover  
      if  $\mathfrak{R}_0^1 < cr$  then  $N_i[j].x \leftarrow \text{binary}(S_i[[\mathfrak{R}_i^{ps}]].x, S_i[[\mathfrak{R}_i^{ps}]].x)$ ;  
      else  $N_i[j].x \leftarrow \text{move}(S_i[[\mathfrak{R}_i^{ps}]].x)$ ;  
       $N_i[j].y \leftarrow f(N_i[j].x)$ ;  
  
     $S_{i+1} \leftarrow \text{Roulette Wheel: select } ps \text{ records from } P_i = S_i \cup N_i$   
      such that, for each of the  $ps$  slots, the probability  
      of  $P_i[j]$  to be chosen is proportional to  $P_i[j].y$ .
```

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary and binary operator (with crossover rate cr).

```
procedure SGA( $f : \mathbb{X} \mapsto \mathbb{R}^+$ ,  $ps$ ,  $cr$ )  
     $x_B \leftarrow \emptyset$ ;  $y_B \leftarrow -\infty$ ; ▷ for maximization!  
    for  $j \in 1 \dots ps$  do ▷ best-so-far solution  
        randomly sample  $S_0[j].x$  from  $\mathbb{X}$ ;  $S_0[j].y \leftarrow f(S_0[j].x)$ ; ▷ random initial population  
  
    for  $i \in 0 \dots \infty$  do ▷ iterate "generations"  
        for  $j \in 1 \dots ps$  do ▷ new pop. via mutation and crossover  
            if  $\mathfrak{R}_0^1 < cr$  then  $N_i[j].x \leftarrow \text{binary}(S_i[[\mathfrak{R}_i^{ps}]].x, S_i[[\mathfrak{R}_i^{ps}]].x)$ ;  
            else  $N_i[j].x \leftarrow \text{move}(S_i[[\mathfrak{R}_i^{ps}]].x)$ ;  
             $N_i[j].y \leftarrow f(N_i[j].x)$ ;  
  
     $S_{i+1} \leftarrow \text{Roulette Wheel: select } ps \text{ records from } P_i = S_i \cup N_i$   
    such that, for each of the  $ps$  slots, the probability  
    of  $P_i[j]$  to be chosen is proportional to  $P_i[j].y$ .
```

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary and binary operator (with crossover rate cr).

```
procedure SGA( $f : \mathbb{X} \mapsto \mathbb{R}^+$ ,  $ps$ ,  $cr$ )  
     $x_B \leftarrow \emptyset$ ;  $y_B \leftarrow -\infty$ ; ▷ best-so-far solution  
    for  $j \in 1 \dots ps$  do ▷ random initial population  
        randomly sample  $S_0[j].x$  from  $\mathbb{X}$ ;  $S_0[j].y \leftarrow f(S_0[j].x)$ ;  
        if  $S_0[j].y > y_B$  then  $x_B \leftarrow S_0[j].x$ ;  $y_B \leftarrow S_0[j].y$ ;  
    for  $i \in 0 \dots \infty$  do ▷ iterate “generations”  
        for  $j \in 1 \dots ps$  do ▷ new pop. via mutation and crossover  
            if  $\mathfrak{R}_0^i < cr$  then  $N_i[j].x \leftarrow \text{binary}(S_i[[\mathfrak{R}_1^{ps}]].x, S_i[[\mathfrak{R}_1^{ps}]].x)$ ;  
            else  $N_i[j].x \leftarrow \text{move}(S_i[[\mathfrak{R}_1^{ps}]].x)$ ;  
             $N_i[j].y \leftarrow f(N_i[j].x)$ ;  
  
     $S_{i+1} \leftarrow \text{Roulette Wheel: select } ps \text{ records from } P_i = S_i \cup N_i$   
    such that, for each of the  $ps$  slots, the probability  
    of  $P_i[j]$  to be chosen is proportional to  $P_i[j].y$ .
```

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary and binary operator (with crossover rate cr).

```
procedure SGA( $f : \mathbb{X} \mapsto \mathbb{R}^+$ ,  $ps$ ,  $cr$ )  
     $x_B \leftarrow \emptyset$ ;  $y_B \leftarrow -\infty$ ; ▷ for maximization!  
    for  $j \in 1 \dots ps$  do ▷ random initial population  
        randomly sample  $S_0[j].x$  from  $\mathbb{X}$ ;  $S_0[j].y \leftarrow f(S_0[j].x)$ ;  
        if  $S_0[j].y > y_B$  then  $x_B \leftarrow S_0[j].x$ ;  $y_B \leftarrow S_0[j].y$ ;  
    for  $i \in 0 \dots \infty$  do ▷ iterate "generations"  
        for  $j \in 1 \dots ps$  do ▷ new pop. via mutation and crossover  
            if  $\mathfrak{R}_0^i < cr$  then  $N_i[j].x \leftarrow \text{binary}(S_i[\lceil \mathfrak{R}_i^{ps} \rceil].x, S_i[\lceil \mathfrak{R}_i^{ps} \rceil].x)$ ;  
            else  $N_i[j].x \leftarrow \text{move}(S_i[\lceil \mathfrak{R}_i^{ps} \rceil].x)$ ;  
             $N_i[j].y \leftarrow f(N_i[j].x)$ ;  
            if  $N_i[j].y > y_B$  then  $x_B \leftarrow N_i[j].x$ ;  $y_B \leftarrow N_i[j].y$ ;  
     $S_{i+1} \leftarrow \text{Roulette Wheel: select } ps \text{ records from } P_i = S_i \cup N_i$   
        such that, for each of the  $ps$  slots, the probability  
        of  $P_i[j]$  to be chosen is proportional to  $P_i[j].y$ .
```

Standard Genetic Algorithm with Roulette Wheel Selection



- The Standard Genetic Algorithm (SGA) with Fitness Proportionate Selection (Roulette Wheel) is for **maximization** ^{5,21,38,66,67,95}.
- It uses a population of size ps as well as a unary and binary operator (with crossover rate cr).

```
procedure SGA( $f : \mathbb{X} \mapsto \mathbb{R}^+$ ,  $ps$ ,  $cr$ )  
     $x_B \leftarrow \emptyset$ ;  $y_B \leftarrow -\infty$ ; ▷ for maximization!  
    for  $j \in 1 \dots ps$  do ▷ random initial population  
        randomly sample  $S_0[j].x$  from  $\mathbb{X}$ ;  $S_0[j].y \leftarrow f(S_0[j].x)$ ;  
        if  $S_0[j].y > y_B$  then  $x_B \leftarrow S_0[j].x$ ;  $y_B \leftarrow S_0[j].y$ ;  
    for  $i \in 0 \dots \infty$  do ▷ iterate "generations"  
        for  $j \in 1 \dots ps$  do ▷ new pop. via mutation and crossover  
            if  $\mathfrak{R}_0^i < cr$  then  $N_i[j].x \leftarrow \text{binary}(S_i[\lceil \mathfrak{R}_i^{ps} \rceil].x, S_i[\lceil \mathfrak{R}_i^{ps} \rceil].x)$ ;  
            else  $N_i[j].x \leftarrow \text{move}(S_i[\lceil \mathfrak{R}_i^{ps} \rceil].x)$ ;  
             $N_i[j].y \leftarrow f(N_i[j].x)$ ;  
            if  $N_i[j].y > y_B$  then  $x_B \leftarrow N_i[j].x$ ;  $y_B \leftarrow N_i[j].y$ ;  
         $S_{i+1} \leftarrow \text{Roulette Wheel: select } ps \text{ records from } P_i = S_i \cup N_i$   
            such that, for each of the  $ps$  slots, the probability  
            of  $P_i[j]$  to be chosen is proportional to  $P_i[j].y$ .  
    return  $x_B$ ,  $y_B$ 
```

Metaheuristic Optimization



- What we have learned is that different metaheuristics realize the trial-and-error scheme differently.

Metaheuristic Optimization



- What we have learned is that different metaheuristics realize the trial-and-error scheme differently.
- During their selection steps, they all prefer better solutions over worse ones.

Metaheuristic Optimization



- What we have learned is that different metaheuristics realize the trial-and-error scheme differently.
- During their selection steps, they all prefer better solutions over worse ones.
- If they would always and only accept the better solutions, they could get trapped in local optima.

Metaheuristic Optimization



- What we have learned is that different metaheuristics realize the trial-and-error scheme differently.
- During their selection steps, they all prefer better solutions over worse ones.
- If they would always and only accept the better solutions, they could get trapped in local optima.
- So some of them sometimes accept worse solutions, but the probability to choose a better solution is always higher in average.

Metaheuristic Optimization



- What we have learned is that different metaheuristics realize the trial-and-error scheme differently.
- During their selection steps, they all prefer better solutions over worse ones.
- If they would always and only accept the better solutions, they could get trapped in local optima.
- So some of them sometimes accept worse solutions, but the probability to choose a better solution is always higher in average.
- This is the most fundamental concept of metaheuristic optimization:

If you keep good solutions and modify them, you are likely to get better solutions.

Metaheuristic Optimization



- What we have learned is that different metaheuristics realize the trial-and-error scheme differently.
- During their selection steps, they all prefer better solutions over worse ones.
- If they would always and only accept the better solutions, they could get trapped in local optima.
- So some of them sometimes accept worse solutions, but the probability to choose a better solution is always higher in average.
- This is the most fundamental concept of metaheuristic optimization:

If you keep good solutions and modify them, you are likely to get better solutions.

If you keep accepting better and better solutions, you will get really good solutions eventually.



Invariance Properties



Invariance Properties



- Research in optimization, Machine Learning (ML), and Artificial Intelligence (AI) often use simple problems to test and benchmark algorithms.

Invariance Properties



- Research in optimization, Machine Learning (ML), and Artificial Intelligence (AI) often use simple problems to test and benchmark algorithms.
- These allow for many experiments in a short time.

Invariance Properties



- Research in optimization, Machine Learning (ML), and Artificial Intelligence (AI) often use simple problems to test and benchmark algorithms.
- These allow for many experiments in a short time.
- We often know the optimal solutions or at lower bounds for the optimal objective values.

Invariance Properties



- Research in optimization, Machine Learning (ML), and Artificial Intelligence (AI) often use simple problems to test and benchmark algorithms.
- These allow for many experiments in a short time.
- We often know the optimal solutions or at lower bounds for the optimal objective values.
- We can understand the results well.

Invariance Properties



- Research in optimization, Machine Learning (ML), and Artificial Intelligence (AI) often use simple problems to test and benchmark algorithms.
- These allow for many experiments in a short time.
- We often know the optimal solutions or at lower bounds for the optimal objective values.
- We can understand the results well.
- What we want is that algorithms perform similar to our benchmarking results also on actual, real-world problems.

Invariance Properties



- Research in optimization, Machine Learning (ML), and Artificial Intelligence (AI) often use simple problems to test and benchmark algorithms.
- These allow for many experiments in a short time.
- We often know the optimal solutions or at lower bounds for the optimal objective values.
- We can understand the results well.
- What we want is that algorithms perform similar to our benchmarking results also on actual, real-world problems.
- **We want invariance properties.** ^{41,49,71}

Acceptance Criteria



- We have now three example algorithms: $(1 + 1)$ EA, SA, and SGA.

Acceptance Criteria



- We have now three example algorithms: $(1 + 1)$ EA, SA, and SGA.
- Assume that they all work on a population of two solutions, where the new solution be x_n and the old/parent solution be x_c .

Acceptance Criteria



- We have now three example algorithms: $(1 + 1)$ EA, SA, and SGA.
- Assume that they all work on a population of two solutions, where the new solution be x_n and the old/parent solution be x_c and we look at the probability P to accept x_n .

Acceptance Criteria



- We have now three example algorithms: $(1 + 1)$ EA, SA, and SGA.
- Assume that they all work on a population of two solutions, where the new solution be x_n and the old/parent solution be x_c and we look at the probability P to accept x_n .
- For the $(1 + 1)$ EA, $P = 1$ if $f(x_n) \leq f(x_c)$ and 0 otherwise.

Acceptance Criteria



- We have now three example algorithms: $(1 + 1)$ EA, SA, and SGA.
- Assume that they all work on a population of two solutions, where the new solution be x_n and the old/parent solution be x_c and we look at the probability P to accept x_n .
- For the $(1 + 1)$ EA, $P = 1$ if $f(x_n) \leq f(x_c)$ and 0 otherwise.
- In the SA, P depends on the difference $\Delta = f(x_c) - f(x_n)$ and is 1 if x_n is better than x_c .

Acceptance Criteria



- We have now three example algorithms: $(1 + 1)$ EA, SA, and SGA.
- Assume that they all work on a population of two solutions, where the new solution be x_n and the old/parent solution be x_c and we look at the probability P to accept x_n .
- For the $(1 + 1)$ EA, $P = 1$ if $f(x_n) \leq f(x_c)$ and 0 otherwise.
- In the SA, P depends on the difference $\Delta = f(x_c) - f(x_n)$ and is 1 if x_n is better than x_c .
- In an SGA for maximization, P is $f(x_n)/(f(x_c) + f(x_n))$.

Acceptance Criteria



- We have now three example algorithms: $(1 + 1)$ EA, SA, and SGA.
- Assume that they all work on a population of two solutions, where the new solution be x_n and the old/parent solution be x_c and we look at the probability P to accept x_n .
- For the $(1 + 1)$ EA, $P = 1$ if $f(x_n) \leq f(x_c)$ and 0 otherwise.
- In the SA, P depends on the difference $\Delta = f(x_c) - f(x_n)$ and is 1 if x_n is better than x_c .
- In an SGA for maximization, P is $f(x_n)/(f(x_c) + f(x_n))$. If we wanted to do minimization, maybe we could use $1 - f(x_n)/(f(x_c) + f(x_n))$ instead.

Acceptance Criteria



- We have now three example algorithms: $(1 + 1)$ EA, SA, and SGA.
- Assume that they all work on a population of two solutions, where the new solution be x_n and the old/parent solution be x_c and we look at the probability P to accept x_n .
- For the $(1 + 1)$ EA, $P = 1$ if $f(x_n) \leq f(x_c)$ and 0 otherwise.
- In the SA, P depends on the difference $\Delta = f(x_c) - f(x_n)$ and is 1 if x_n is better than x_c .
- In an SGA for maximization, P is $f(x_n)/(f(x_c) + f(x_n))$. If we wanted to do minimization, maybe we could use $1 - f(x_n)/(f(x_c) + f(x_n))$ instead.
- $(1 + 1)$ EA and SA always accept x_n if it is better than x_c , while in the SGA, a better x_n at least has a $P > 0.5$.

Acceptance Criteria

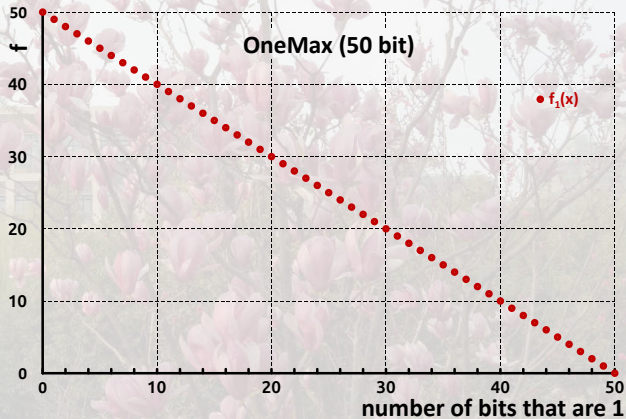


- We have now three example algorithms: $(1 + 1)$ EA, SA, and SGA.
- Assume that they all work on a population of two solutions, where the new solution be x_n and the old/parent solution be x_c and we look at the probability P to accept x_n .
- For the $(1 + 1)$ EA, $P = 1$ if $f(x_n) \leq f(x_c)$ and 0 otherwise.
- In the SA, P depends on the difference $\Delta = f(x_c) - f(x_n)$ and is 1 if x_n is better than x_c .
- In an SGA for maximization, P is $f(x_n)/(f(x_c) + f(x_n))$. If we wanted to do minimization, maybe we could use $1 - f(x_n)/(f(x_c) + f(x_n))$ instead.
- $(1 + 1)$ EA and SA always accept x_n if it is better than x_c , while in the SGA, a better x_n at least has a $P > 0.5$.
- Let's keep these things in mind.

Invariance Properties: OneMax Example



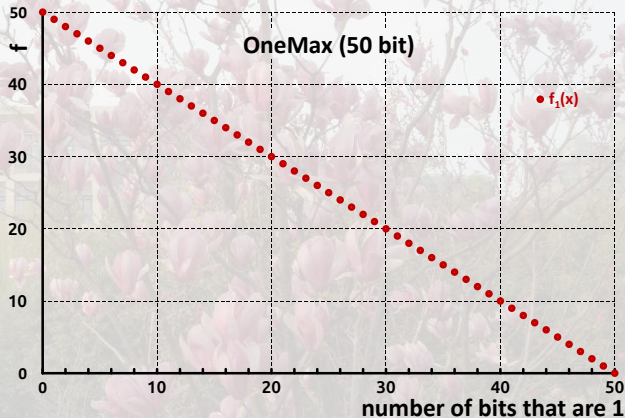
- OneMax is the simplest benchmark problem in discrete optimization^{26,68,116}.



Invariance Properties: OneMax Example



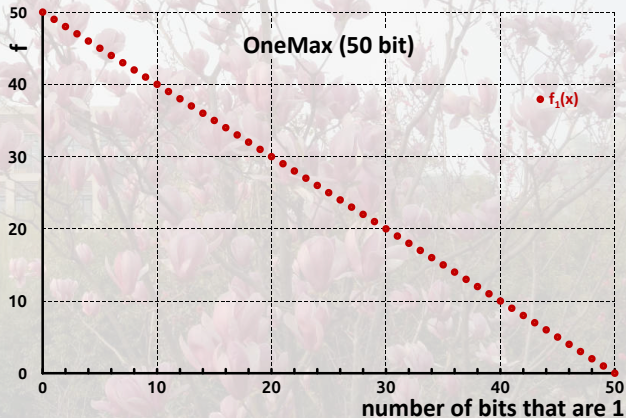
- OneMax is the simplest benchmark problem in discrete optimization^{26,68,116}.
- It is defined over $\mathbb{X} = \{0, 1\}^n$, i.e., the bit strings of length n .



Invariance Properties: OneMax Example



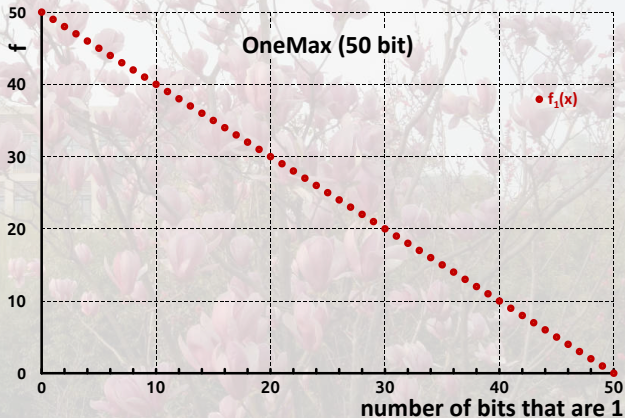
- OneMax is the simplest benchmark problem in discrete optimization^{26,68,116}.
- It is defined over $\mathbb{X} = \{0, 1\}^n$, i.e., the bit strings of length n .
- Goal: "Find the bit string of all 1s!"



Invariance Properties: OneMax Example



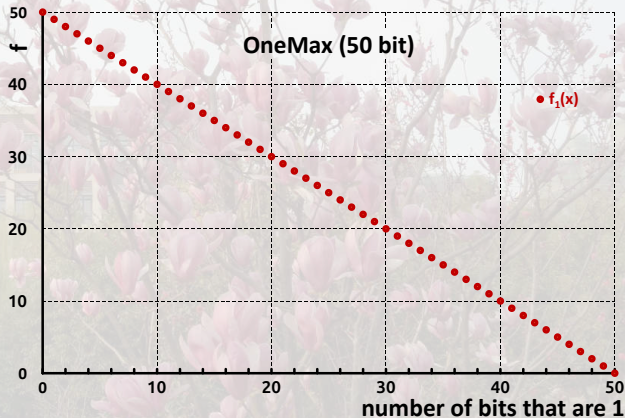
- OneMax is the simplest benchmark problem in discrete optimization^{26,68,116}.
- It is defined over $\mathbb{X} = \{0, 1\}^n$, i.e., the bit strings of length n .
- Goal: “Find the bit string of all 1s!”
- “Maximize the number of 1s!”



Invariance Properties: OneMax Example



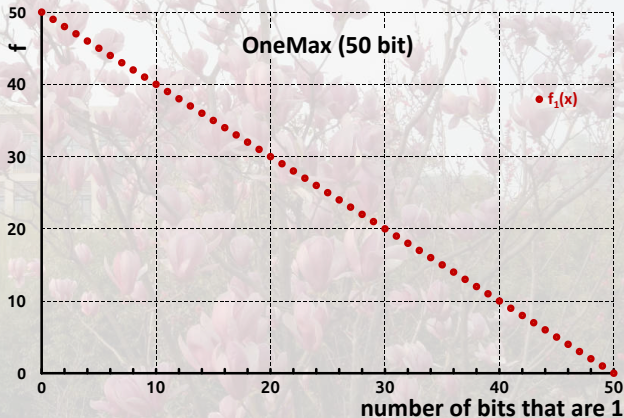
- OneMax is the simplest benchmark problem in discrete optimization^{26,68,116}.
- It is defined over $\mathbb{X} = \{0, 1\}^n$, i.e., the bit strings of length n .
- Goal: “Find the bit string of all 1s!”
- “Maximize the number of 1s!”
- $f_1(x) = n - \sum x$ (for minimization).



Invariance Properties: OneMax Example



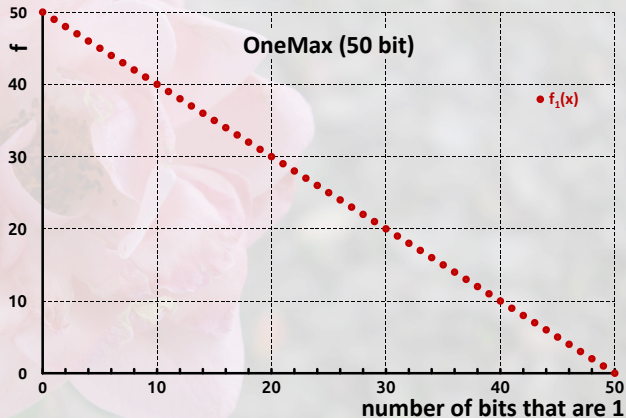
- OneMax is the simplest benchmark problem in discrete optimization^{26,68,116}.
- It is defined over $\mathbb{X} = \{0, 1\}^n$, i.e., the bit strings of length n .
- Goal: “Find the bit string of all 1s!”
- “Maximize the number of 1s!”
- $f_1(x) = n - \sum x$ (for minimization).
- Every reasonable algorithm can solve this problem.



Invariance Properties: OneMax Example – Translation



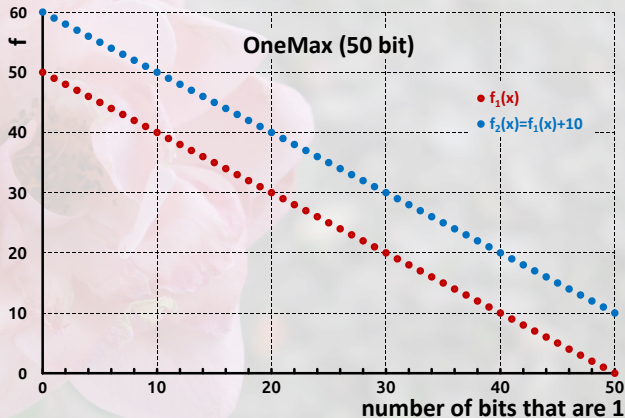
- Now I create a slightly modified variant of this problem.



Invariance Properties: OneMax Example – Translation



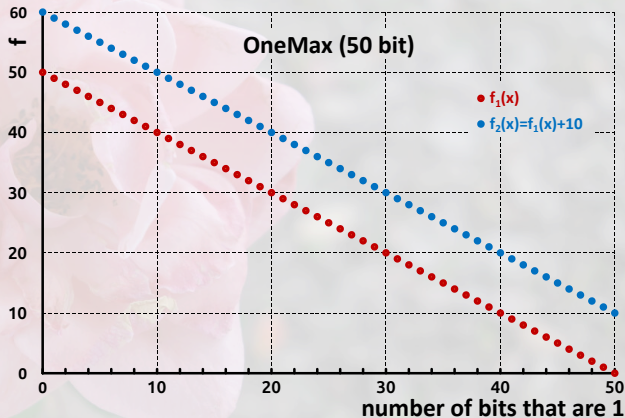
- Now I create a slightly modified variant of this problem.
- $f_2(x) = f_1(x) + 10$.



Invariance Properties: OneMax Example – Translation



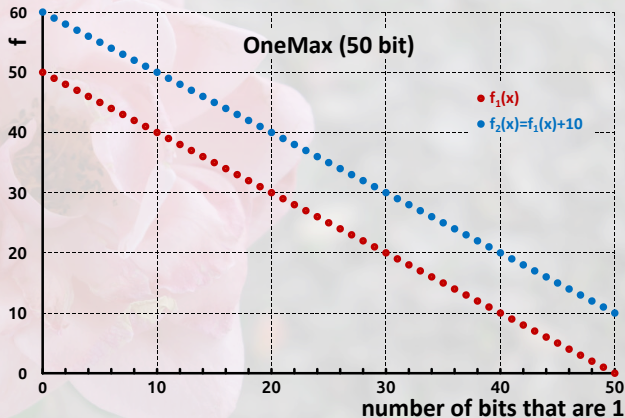
- Now I create a slightly modified variant of this problem.
- $f_2(x) = f_1(x) + 10$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_2 .



Invariance Properties: OneMax Example – Translation



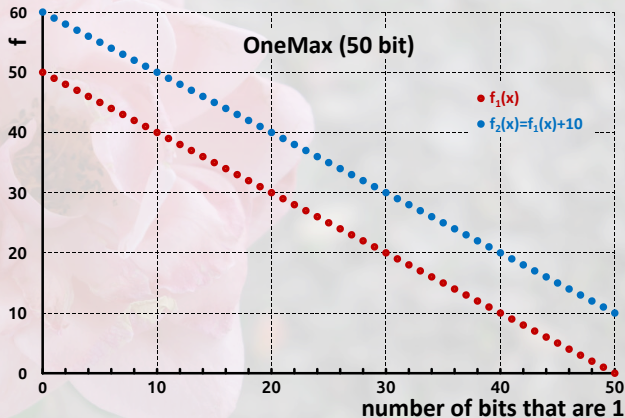
- Now I create a slightly modified variant of this problem.
- $f_2(x) = f_1(x) + 10$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_2 , i.e., make the exact same decisions.



Invariance Properties: OneMax Example – Translation



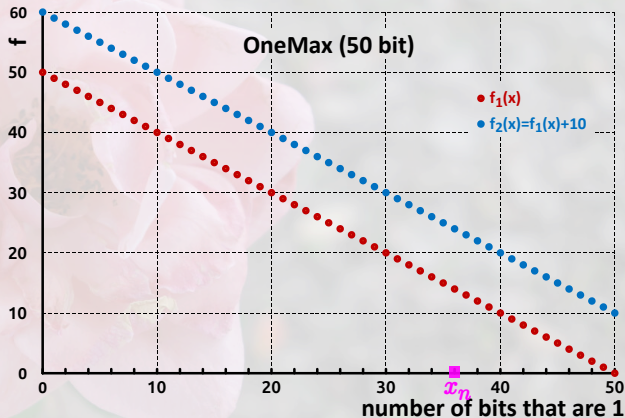
- Now I create a slightly modified variant of this problem.
- $f_2(x) = f_1(x) + 10$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_2 , i.e., make the exact same decisions given the same random seeds.



Invariance Properties: OneMax Example – Translation



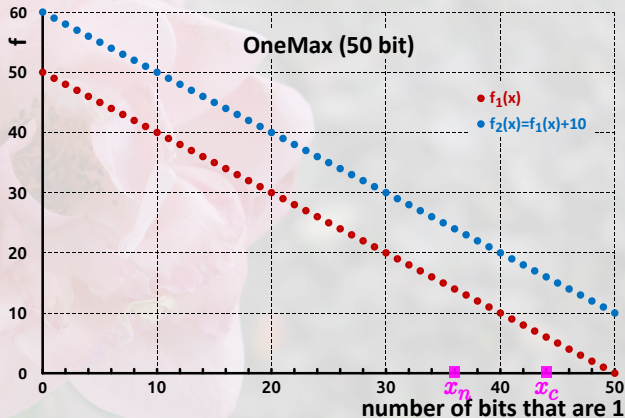
- Now I create a slightly modified variant of this problem.
- $f_2(x) = f_1(x) + 10$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_2 , i.e., make the exact same decisions given the same random seeds.



Invariance Properties: OneMax Example – Translation



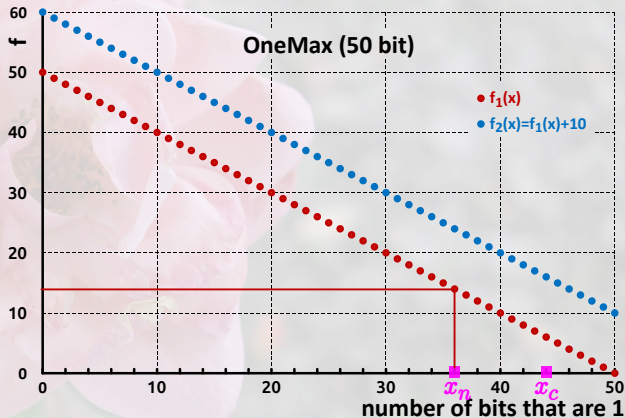
- Now I create a slightly modified variant of this problem.
- $f_2(x) = f_1(x) + 10$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_2 , i.e., make the exact same decisions given the same random seeds.



Invariance Properties: OneMax Example – Translation



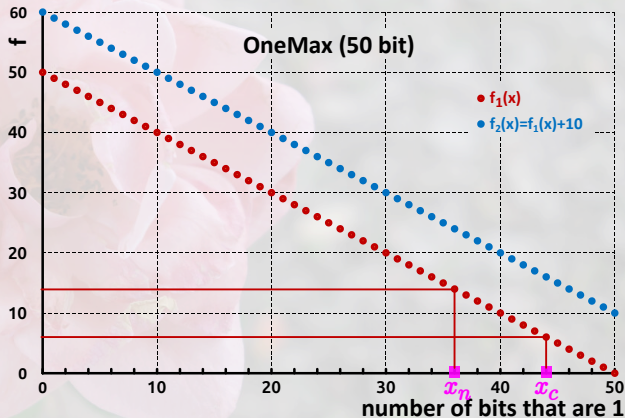
- Now I create a slightly modified variant of this problem.
- $f_2(x) = f_1(x) + 10$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_2 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$



Invariance Properties: OneMax Example – Translation



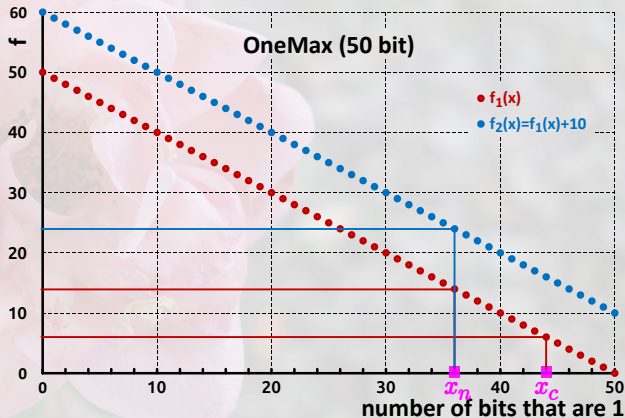
- Now I create a slightly modified variant of this problem.
- $f_2(x) = f_1(x) + 10$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_2 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$



Invariance Properties: OneMax Example – Translation



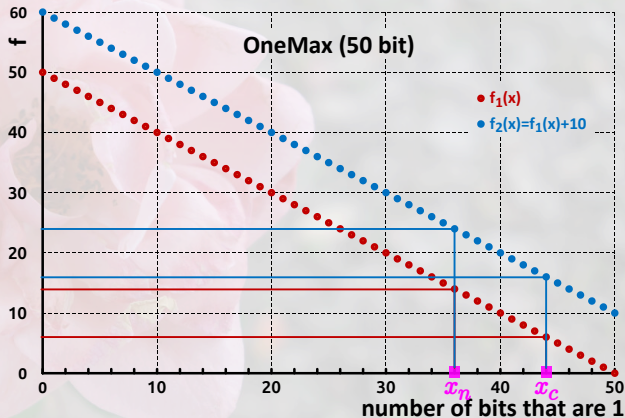
- Now I create a slightly modified variant of this problem.
- $f_2(x) = f_1(x) + 10$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_2 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_2(x_n) = 24$



Invariance Properties: OneMax Example – Translation



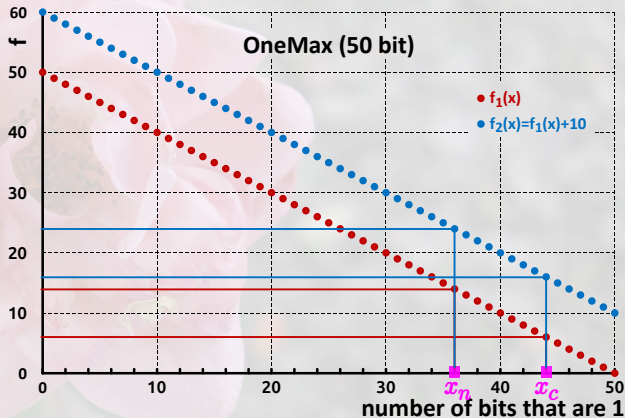
- Now I create a slightly modified variant of this problem.
- $f_2(x) = f_1(x) + 10$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_2 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_2(x_n) = 24$, $f_2(x_c) = 16$



Invariance Properties: OneMax Example – Translation



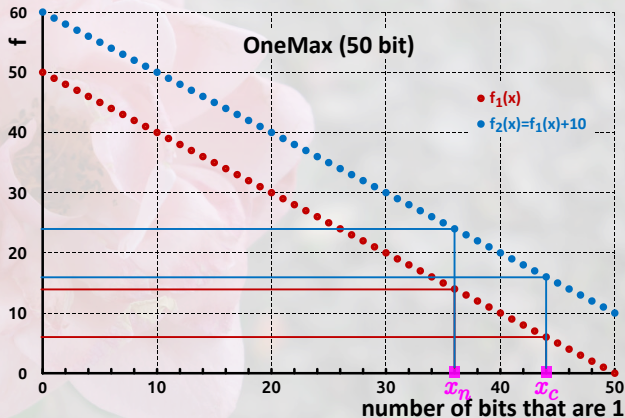
- Now I create a slightly modified variant of this problem.
- $f_2(x) = f_1(x) + 10$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_2 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_2(x_n) = 24$, $f_2(x_c) = 16$
- $(1 + 1)$ EA: $14 \leq 6 = 24 \leq 16$ ✓



Invariance Properties: OneMax Example – Translation



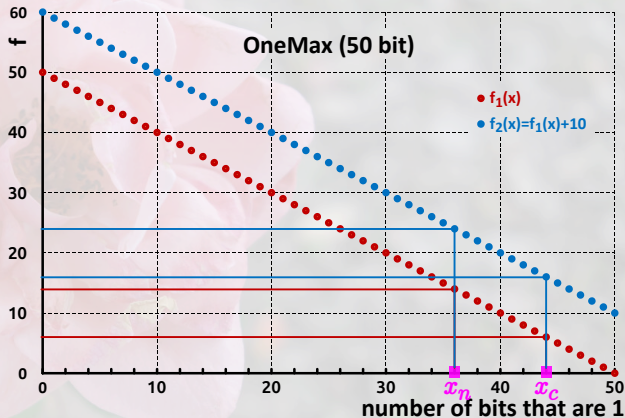
- Now I create a slightly modified variant of this problem.
- $f_2(x) = f_1(x) + 10$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_2 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_2(x_n) = 24$, $f_2(x_c) = 16$
- $(1 + 1)$ EA: $14 \leq 6 = 24 \leq 16$ ✓
- SA: $6 - 14 = 16 - 24$ ✓



Invariance Properties: OneMax Example – Translation



- Now I create a slightly modified variant of this problem.
- $f_2(x) = f_1(x) + 10$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_2 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_2(x_n) = 24$, $f_2(x_c) = 16$
- $(1 + 1)$ EA: $14 \leq 6 = 24 \leq 16$ ✓
- SA: $6 - 14 = 16 - 24$ ✓
- SGA: $1 - 14/(14 + 6) = 0.3 \neq$
 $1 - 24/(24 + 16) = 0.4$ ✗



Invariance Properties: OneMax Example – Translation



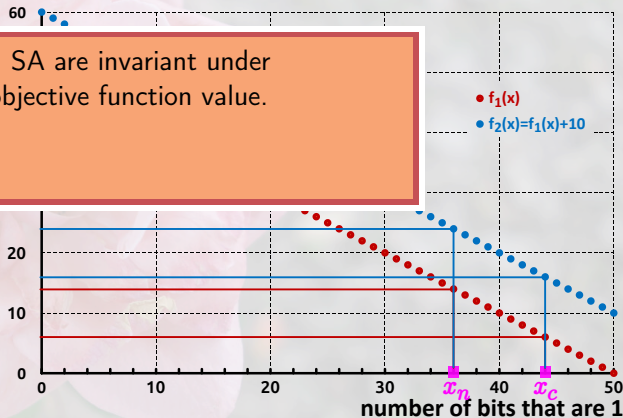
- Now I create a slightly modified variant of this problem.

- $f_2(x) = f_1(x) + 10$.

- Expectation: algorithm should make the same decision on f_1 and f_2 from the same decision random seeds:

- The $(1 + 1)$ EA and SA are invariant under translations of the objective function value.

- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_2(x_n) = 24$, $f_2(x_c) = 16$
- $(1 + 1)$ EA: $14 \leq 6 = 24 \leq 16$ ✓
- SA: $6 - 14 = 16 - 24$ ✓
- SGA: $1 - 14/(14 + 6) = 0.3 \neq$
 $1 - 24/(24 + 16) = 0.4$ ✗



Invariance Properties: OneMax Example – Translation



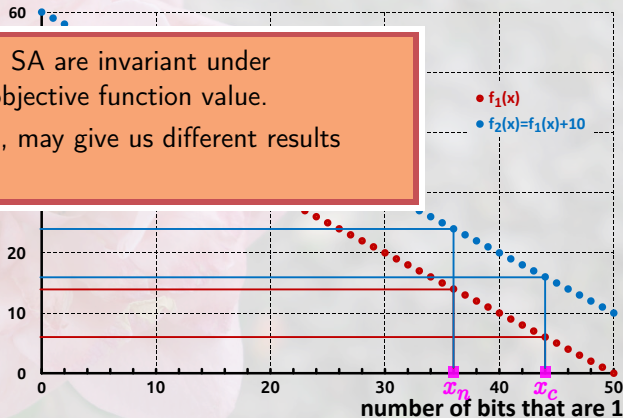
- Now I create a slightly modified variant of this problem.

- $f_2(x) = f_1(x) + 10.$

- Expectation: algorithm should give the same decision on f_1 and f_2 for the same decision and random seed.

- The $(1 + 1)$ EA and SA are invariant under translations of the objective function value.
- The SGA is not, i.e., may give us different results for f_1 and f_2 .

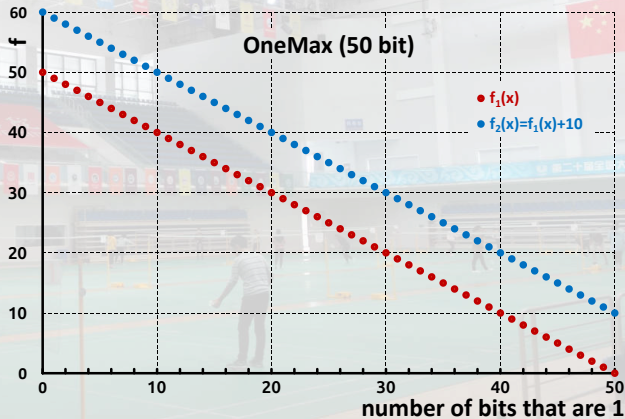
- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_2(x_n) = 24$, $f_2(x_c) = 16$
- $(1 + 1)$ EA: $14 \leq 6 = 24 \leq 16$ ✓
- SA: $6 - 14 = 16 - 24$ ✓
- SGA: $1 - 14/(14 + 6) = 0.3 \neq$
 $1 - 24/(24 + 16) = 0.4$ ✗



Invariance Properties: OneMax Example – Scaling



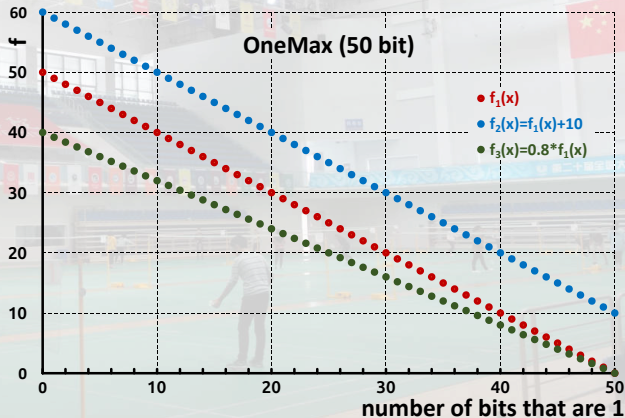
- Now I create another slightly modified variant of this problem.



Invariance Properties: OneMax Example – Scaling



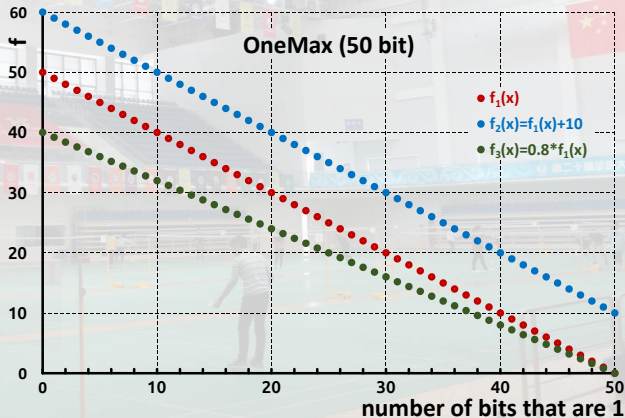
- Now I create another slightly modified variant of this problem.
- $f_3(x) = 0.8f_1(x)$.



Invariance Properties: OneMax Example – Scaling



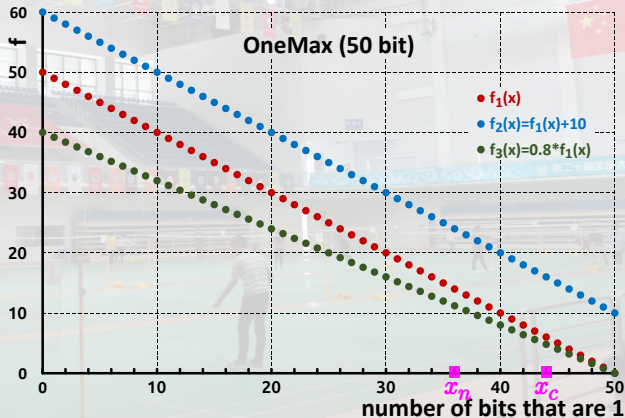
- Now I create another slightly modified variant of this problem.
- $f_3(x) = 0.8f_1(x)$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_3 , i.e., make the exact same decisions given the same random seeds.



Invariance Properties: OneMax Example – Scaling



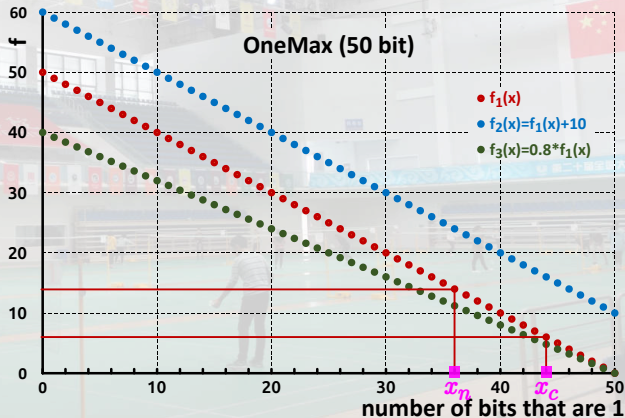
- Now I create another slightly modified variant of this problem.
- $f_3(x) = 0.8f_1(x)$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_3 , i.e., make the exact same decisions given the same random seeds.



Invariance Properties: OneMax Example – Scaling



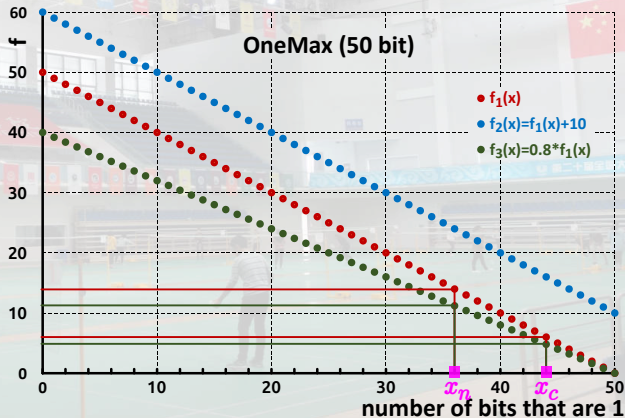
- Now I create another slightly modified variant of this problem.
- $f_3(x) = 0.8f_1(x)$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_3 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$



Invariance Properties: OneMax Example – Scaling



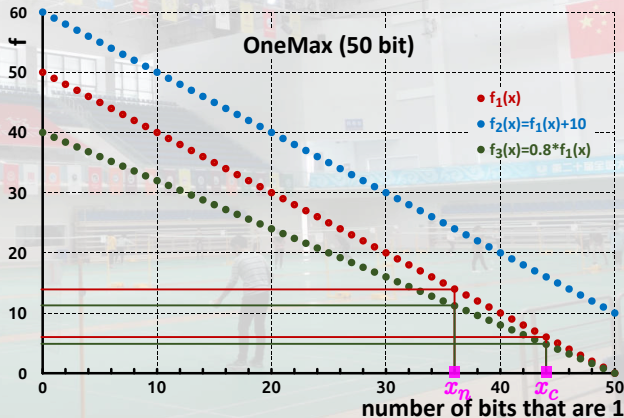
- Now I create another slightly modified variant of this problem.
- $f_3(x) = 0.8f_1(x)$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_3 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_3(x_n) = 11.2$, $f_3(x_c) = 4.8$



Invariance Properties: OneMax Example – Scaling



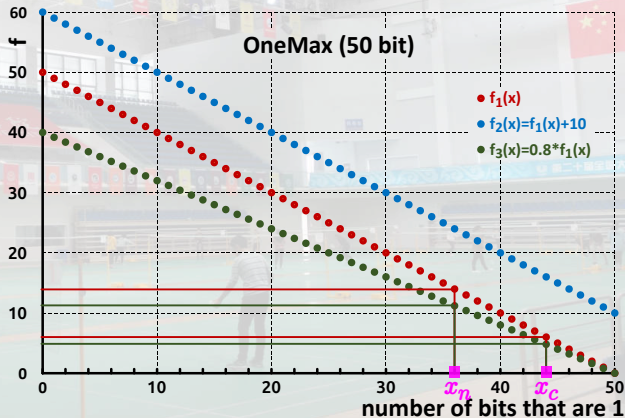
- Now I create another slightly modified variant of this problem.
- $f_3(x) = 0.8f_1(x)$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_3 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_3(x_n) = 11.2$, $f_3(x_c) = 4.8$
- $(1 + 1)$ EA: $14 \leq 6 = 11.2 \leq 4.8$ ✓



Invariance Properties: OneMax Example – Scaling



- Now I create another slightly modified variant of this problem.
- $f_3(x) = 0.8f_1(x)$.
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_3 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_3(x_n) = 11.2$, $f_3(x_c) = 4.8$
- $(1 + 1)$ EA: $14 \leq 6 = 11.2 \leq 4.8$ ✓
- SA: $6 - 14 \neq 4.8 - 11.2$ ✗



Invariance Properties: OneMax Example – Scaling



- Now I create another slightly modified variant of this problem.

- $f_3(x) = 0.8f_1(x)$.

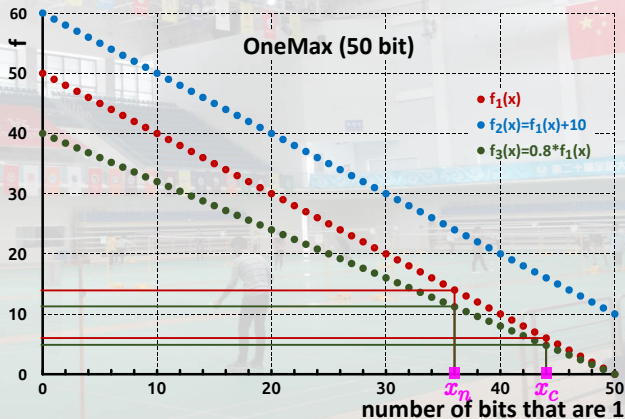
- Expectation: Any reasonable algorithm should perform the same on f_1 and f_3 , i.e., make the exact same decisions given the same random seeds.

- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_3(x_n) = 11.2$, $f_3(x_c) = 4.8$

- $(1 + 1)$ EA: $14 \leq 6 = 11.2 \leq 4.8$ ✓

- SA: $6 - 14 \neq 4.8 - 11.2$ ✗

- SGA: $1 - 14/(14 + 6) = 0.3 =$
 $1 - 11.2/(11.2 + 4.8) = 0.3$ ✓



Invariance Properties: OneMax Example – Scaling



- Now I create another slightly modified variant of this problem.

- $f_3(x) = 0.8f_1(x)$.

- Expectation: algorithm should reach the same decision on f_1 and f_3 for the same random seed.

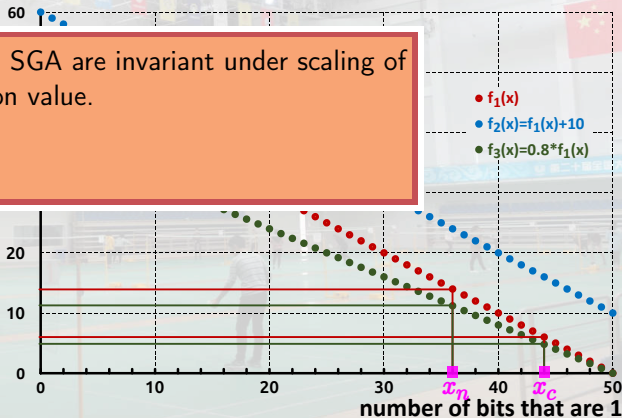
- The $(1+1)$ EA and SGA are invariant under scaling of the objective function value.

- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_3(x_n) = 11.2$, $f_3(x_c) = 4.8$

- $(1+1)$ EA: $14 \leq 6 = 11.2 \leq 4.8$ ✓

- SA: $6 - 14 \neq 4.8 - 11.2$ ✗

- SGA: $1 - 14/(14 + 6) = 0.3 =$
 $1 - 11.2/(11.2 + 4.8) = 0.3$ ✓



Invariance Properties: OneMax Example – Scaling



- Now I create another slightly modified variant of this problem.

- $f_3(x) = 0.8f_1(x)$.

- Expectation: algorithm should converge on f_1 and f_3 with the same decision with different random seeds.

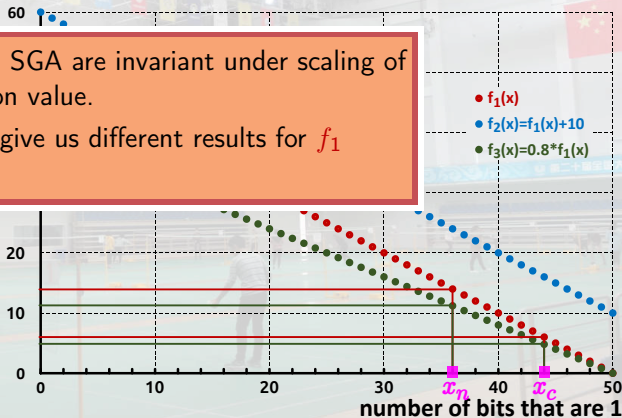
- The $(1+1)$ EA and SGA are invariant under scaling of the objective function value.
- SA is not, i.e., may give us different results for f_1 and f_3 .

- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_3(x_n) = 11.2$, $f_3(x_c) = 4.8$

- $(1+1)$ EA: $14 \leq 6 = 11.2 \leq 4.8$ ✓

- SA: $6 - 14 \neq 4.8 - 11.2$ ✗

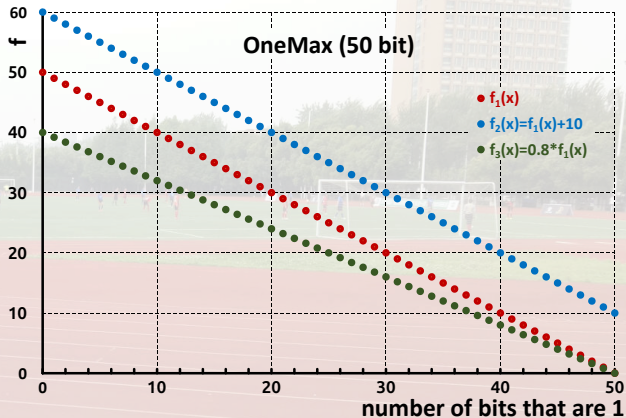
- SGA: $1 - 14/(14 + 6) = 0.3 =$
 $1 - 11.2/(11.2 + 4.8) = 0.3$ ✓



Invariance Properties: OneMax Example – Squaring



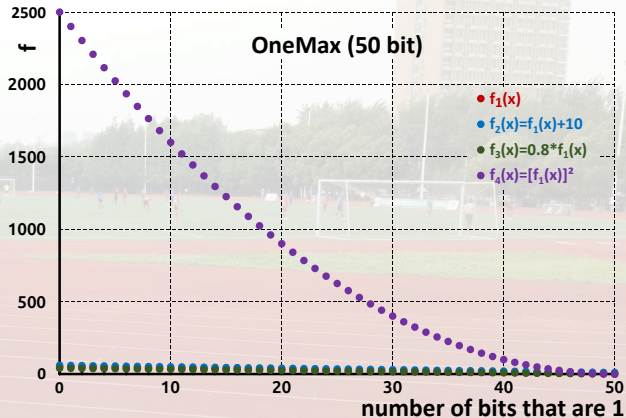
- Now I create another slightly modified variant of this problem.



Invariance Properties: OneMax Example – Squaring



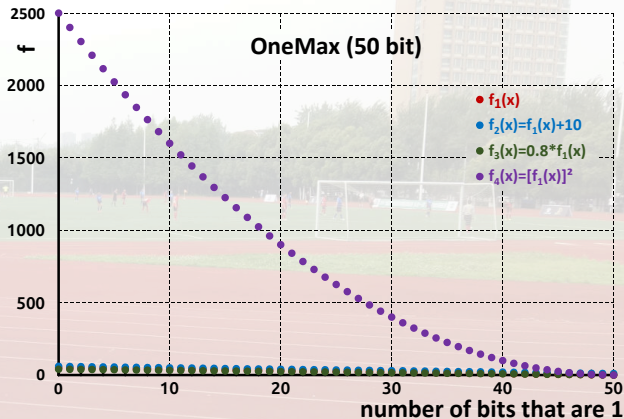
- Now I create another slightly modified variant of this problem.
- $f_4(x) = [f_1(x)]^2$.



Invariance Properties: OneMax Example – Squaring



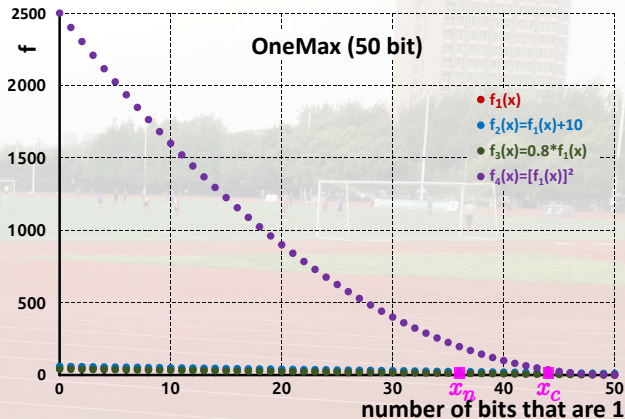
- Now I create another slightly modified variant of this problem.
- $f_4(x) = [f_1(x)]^2$.
- Expectation: A nice algorithm should perform the same on f_1 and f_4 , i.e., make the exact same decisions given the same random seeds.



Invariance Properties: OneMax Example – Squaring



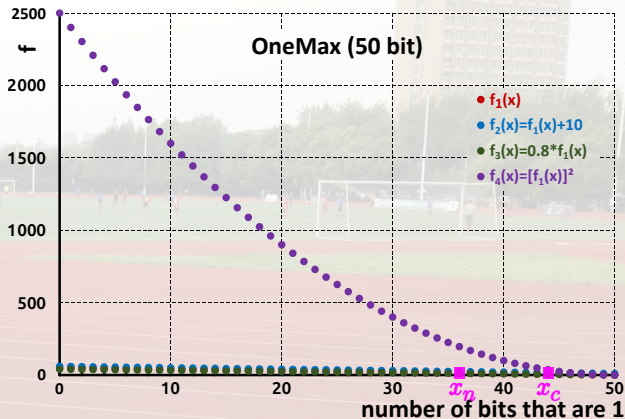
- Now I create another slightly modified variant of this problem.
- $f_4(x) = [f_1(x)]^2$.
- Expectation: A nice algorithm should perform the same on f_1 and f_4 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$



Invariance Properties: OneMax Example – Squaring



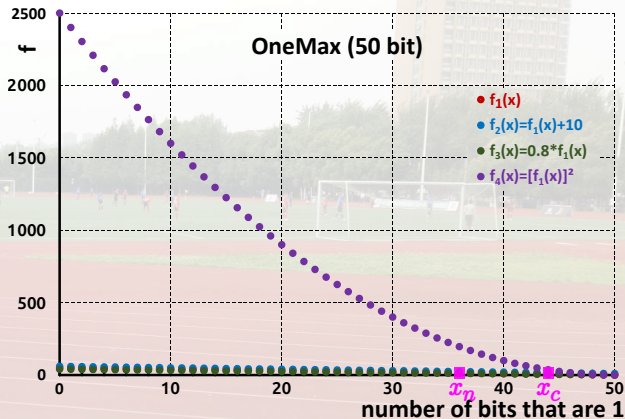
- Now I create another slightly modified variant of this problem.
- $f_4(x) = [f_1(x)]^2$.
- Expectation: A nice algorithm should perform the same on f_1 and f_4 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_4(x_n) = 196$, $f_4(x_c) = 36$



Invariance Properties: OneMax Example – Squaring



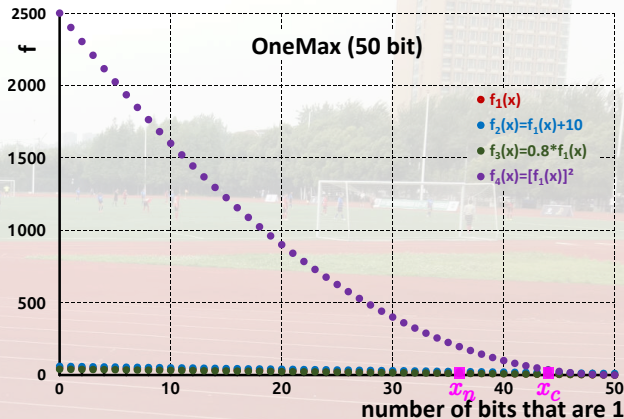
- Now I create another slightly modified variant of this problem.
- $f_4(x) = [f_1(x)]^2$.
- Expectation: A nice algorithm should perform the same on f_1 and f_4 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_4(x_n) = 196$, $f_4(x_c) = 36$
- $(1 + 1)$ EA: $14 \leq 6 = 196 \leq 36$ ✓



Invariance Properties: OneMax Example – Squaring



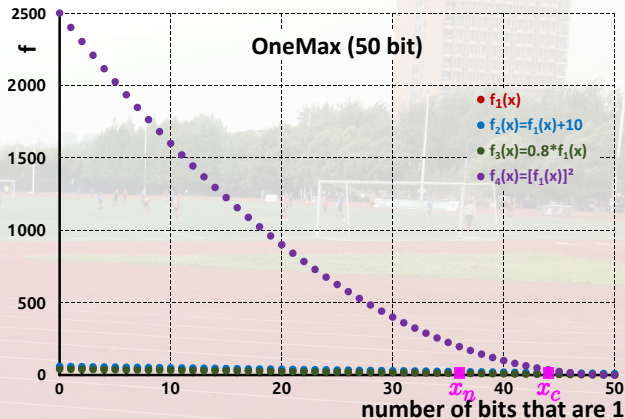
- Now I create another slightly modified variant of this problem.
- $f_4(x) = [f_1(x)]^2$.
- Expectation: A nice algorithm should perform the same on f_1 and f_4 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_4(x_n) = 196$, $f_4(x_c) = 36$
- $(1 + 1)$ EA: $14 \leq 6 \leq 196 \leq 36$ ✓
- SA: $6 - 14 \neq 36 - 196$ ✗



Invariance Properties: OneMax Example – Squaring



- Now I create another slightly modified variant of this problem.
- $f_4(x) = [f_1(x)]^2$.
- Expectation: A nice algorithm should perform the same on f_1 and f_4 , i.e., make the exact same decisions given the same random seeds.
- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_4(x_n) = 196$, $f_4(x_c) = 36$
- $(1 + 1)$ EA: $14 \leq 6 \leq 196 \leq 36$ ✓
- SA: $6 - 14 \neq 36 - 196$ ✗
- SGA: $1 - 14/(14 + 6) = 0.3 \neq 1 - 196/(196 + 36) \approx 0.156$ ✗



Invariance Properties: OneMax Example – Squaring



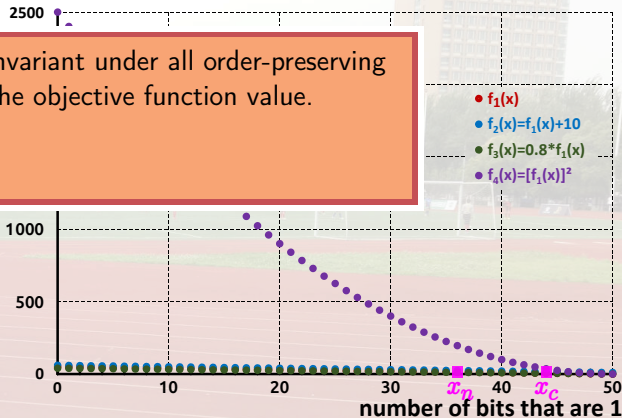
- Now I create another slightly modified variant of this problem.

- $f_4(x) = [f_1(x)]^2$.

- Expectation: should perform the same for f_1 and f_4 , i.e., the number of decisions given the same seeds.

- The $(1 + 1)$ EA is invariant under all order-preserving transformations of the objective function value.

- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_4(x_n) = 196$, $f_4(x_c) = 36$
- $(1 + 1)$ EA: $14 \leq 6 \leq 196 \leq 36$ ✓
- SA: $6 - 14 \neq 36 - 196$ ✗
- SGA: $1 - 14/(14 + 6) = 0.3 \neq 1 - 196/(196 + 36) \approx 0.156$ ✗



Invariance Properties: OneMax Example – Squaring



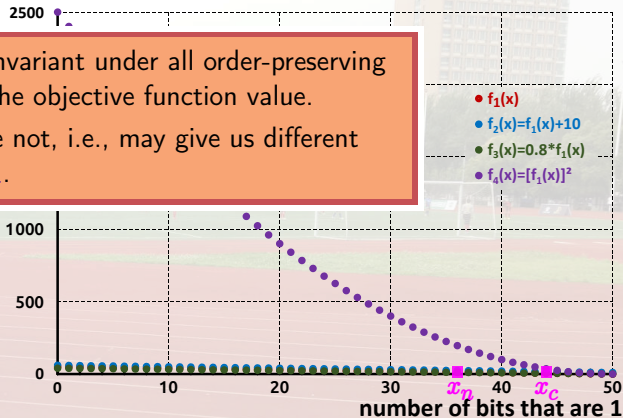
- Now I create another slightly modified variant of this problem.

- $f_4(x) = [f_1(x)]^2$.

- Expectation: should perform similarly for f_1 and f_4 , i.e., decisions given by the same seeds.

- The $(1 + 1)$ EA is invariant under all order-preserving transformations of the objective function value.
- SA and the SGA are not, i.e., may give us different results for f_1 and f_4 .

- $f_1(x_n) = 14$, $f_1(x_c) = 6$,
 $f_4(x_n) = 196$, $f_4(x_c) = 36$
- $(1 + 1)$ EA: $14 \leq 6 \leq 196 \leq 36$ ✓
- SA: $6 - 14 \neq 36 - 196$ ✗
- SGA: $1 - 14/(14 + 6) = 0.3 \neq 1 - 196/(196 + 36) \approx 0.156$ ✗



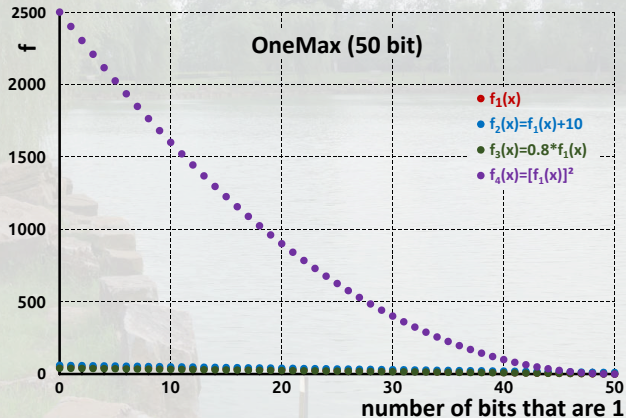


Now let's enter eerie territory...

Invariance Properties: OneMax Example – Trap



- Now I create another modified variant of this problem: a Trap^{25,69}.

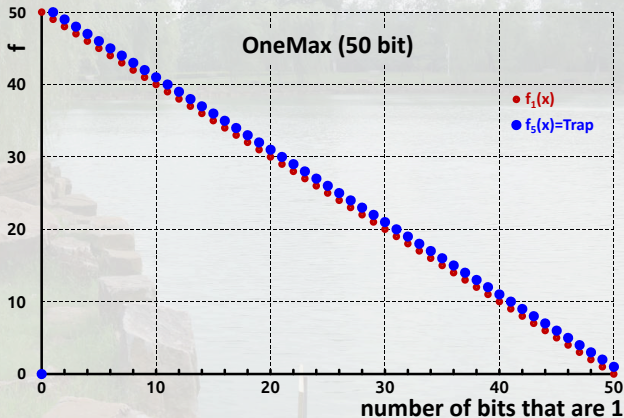


Invariance Properties: OneMax Example – Trap



- Now I create another modified variant of this problem: a Trap^{25,69}.

- $f_5(x) = \begin{cases} 0 & \text{if } f_1(x) = 50 \\ 1 + f_1(x) & \text{otherwise} \end{cases}$

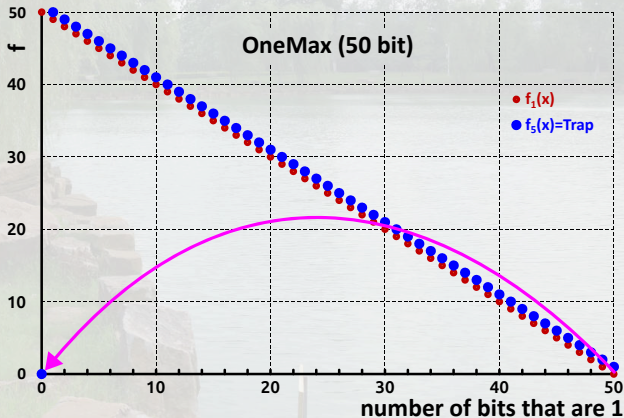


Invariance Properties: OneMax Example – Trap



- Now I create another modified variant of this problem: a Trap^{25,69}.

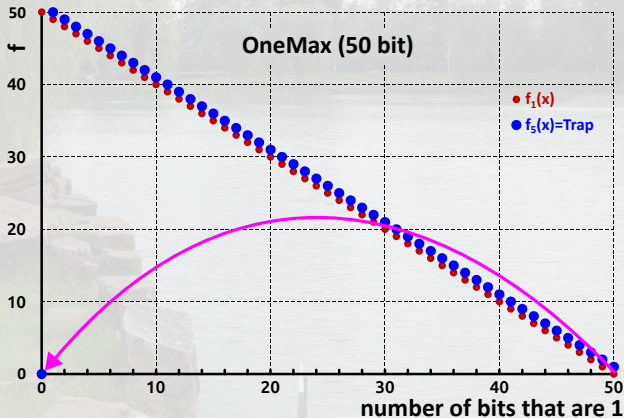
- $f_5(x) = \begin{cases} 0 & \text{if } f_1(x) = 50 \\ 1 + f_1(x) & \text{otherwise} \end{cases}$



Invariance Properties: OneMax Example – Trap



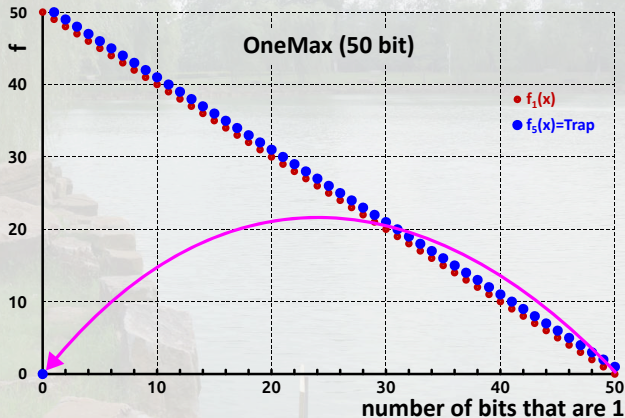
- Now I create another modified variant of this problem: a Trap^{25,69}.
- $f_5(x) = \begin{cases} 0 & \text{if } f_1(x) = 50 \\ 1 + f_1(x) & \text{otherwise} \end{cases}$
- Expectation: Algorithm performance on f_1 does not carry over to f_5 .



Invariance Properties: OneMax Example – Trap



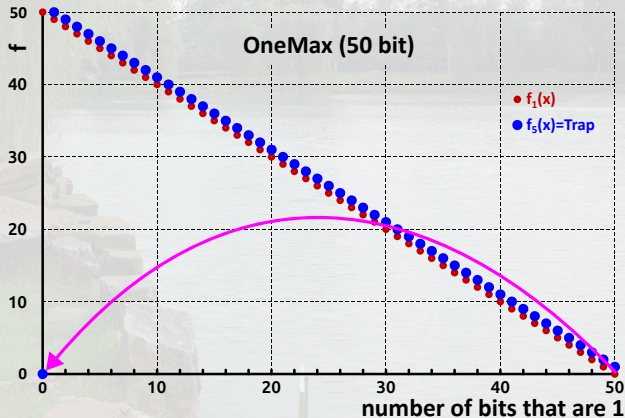
- Now I create another modified variant of this problem: a Trap^{25,69}.
- $f_5(x) = \begin{cases} 0 & \text{if } f_1(x) = 50 \\ 1 + f_1(x) & \text{otherwise} \end{cases}$
- Expectation: Algorithm performance on f_1 does not carry over to f_5 .
- Neither the $(1 + 1)$ EA, SA, nor the SGA can deal with this well.



Invariance Properties: OneMax Example – Trap



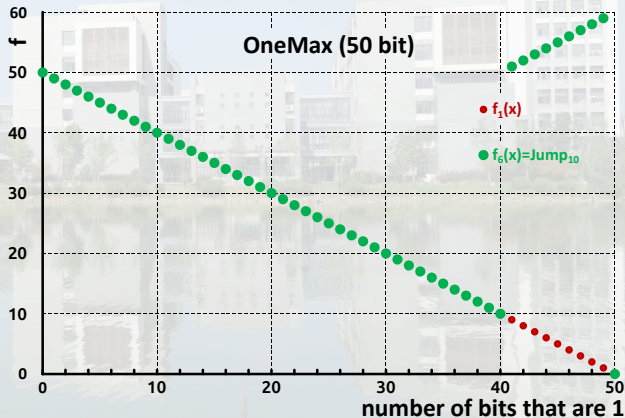
- Now I create another modified variant of this problem: a Trap^{25,69}.
- $f_5(x) = \begin{cases} 0 & \text{if } f_1(x) = 50 \\ 1 + f_1(x) & \text{otherwise} \end{cases}$
- Expectation: Algorithm performance on f_1 does not carry over to f_5 .
- Neither the $(1 + 1)$ EA, SA, nor the SGA can deal with this well.
- The $(1 + 1)$ EA has expected runtime $\Theta(n^n)$ on Traps²⁵.



Invariance Properties: OneMax Example – Jump



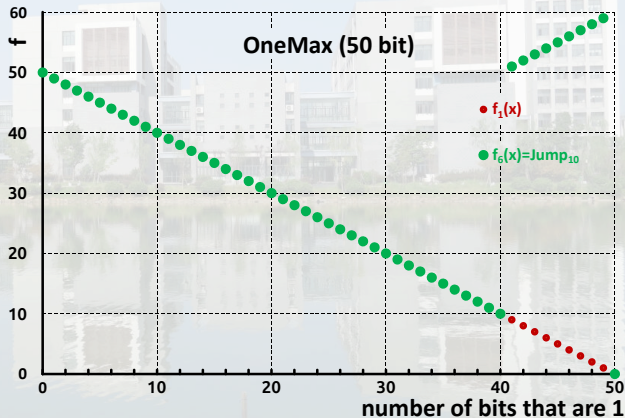
- Now I create another modified variant of this problem: a jump of width $\omega = 10$ bits.



Invariance Properties: OneMax Example – Jump



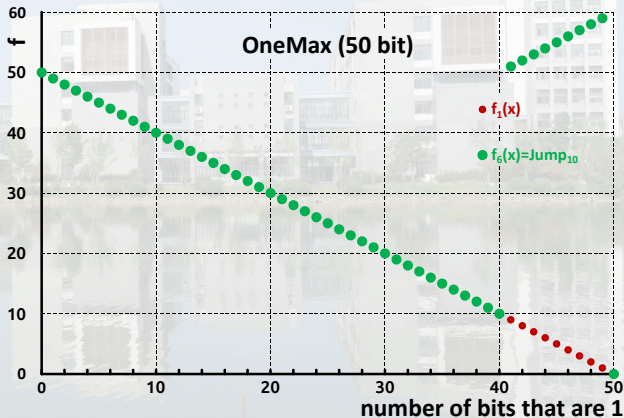
- Now I create another modified variant of this problem: a jump of width $\omega = 10$ bits.
- $f_6(x)$ has a deceptive area of $\omega - 1$ bits before the optimum.



Invariance Properties: OneMax Example – Jump



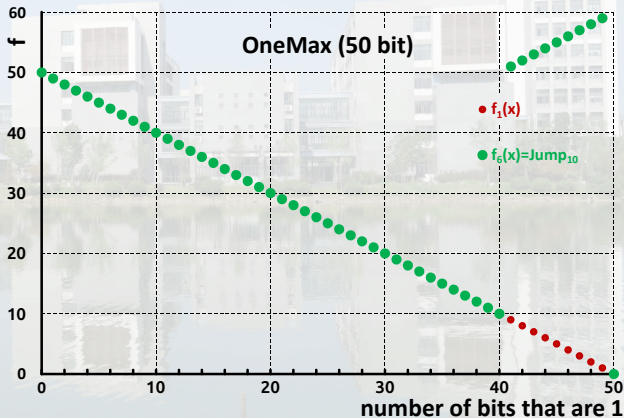
- Now I create another modified variant of this problem: a jump of width $\omega = 10$ bits.
- $f_6(x)$ has a deceptive area of $\omega - 1$ bits before the optimum.
- Expectation: Algorithm performance on f_1 does not carry over to f_6 .



Invariance Properties: OneMax Example – Jump



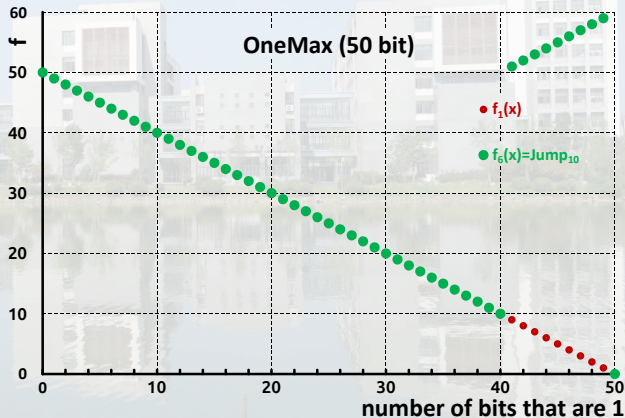
- Now I create another modified variant of this problem: a jump of width $\omega = 10$ bits.
- $f_6(x)$ has a deceptive area of $\omega - 1$ bits before the optimum.
- Expectation: Algorithm performance on f_1 does not carry over to f_6 .
- Neither the $(1 + 1)$ EA, SA, nor the SGA can deal with this well.



Invariance Properties: OneMax Example – Jump



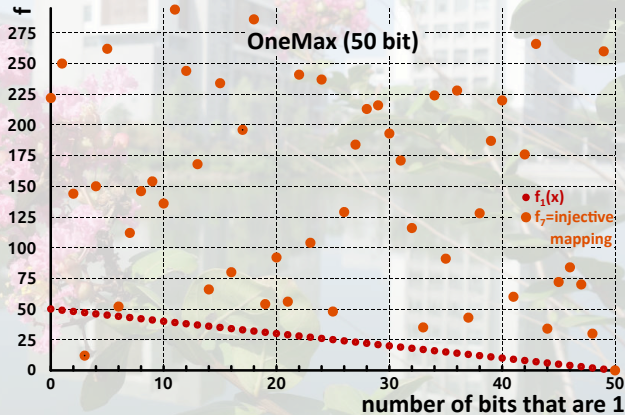
- Now I create another modified variant of this problem: a jump of width $\omega = 10$ bits.
- $f_6(x)$ has a deceptive area of $\omega - 1$ bits before the optimum.
- Expectation: Algorithm performance on f_1 does not carry over to f_6 .
- Neither the $(1 + 1)$ EA, SA, nor the SGA can deal with this well.
- The $(1 + 1)$ EA has expected runtime $\Theta(n^\omega + n \ln n)$ on jumps²⁵.



Invariance Properties: OneMax Example – Injection



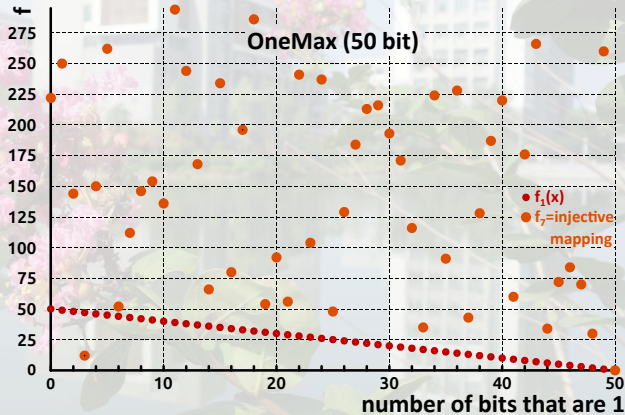
- How about I apply an arbitrary injection g that preserves the location of optimum to f_1 and get $f_7(x) = g(f_1(x))$?



Invariance Properties: OneMax Example – Injection



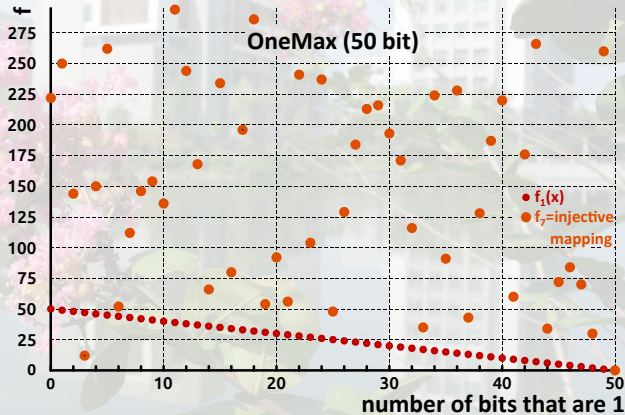
- How about I apply an arbitrary injection g that preserves the location of optimum to f_1 and get $f_7(x) = g(f_1(x))$?
- Expectation: Algorithm performance on f_1 is probably unrelated to performance on f_7 .



Invariance Properties: OneMax Example – Injection



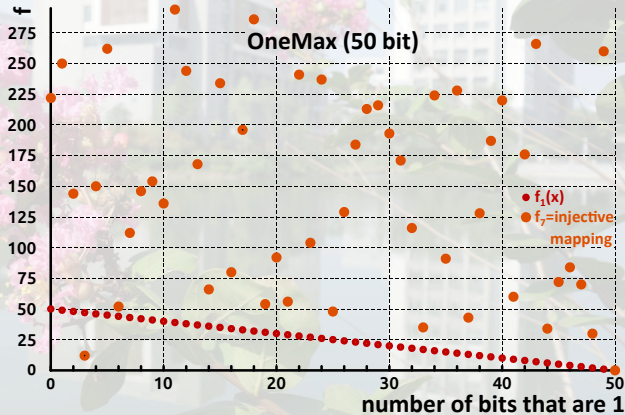
- How about I apply an arbitrary injection g that preserves the location of optimum to f_1 and get $f_7(x) = g(f_1(x))$?
- Expectation: Algorithm performance on f_1 is probably unrelated to performance on f_7 .
- Neither the $(1 + 1)$ EA, SA, nor the SGA can deal with this well.



Invariance Properties: OneMax Example – Injection



- How about I apply an arbitrary injection g that preserves the location of optimum to f_1 and get $f_7(x) = g(f_1(x))$?
- Expectation: Algorithm performance on f_1 is probably unrelated to performance on f_7 .
- Neither the $(1 + 1)$ EA, SA, nor the SGA can deal with this well.
- Indeed, **no** existing algorithm can deal with this, let alone have some invariance properties with respect to *that*.





Indeed, **no** existing algorithm can deal with this, let alone have some invariance properties with respect to *that*.



Indeed, **no** existing algorithm can deal with this, let alone have some invariance properties with respect to *that*.

well...



Frequency Fitness Assignment: Making Optimization Algorithms Invariant Under Bijective Transformations of the Objective Function Value

Thomas Weise[✉], Member, IEEE, Zhize Wu, Xinlu Li, and Yan Chen

Abstract—Under frequency fitness assignment (FFA), the fitness corresponding to an objective value is its encounter frequency in fitness assignment steps and is subject to minimization. FFA renders optimization processes invariant under bijective transformations of the objective function value. On TwoMax, Jump, and Trap functions of dimension s , the classical $(1+1)$ -EA with standard mutation at rate $1/s$ can have expected runtimes exponential in s . In our experiments, a $(1+1)$ -FEA, the same algorithm but using FFA, exhibits mean runtimes that seem to scale as $s^2 \ln s$. Since Jump and Trap are bijective transformations of OneMax, it behaves identical on all three. On OneMax, LeadingOnes, and Plateau problems, it seems to be slower than the $(1+1)$ -EA by a factor linear in s . The $(1+1)$ -FEA performs much better than the $(1+1)$ -EA on W-Model and MaxSat instances. We further verify the bijection invariance by applying the Md5 checksum computation as transformation to some of the above problems and yield the same behaviors. Finally, we show that FFA can improve the performance of a memetic algorithm for job shop scheduling.

Index Terms— $(1+1)$ -EA, evolutionary algorithm (EA), frequency fitness assignment (FFA), job shop scheduling (JSS), Jump problem, LeadingOnes, memetic algorithm, OneMax, Plateau problem, TwoMax, Trap problem.

which are then the basis for selection. Frequency fitness assignment (FFA) [1], [2] was developed to enable algorithms to escape from local optima. In FFA, the fitness corresponding to an objective value is its encounter frequency so far in fitness assignment steps and is subject to minimization. As we discuss in detail in Section II, FFA turns a static optimization problem into a dynamic one where objective values that are often encountered will receive worse and worse fitness.

In this article, we uncover a so-far unexplored property of FFA: it is invariant under any bijective transformation of the objective function values. This is the strongest invariance known to us and encompasses all order-preserving mappings. Other examples for bijective transformations include the negation, permutation, or even encryption of the objective values. According to [3], invariance extends performance observed on a single function to an entire associated invariance class, that is, it generalizes from a single problem class to a problem class. Thus, it hopefully enables a more general presentation of the results.



Frequency Fitness Assignment



FFA: Idea



- Frequency Fitness Assignment (FFA) is a module that can be plugged into different existing algorithms.



FFA: Idea

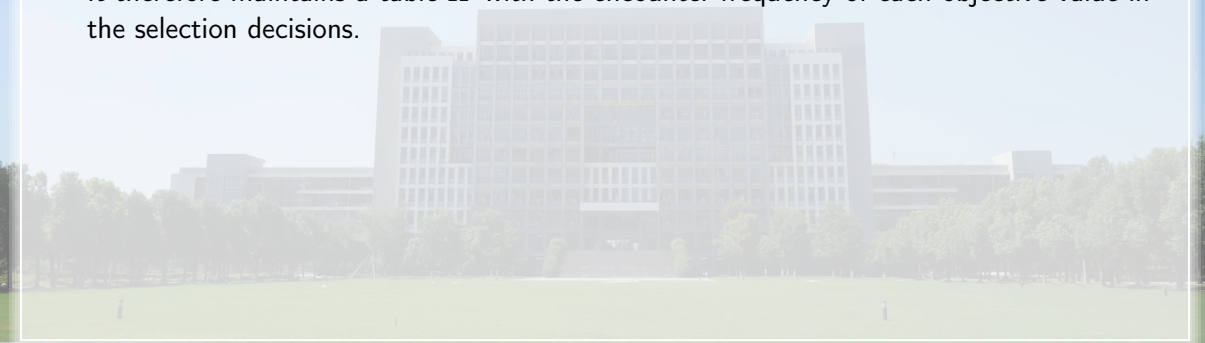


- Frequency Fitness Assignment (FFA) is a module that can be plugged into different existing algorithms.
- It changes the way the algorithm selects the interesting solutions S_{i+1} from the sets $P_i = S_i \cup N_i$.





- Frequency Fitness Assignment (FFA) is a module that can be plugged into different existing algorithms.
- It changes the way the algorithm selects the interesting solutions S_{i+1} from the sets $P_i = S_i \cup N_i$.
- It therefore maintains a table H with the encounter frequency of each objective value in the selection decisions.





- Frequency Fitness Assignment (FFA) is a module that can be plugged into different existing algorithms.
- It changes the way the algorithm selects the interesting solutions S_{i+1} from the sets $P_i = S_i \cup N_i$.
- It therefore maintains a table H with the encounter frequency of each objective value in the selection decisions.
- The table H is initially filled with zeros.



- Frequency Fitness Assignment (FFA) is a module that can be plugged into different existing algorithms.
- It changes the way the algorithm selects the interesting solutions S_{i+1} from the sets $P_i = S_i \cup N_i$.
- It therefore maintains a table H with the encounter frequency of each objective value in the selection decisions.
- The table H is initially filled with zeros.
- Before the selection step of the algorithm, $H[f(P_i[j])]$ for all $j \in 1..|P_i|$ is incremented by 1.



- Frequency Fitness Assignment (FFA) is a module that can be plugged into different existing algorithms.
- It changes the way the algorithm selects the interesting solutions S_{i+1} from the sets $P_i = S_i \cup N_i$.
- It therefore maintains a table H with the encounter frequency of each objective value in the selection decisions.
- The table H is initially filled with zeros.
- Before the selection step of the algorithm, $H[f(P_i[j])]$ for all $j \in 1..|P_i|$ is incremented by 1.
- Then, the frequencies $H[f(P_i[j])]$ replace the objective values $f(P_i[j])$ in the actual selection decisions.

FFA: (1+1) EA and (1+1) FEA



- Let's plug FFA into the (1 + 1) EA and obtain the (1 + 1) EA with FFA ((1 + 1) FEA).

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```



FFA: (1+1) EA and (1+1) FEA



- We start with the (1 + 1) EA.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{N}$ )
```

```
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;
```

```
  while  $\neg$  terminate do
```

```
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;
```

```
    if  $y_n \leq y_c$  then
```

```
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```

```
  return  $x_c, y_c$ 
```

FFA: (1+1) EA and (1+1) FEA



- We begin by initializing the frequency table H with all zeros.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) FEA( $f : \mathbb{X} \mapsto \mathbb{N}$ )  
   $H \leftarrow (0, 0, \dots, 0)$ ;  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
  
    if  $y_n \leq y_c$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  
  return  $x_c, y_c$ 
```

FFA: (1+1) EA and (1+1) FEA



- Before the selection decision, we increment the frequency values of the objective values of all current solutions.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) FEA( $f : \mathbb{X} \mapsto \mathbb{N}$ )  
   $H \leftarrow (0, 0, \dots, 0)$ ;  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $H[y_c] \leftarrow H[y_c] + 1$ ;  $H[y_n] \leftarrow H[y_n] + 1$ ;  
    if  $y_n \leq y_c$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  
  return  $x_c, y_c$ 
```

FFA: (1+1) EA and (1+1) FEA



- Now the frequency values replace the objective values in the selection decisions.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) FEA( $f : \mathbb{X} \mapsto \mathbb{N}$ )  
   $H \leftarrow (0, 0, \dots, 0)$ ;  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $H[y_c] \leftarrow H[y_c] + 1$ ;  $H[y_n] \leftarrow H[y_n] + 1$ ;  
    if  $H[y_n] \leq H[y_c]$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  
  return  $x_c, y_c$ 
```

FFA: (1+1) EA and (1+1) FEA



- Since we may now lose the best-so-far solution, we need to track it in additional variables.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) FEA( $f : \mathbb{X} \mapsto \mathbb{N}$ )  
   $H \leftarrow (0, 0, \dots, 0)$ ;  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $H[y_c] \leftarrow H[y_c] + 1$ ;  $H[y_n] \leftarrow H[y_n] + 1$ ;  
    if  $H[y_n] \leq H[y_c]$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  
  return  $x_c, y_c$ 
```

FFA: (1+1) EA and (1+1) FEA



- Since we may now lose the best-so-far solution, we need to track it in additional variables.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) FEA( $f : \mathbb{X} \mapsto \mathbb{N}$ )  
   $H \leftarrow (0, 0, \dots, 0)$ ;  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $H[y_c] \leftarrow H[y_c] + 1$ ;  $H[y_n] \leftarrow H[y_n] + 1$ ;  
    if  $H[y_n] \leq H[y_c]$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
      if  $y_c < y_B$  then  $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;  
  return  $x_c, y_c$ 
```

FFA: (1+1) EA and (1+1) FEA



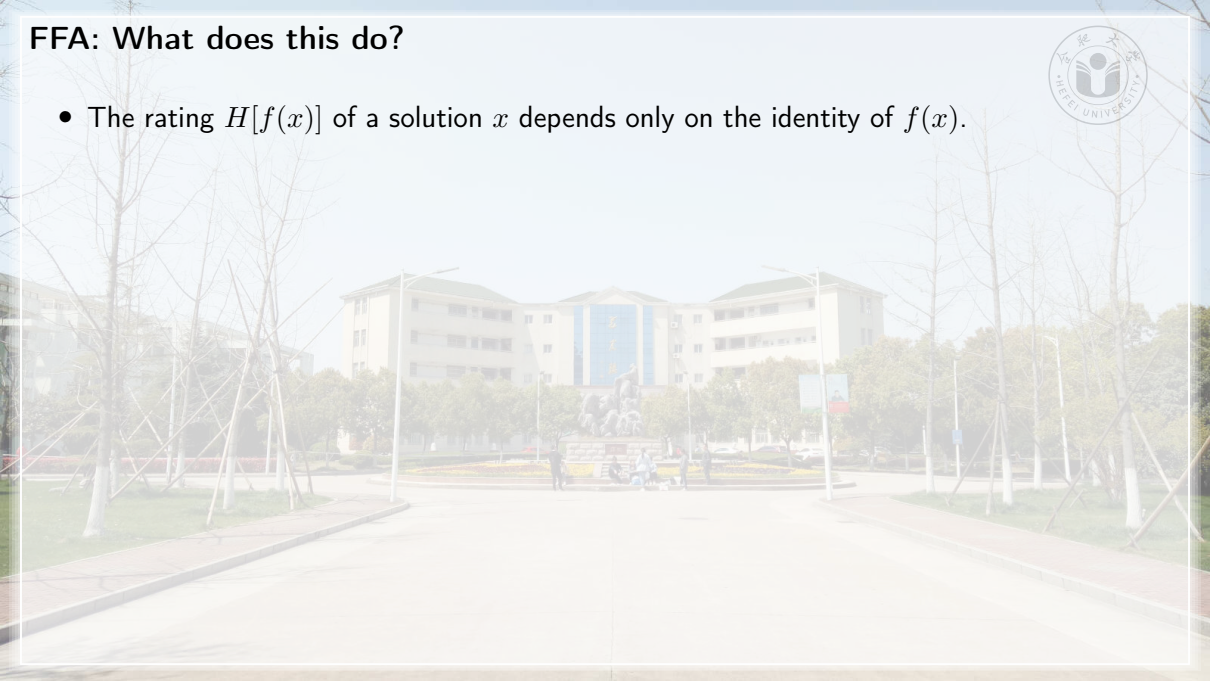
- Since we may now lose the best-so-far solution, we need to track it in additional variables.
- ... which are then the return values of the (1 + 1) FEA.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) FEA( $f : \mathbb{X} \mapsto \mathbb{N}$ )  
   $H \leftarrow (0, 0, \dots, 0)$ ;  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $H[y_c] \leftarrow H[y_c] + 1$ ;  $H[y_n] \leftarrow H[y_n] + 1$ ;  
    if  $H[y_n] \leq H[y_c]$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
      if  $y_c < y_B$  then  $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;  
  return  $x_B, y_B$ 
```

FFA: What does this do?

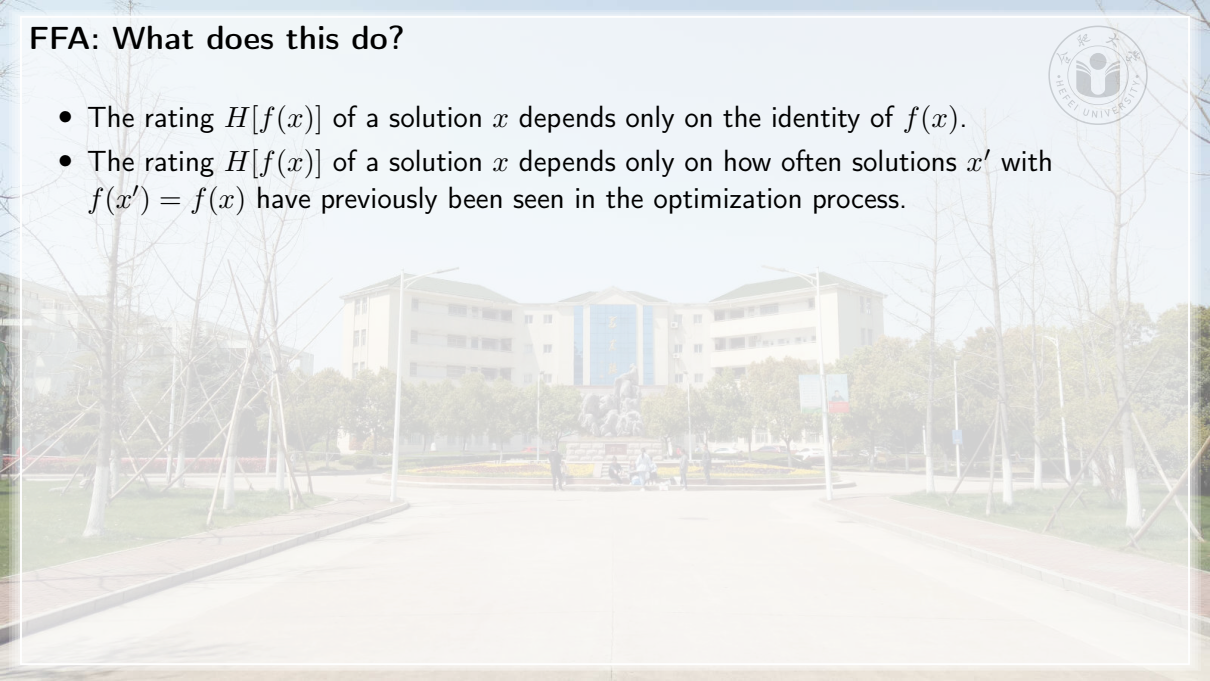
- The rating $H[f(x)]$ of a solution x depends only on the identity of $f(x)$.



FFA: What does this do?



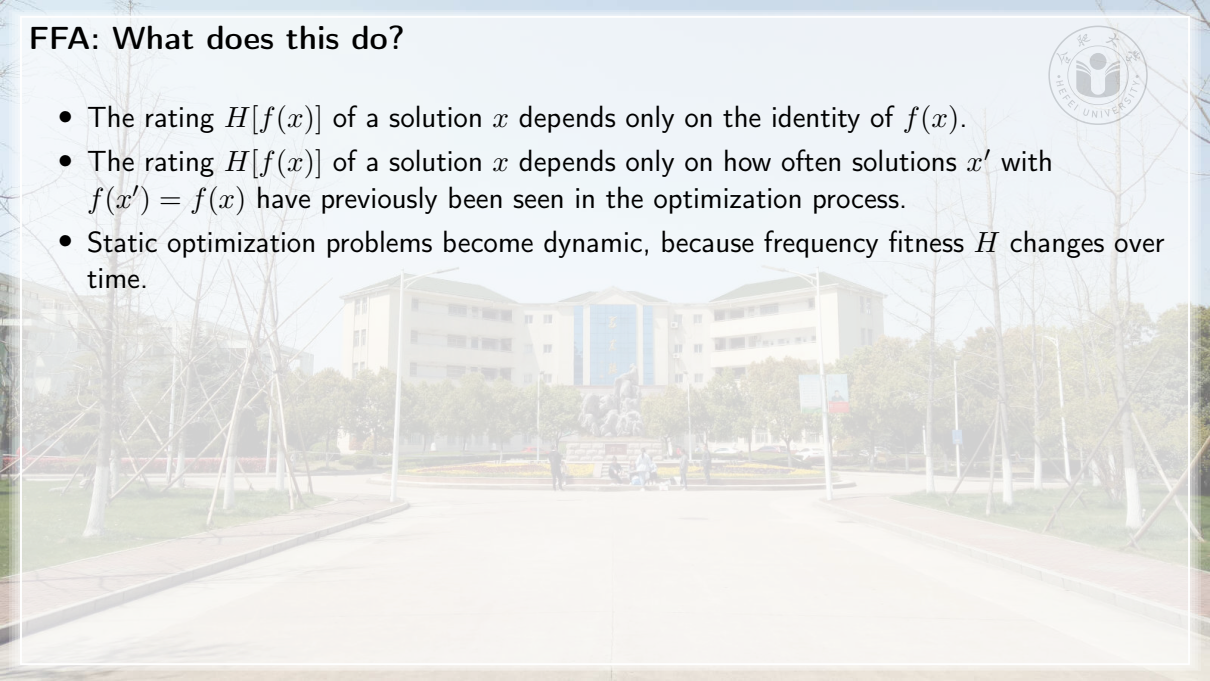
- The rating $H[f(x)]$ of a solution x depends only on the identity of $f(x)$.
- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.



FFA: What does this do?



- The rating $H[f(x)]$ of a solution x depends only on the identity of $f(x)$.
- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.
- Static optimization problems become dynamic, because frequency fitness H changes over time.



FFA: What does this do?



- The rating $H[f(x)]$ of a solution x depends only on the identity of $f(x)$.
- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.
- Static optimization problems become dynamic, because frequency fitness H changes over time.
- Solutions get less attractive the more often their corresponding objective values have been seen. This also holds for local optima...

FFA: What does this do?



- The rating $H[f(x)]$ of a solution x depends only on the identity of $f(x)$.
- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.
- Static optimization problems become dynamic, because frequency fitness H changes over time.
- Solutions get less attractive the more often their corresponding objective values have been seen. This also holds for local optima. . .
- Solutions with better objective values are no longer preferred over such with worse objective value.

FFA: What does this do?



- The rating $H[f(x)]$ of a solution x depends only on the identity of $f(x)$.
- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.
- Static optimization problems become dynamic, because frequency fitness H changes over time.
- Solutions get less attractive the more often their corresponding objective values have been seen. This also holds for local optima. . .
- Solutions with better objective values are no longer preferred over such with worse objective value.
- Instead, solutions with less-frequent objective values are preferred.

FFA: What does this do?



- The rating $H[f(x)]$ of a solution x depends only on the identity of $f(x)$.
- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.
- Static optimization problems become dynamic, because frequency fitness H changes over time.
- Solutions get less attractive the more often their corresponding objective values have been seen. This also holds for local optima. . .
- Solutions with better objective values are no longer preferred over such with worse objective value.
- Instead, solutions with less-frequent objective values are preferred.
- Algorithms using FFA are invariant under all injective transformations of the objective function value.

FFA: What does this do?



- The rating $H[f(x)]$ of a solution x depends only on the identity of $f(x)$.
- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.
- Static optimization problems become dynamic, because frequency fitness H changes over time.
- Solutions get less attractive the more often their corresponding objective values have been seen. This also holds for local optima. . .
- Solutions with better objective values are no longer preferred over such with worse objective value.
- Instead, solutions with less-frequent objective values are preferred.
- Algorithms using FFA are invariant under all injective transformations of the objective function value.
- They perform identical on ALL of the OneMax-based functions from before!

Discrete Optimization Theory Benchmarks



- If we use the encounter frequency of objective values as **only** guide for optimization ... can this work?

Discrete Optimization Theory Benchmarks

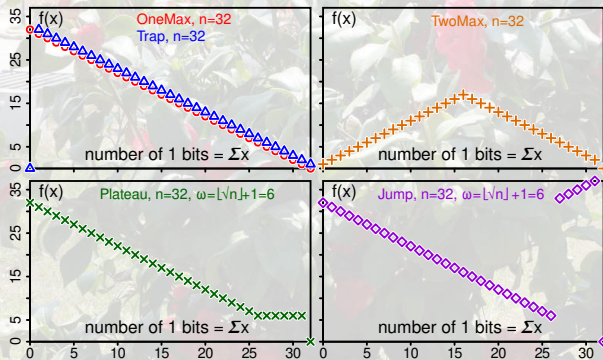


- If we use the encounter frequency of objective values as **only** guide for optimization ... can this work?
- Why should this lead to anything but strange random walks?

Discrete Optimization Theory Benchmarks

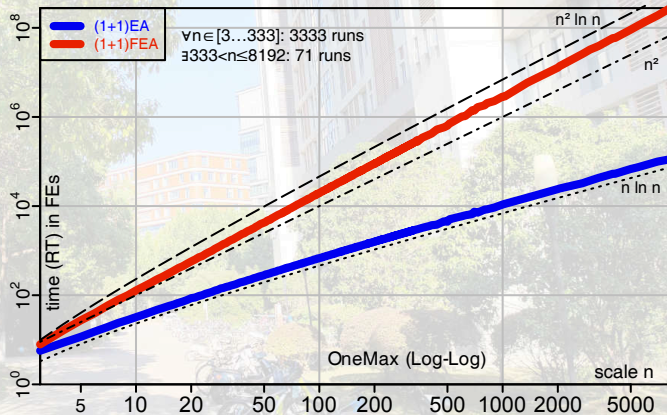


- If we use the encounter frequency of objective values as **only** guide for optimization ... can this work?
- Why should this lead to anything but strange random walks?
- So we tested it on discrete optimization benchmarks.



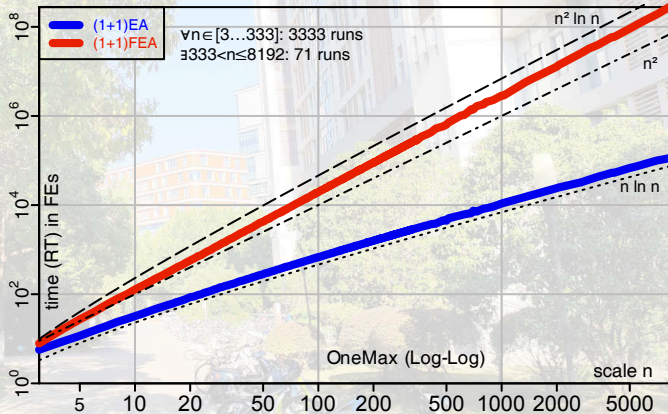
(1+1) FEA on OneMax

- As first example, we again take OneMax⁶⁸.



(1+1) FEA on OneMax

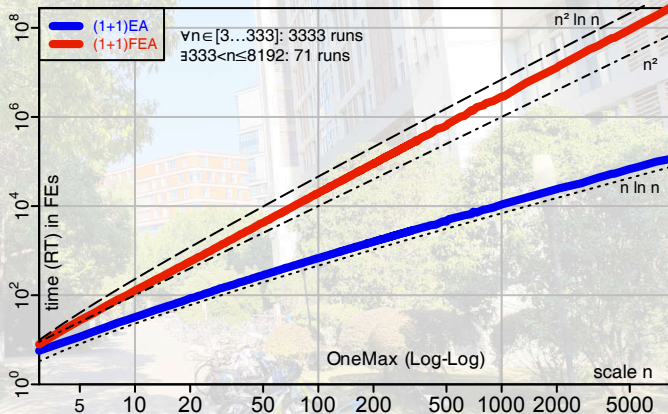
- As first example, we again take OneMax⁶⁸.
- The expected runtime of (1 + 1) EA on OneMax is in $\Theta(n \ln n)$ ^{24,68}.



(1+1) FEA on OneMax



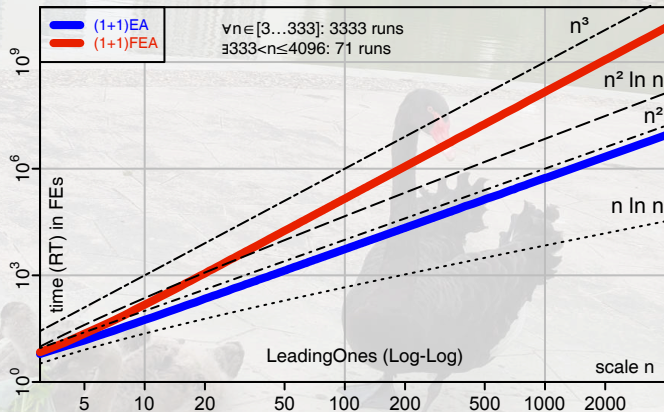
- As first example, we again take OneMax⁶⁸.
- The expected runtime of $(1 + 1)$ EA on OneMax is in $\Theta(n \ln n)$ ^{24,68}.
- The average runtime $(1 + 1)$ FEA on OneMax seems to be slower by factor proportional to n , i.e., seems to be in $\mathcal{O}(n^2 \ln n)$ ¹⁰⁸.



(1+1) FEA on LeadingOnes



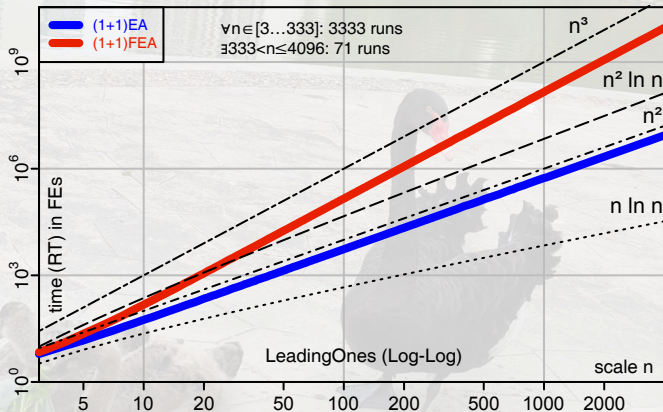
- The LeadingOnes problem maximizes the number of leading 1 bits in a bit string^{84,111}.



(1+1) FEA on LeadingOnes



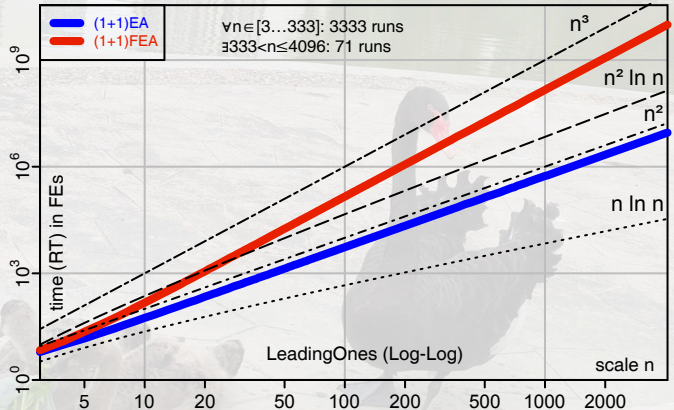
- The LeadingOnes problem maximizes the number of leading 1 bits in a bit string^{84,111}.
- The expected runtime of (1 + 1) EA on LeadingOnes is in $\Theta(n^2)$ ²⁵.



(1+1) FEA on LeadingOnes



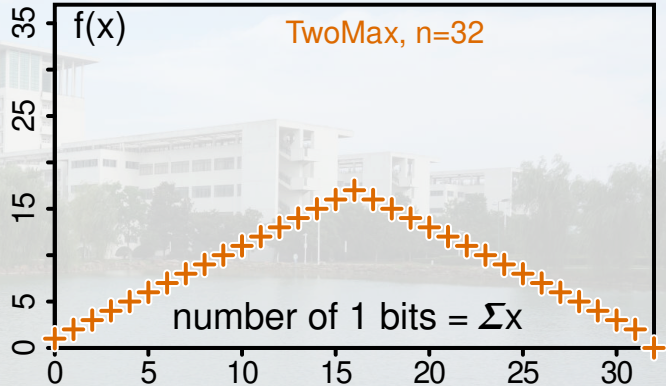
- The LeadingOnes problem maximizes the number of leading 1 bits in a bit string^{84,111}.
- The expected runtime of (1 + 1) EA on LeadingOnes is in $\Theta(n^2)$ ²⁵.
- The average runtime (1 + 1) FEA on LeadingOnes seems to be slower by factor proportional to n , i.e., seems to be in $\mathcal{O}(n^3)$ ¹⁰⁸.



(1+1) FEA on TwoMax



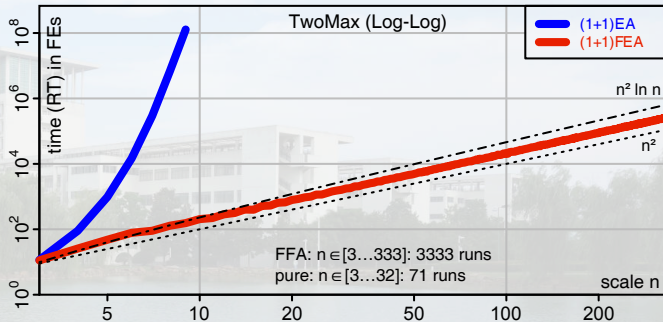
- The TwoMax problem has one local opposite to a global optimum^{32,92,116}.



(1+1) FEA on TwoMax



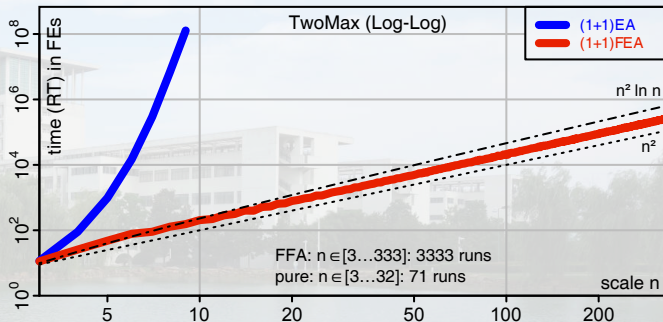
- The TwoMax problem has one local opposite to a global optimum^{32,92,116}.
- An $(1+1)$ EA has a 50/50 chance to converge to the local or global optimum and thus has an expected runtime in $\Theta(n^n)$.^{31,32}



(1+1) FEA on TwoMax



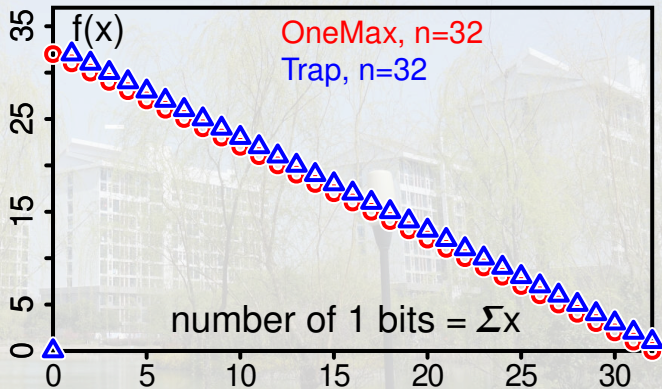
- The TwoMax problem has one local opposite to a global optimum^{32,92,116}.
- An $(1 + 1)$ EA has a 50/50 chance to converge to the local or global optimum and thus has an expected runtime in $\Theta(n^n)$.^{31,32}
- The average runtime $(1 + 1)$ FEA on TwoMax seems to be in $\mathcal{O}(n^2 \ln n)$.¹⁰⁸



(1+1) FEA on Trap



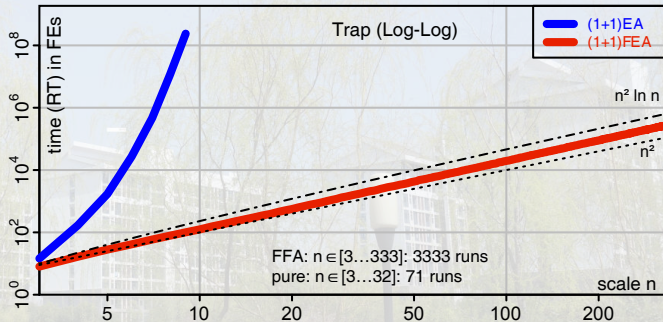
- The Trap problem basically swaps the global optimum with the worst solution^{25,69}.



(1+1) FEA on Trap



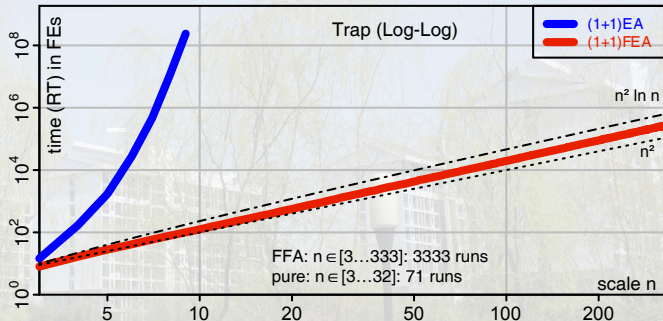
- The Trap problem basically swaps the global optimum with the worst solution^{25,69}.
- An $(1 + 1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^n)$ ²⁵.



(1+1) FEA on Trap



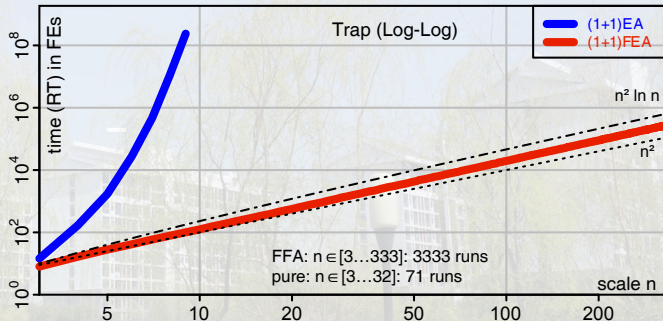
- The Trap problem basically swaps the global optimum with the worst solution^{25,69}.
- An $(1 + 1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^n)$ ²⁵.
- The average runtime $(1 + 1)$ FEA on Trap seems to be in $\mathcal{O}(n^2 \ln n)$ ¹⁰⁸.



(1+1) FEA on Trap

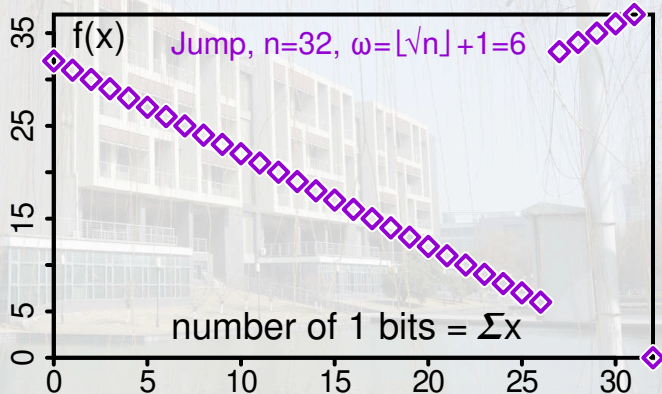


- The Trap problem basically swaps the global optimum with the worst solution^{25,69}.
- An $(1 + 1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^n)$ ²⁵.
- The average runtime $(1 + 1)$ FEA on Trap seems to be in $\mathcal{O}(n^2 \ln n)$ ¹⁰⁸.
- The expected runtime is the same as on OneMax, because Trap is an injective transformation of OneMax, symmetric, and FFA “converges” into both directions.



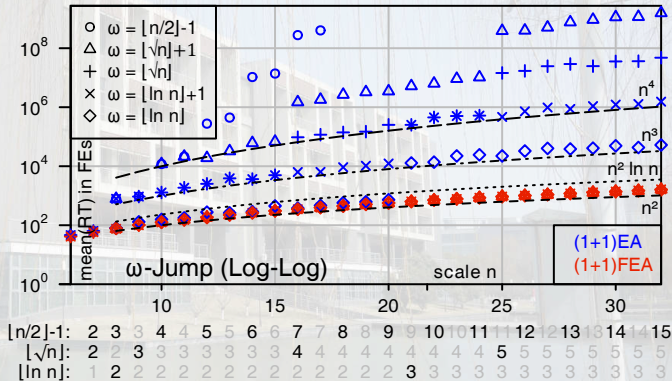
(1+1) FEA on Jumps

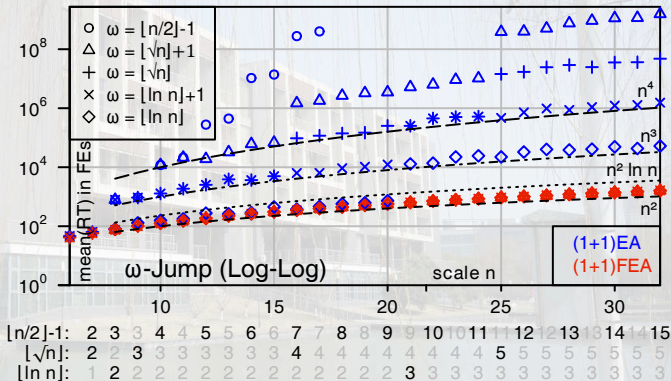
- The Jump problem inserts a deceptive region of $\omega - 1$ bits before the optimum, forcing the algorithm to toggle ω bits at once^{25,32}.

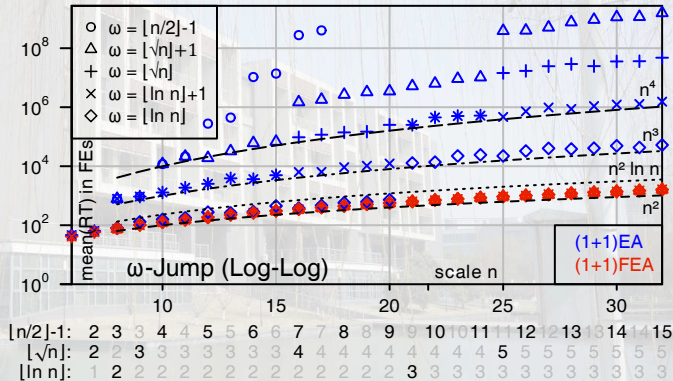


(1+1) FEA on Jumps

- The Jump problem inserts a deceptive region of $\omega - 1$ bits before the optimum, forcing the algorithm to toggle ω bits at once^{25,32}.
- An $(1 + 1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^\omega + n \ln n)$ ²⁵.



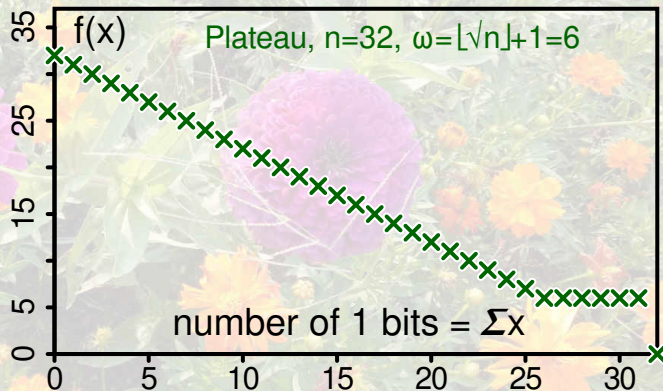






(1+1) FEA on Plateaus

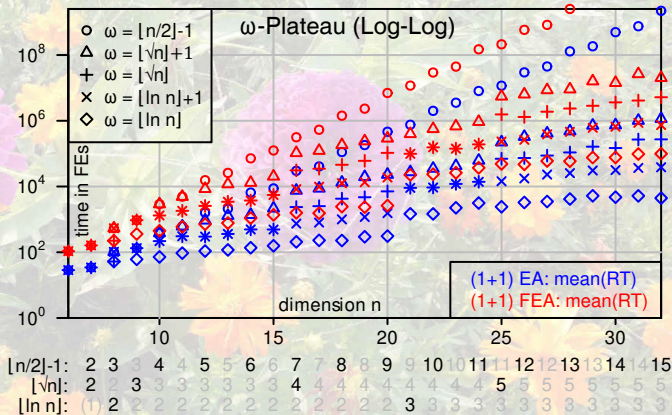
- The Plateau problem inserts a neutral region of $\omega - 1$ bits before the optimum, forcing the algorithm to toggle ω bits at once².





(1+1) FEA on Plateaus

- The Plateau problem inserts a neutral region of $\omega - 1$ bits before the optimum, forcing the algorithm to toggle ω bits at once².
- An $(1 + 1)$ EA has an expected runtime of $\Theta(n^\omega)$ on plateaus².



- [illegible]

-
- ω-Plateau (Log-Log)**
- Legend:
- $\omega = \lfloor n/2 \rfloor - 1$
 - △ $\omega = \lfloor \sqrt{n} \rfloor + 1$
 - + $\omega = \lfloor \sqrt{n} \rfloor$
 - × $\omega = \lfloor \ln n \rfloor + 1$
 - ◇ $\omega = \lfloor \ln n \rfloor$
- Y-axis: time in FEs (log scale)
- X-axis: dimension n
- Shaded regions:
- (1+1) EA: mean(RT) (green)
 - (1+1) FEA: mean(RT) (red)
- | n | $\lfloor n/2 \rfloor - 1$ | $\lfloor \sqrt{n} \rfloor$ | $\lfloor \ln n \rfloor$ |
|----|---------------------------|----------------------------|-------------------------|
| 2 | 2 | 2 | 1 |
| 3 | 3 | 2 | 2 |
| 4 | 4 | 3 | 2 |
| 5 | 5 | 3 | 2 |
| 6 | 6 | 3 | 2 |
| 7 | 7 | 4 | 2 |
| 8 | 8 | 4 | 2 |
| 9 | 9 | 4 | 2 |
| 10 | 10 | 4 | 3 |
| 11 | 11 | 5 | 3 |
| 12 | 12 | 5 | 3 |
| 13 | 13 | 5 | 3 |
| 14 | 14 | 5 | 3 |
| 15 | 15 | 5 | 3 |

(1+1) FEA on MaxSAT



- The Maximum Satisfiability (MaxSAT)^{17,42} problem is $\mathcal{NP}$ -complete¹⁷.

(1+1) FEA on MaxSAT



- The Maximum Satisfiability (MaxSAT)^{17,42} problem is $\mathcal{NP}$ -complete¹⁷.
- SATLIB⁴⁴ provides satisfiable benchmark instances from the phase transition region (i.e., the hardest type of instances) for different scales $n \in 20 \cup \{25i \mid i \in 2..10\}$.

(1+1) FEA on MaxSAT



- The Maximum Satisfiability (MaxSAT)^{17,42} problem is $\mathcal{NP}$ -complete¹⁷.
- SATLIB⁴⁴ provides satisfiable benchmark instances from the phase transition region (i.e., the hardest type of instances) for different scales $n \in 20 \cup \{25i \mid i \in 2..10\}$.
- We conducted 11 000 runs with the $(1 + 1)$ FEA on each instance scale.

(1+1) FEA on MaxSAT



- The Maximum Satisfiability (MaxSAT)^{17,42} problem is $\mathcal{NP}$ -complete¹⁷.
- SATLIB⁴⁴ provides satisfiable benchmark instances from the phase transition region (i.e., the hardest type of instances) for different scales $n \in 20 \cup \{25i \mid i \in 2..10\}$.
- We conducted 11 000 runs with the $(1 + 1)$ FEA on each instance scale.
- The $(1 + 1)$ EA is very much slower than the $(1 + 1)$ FEA, so we could use it only on smaller scales.

(1+1) FEA on MaxSAT



- The Maximum Satisfiability (MaxSAT)^{17,42} problem is $\mathcal{NP}$ -complete¹⁷.
- SATLIB⁴⁴ provides satisfiable benchmark instances from the phase transition region (i.e., the hardest type of instances) for different scales $n \in 20 \cup \{25i \mid i \in 2..10\}$.
- We conducted 11 000 runs with the $(1 + 1)$ FEA on each instance scale.
- The $(1 + 1)$ EA is very much slower than the $(1 + 1)$ FEA, so we could use it only on smaller scales.
- Our computational budget was always 10^{10} objective function evaluations (FEs).

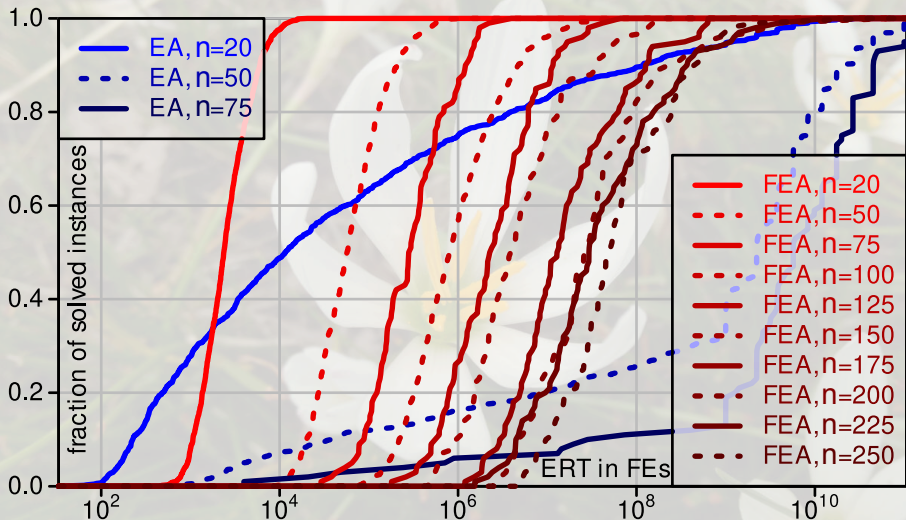
(1+1) FEA on MaxSAT



- The (1 + 1) FEA is better on problems with 250 variables than the (1 + 1) EA on problems with 50.

instance type	(1+1) EA			(1+1) FEA		
	success rate	ERT	mean y_c	success rate	ERT	mean y_B
uf20_*	0.985	$1.91 * 10^8$	0.015	1	3'091	0
uf50_*	0.748	$3.56 * 10^9$	0.299	1	93'459	0
uf75_*	0.583	$7.41 * 10^9$	0.528	1	490'166	0
uf100_*				1	$2.14 * 10^6$	0
uf125_*				1	$5.27 * 10^6$	0
uf150_*				1	$1.40 * 10^7$	0
uf175_*				1	$5.78 * 10^7$	0
uf200_*				0.991	$2.44 * 10^8$	0.00945
uf225_*				0.994	$2.43 * 10^8$	0.00555
uf250_*				0.992	$2.43 * 10^8$	0.00782

(1+1) FEA on MaxSAT: ERT-ECDF



FFA: What does this do?



- So it turns out, FFA does work.

FFA: What does this do?



- So it turns out, FFA does work.
- It works because it prefers solutions with rare objective values.

FFA: What does this do?



- So it turns out, FFA does work.
- It works because it prefers solutions with rare objective values.
- Solutions with very good objective values are also very rare on any problem worth tackling.

FFA: What does this do?



- So it turns out, FFA does work.
- It works because it prefers solutions with rare objective values.
- Solutions with very good objective values are also very rare on any problem worth tackling.
- FFA makes the simple $(1 + 1)$ EA slower on problems that it can easily solve.

FFA: What does this do?



- So it turns out, FFA does work.
- It works because it prefers solutions with rare objective values.
- Solutions with very good objective values are also very rare on any problem worth tackling.
- FFA makes the simple $(1 + 1)$ EA slower on problems that it can easily solve.
- The slowdown is roughly proportional to the number of possible different objective values.

FFA: What does this do?



- So it turns out, FFA does work.
- It works because it prefers solutions with rare objective values.
- Solutions with very good objective values are also very rare on any problem worth tackling.
- FFA makes the simple $(1 + 1)$ EA slower on problems that it can easily solve.
- The slowdown is roughly proportional to the number of possible different objective values.
- On some non- $\mathcal{NP}$ -hard problems for which the $(1 + 1)$ EA needs **exponential** runtime, the $(1 + 1)$ FEA needs **polynomial** mean runtime.

FFA: What does this do?



- So it turns out, FFA does work.
- It works because it prefers solutions with rare objective values.
- Solutions with very good objective values are also very rare on any problem worth tackling.
- FFA makes the simple $(1 + 1)$ EA slower on problems that it can easily solve.
- The slowdown is roughly proportional to the number of possible different objective values.
- On some non- $\mathcal{NP}$ -hard problems for which the $(1 + 1)$ EA needs **exponential** runtime, the $(1 + 1)$ FEA needs **polynomial** mean runtime.
- But not for all, because plateaus of the objective are still plateaus under FFA.

FFA: What does this do?



- So it turns out, FFA does work.
- It works because it prefers solutions with rare objective values.
- Solutions with very good objective values are also very rare on any problem worth tackling.
- FFA makes the simple $(1 + 1)$ EA slower on problems that it can easily solve.
- The slowdown is roughly proportional to the number of possible different objective values.
- On some non- $\mathcal{NP}$ -hard problems for which the $(1 + 1)$ EA needs **exponential** runtime, the $(1 + 1)$ FEA needs **polynomial** mean runtime.
- But not for all, because plateaus of the objective are still plateaus under FFA.
- FFA very significantly speeds up the $(1 + 1)$ EA on the $\mathcal{NP}$ -complete MaxSAT problem.

FFA: Let's get weird!



Let's get weird!



FFA: Let's get weird!



- Let's say you have an optimization problem with objective function $f(x)$.

FFA: Let's get weird!



- Let's say you have an optimization problem with objective function $f(x)$.
- You **encrypt** the objective values and do not tell them to the algorithm.

FFA: Let's get weird!



- Let's say you have an optimization problem with objective function $f(x)$.
- You **encrypt** the objective values and do not tell them to the algorithm.
- Let's say you apply AES^{1,27}, RSA^{83,115}, or the Caesar cipher^{72,115} as a function $g : \mathbb{N}_0 \mapsto \mathbb{N}_0$ to f , i.e., do $g \circ f$.

FFA: Let's get weird!



- Let's say you have an optimization problem with objective function $f(x)$.
- You **encrypt** the objective values and do not tell them to the algorithm.
- Let's say you apply AES^{1,27}, RSA^{83,115}, or the Caesar cipher^{72,115} as a function $g : \mathbb{N}_0 \mapsto \mathbb{N}_0$ to f , i.e., do $g \circ f$.
- Encryption removes any order, correlation or causality information, i.e., $g \circ f$ does not correlate with f in any way.

FFA: Let's get weird!



- Let's say you have an optimization problem with objective function $f(x)$.
- You **encrypt** the objective values and do not tell them to the algorithm.
- Let's say you apply AES^{1,27}, RSA^{83,115}, or the Caesar cipher^{72,115} as a function $g : \mathbb{N}_0 \mapsto \mathbb{N}_0$ to f , i.e., do $g \circ f$.
- Encryption removes any order, correlation or causality information, i.e., $g \circ f$ does not correlate with f in any way.
- $g(f(x))$ will be totally random.

FFA: Let's get weird!



- Let's say you have an optimization problem with objective function $f(x)$.
- You **encrypt** the objective values and do not tell them to the algorithm.
- Let's say you apply AES^{1,27}, RSA^{83,115}, or the Caesar cipher^{72,115} as a function $g : \mathbb{N}_0 \mapsto \mathbb{N}_0$ to f , i.e., do $g \circ f$.
- Encryption removes any order, correlation or causality information, i.e., $g \circ f$ does not correlate with f in any way.
- $g(f(x))$ will be totally random. No reasonable optimization algorithm could work with that.

FFA: Let's get weird!



- Let's say you have an optimization problem with objective function $f(x)$.
- You **encrypt** the objective values and do not tell them to the algorithm.
- Let's say you apply AES^{1,27}, RSA^{83,115}, or the Caesar cipher^{72,115} as a function $g : \mathbb{N}_0 \mapsto \mathbb{N}_0$ to f , i.e., do $g \circ f$.
- Encryption removes any order, correlation or causality information, i.e., $g \circ f$ does not correlate with f in any way.
- $g(f(x))$ will be totally random. No reasonable optimization algorithm could work with that.
- If the $(1 + 1)$ FEA can find the optimum of $f(x)$...

FFA: Let's get weird!



- Let's say you have an optimization problem with objective function $f(x)$.
- You **encrypt** the objective values and do not tell them to the algorithm.
- Let's say you apply AES^{1,27}, RSA^{83,115}, or the Caesar cipher^{72,115} as a function $g : \mathbb{N}_0 \mapsto \mathbb{N}_0$ to f , i.e., do $g \circ f$.
- Encryption removes any order, correlation or causality information, i.e., $g \circ f$ does not correlate with f in any way.
- $g(f(x))$ will be totally random. No reasonable optimization algorithm could work with that.
- If the $(1 + 1)$ FEA can find the optimum of $f(x)$...
- ...then it will find exactly the same solution in exactly the same runtime even if you apply it to the encrypted problem $g(f(x))$!

FFA: Let's get weird!



- Let's say you have an optimization problem with objective function $f(x)$.
- You **encrypt** the objective values and do not tell them to the algorithm.
- Let's say you apply AES^{1,27}, RSA^{83,115}, or the Caesar cipher^{72,115} as a function $g : \mathbb{N}_0 \mapsto \mathbb{N}_0$ to f , i.e., do $g \circ f$.
- Encryption removes any order, correlation or causality information, i.e., $g \circ f$ does not correlate with f in any way.
- $g(f(x))$ will be totally random. No reasonable optimization algorithm could work with that.
- If the $(1 + 1)$ FEA can find the optimum of $f(x)$...
- ...then it will find exactly the same solution in exactly the same runtime even if you apply it to the encrypted problem $g(f(x))$!
- How incredibly strange is that?



Where did we try FFA?



Applications of FFA



- Often, when some new type of algorithm is designed, one is tempted to pick some special, niche problems as application area.



Applications of FFA



- Often, when some new type of algorithm is designed, one is tempted to pick some special, niche problems as application area.
- Then, the competition is not so high and it's easier to get some good papers out.

Applications of FFA



- Often, when some new type of algorithm is designed, one is tempted to pick some special, niche problems as application area.
- Then, the competition is not so high and it's easier to get some good papers out.
- We did not do this.

Applications of FFA



- Often, when some new type of algorithm is designed, one is tempted to pick some special, niche problems as application area.
- Then, the competition is not so high and it's easier to get some good papers out.
- We did not do this.
- We picked some of the most popular $\mathcal{NP}$ -hard problems as testbeds.

Applications of FFA



- Often, when some new type of algorithm is designed, one is tempted to pick some special, niche problems as application area.
- Then, the competition is not so high and it's easier to get some good papers out.
- We did not do this.
- We picked some of the most popular $\mathcal{NP}$ -hard problems as testbeds.
- We could not (yet) win against the state-of-the-art that matured over decades.

Applications of FFA



- Often, when some new type of algorithm is designed, one is tempted to pick some special, niche problems as application area.
- Then, the competition is not so high and it's easier to get some good papers out.
- We did not do this.
- We picked some of the most popular $\mathcal{NP}$ -hard problems as testbeds.
- We could not (yet) win against the state-of-the-art that matured over decades.
- But we did get some interesting results on real, hard, and well-known problems.

Maximum Satisfiability Problem



- FFA on the Maximum Satisfiability (MaxSAT) Problem: [106, 108, 109].



Maximum Satisfiability Problem



- FFA on the Maximum Satisfiability (MaxSAT) Problem: [106, 108, 109].

From Table I, we can see that **the highest number of failed runs at scale $s = 250$ of any algorithm using FFA is lower than the lowest number of failed runs of any pure algorithm at $s = 50$.** From Table II, we find that no FFA-based algorithm has a higher ERT at scale $s = 250$ than its pure variant on $s = 50$. On the scales $s \leq 75$, the FFA-based algorithms have a mean runtime which is between three and four orders of magnitude smaller than the ERT of the pure algorithms.

- Snippet of page 10 of [109] (copyright IEEE).
- Several different EAs with and without FFA
- Budget: 10^{10} FEs

Traveling Salesperson Problem



- FFA on the Traveling Salesperson Problem (TSP): [59–61].

Traveling Salesperson Problem

- FFA on the Traveling Salesperson Problem (TSP): [59–61].
- Snippet of page 12 of [59] (copyright Springer).
- $(1 + 1)$ EA, $(1 + 1)$ FEA, SA w/o FFA, hybrids
- Budget: 10^{10} FEs

In this work, we explored both the EA and SA on 56 symmetric instances from the benchmark set TSPLib (Reinelt 1991, 1995). The EA is unsuitable for this problem, but SA can find the optimum on many small and mid-sized instances. We then plug FFA into both algorithms and obtain the FEA and the FSA, respectively, which both exhibit very similar performance. **The FEA solves 27 of the instances in all of its runs, which SA can only achieve for 19 instances.** Plugging FFA into either the EA or SA thus substantially improved the number of runs in which the algorithms can find the optimum, however, both FFA-based variants suffer when the number of unique objective values is high.

• • •

Both types of **hybridization** methods significantly improve the average result quality compared to both the objective-guided and FFA-based FEA variants. **The SAFEA_A discovers the optimal solutions in more runs than any other algorithm setup in our study.** It also finds the optimum most often in most instances and delivers the best approximation quality on most instances, compared to the other algorithms.

Quadratic Assignment Problem



- FFA on the Quadratic Assignment Problem (QAP): [15, 91].

Quadratic Assignment Problem

- FFA on the Quadratic Assignment Problem (QAP): [15, 91].
- Snippet of page 7 of [15] (copyright SciTePress).
- RLS w/o FFA
- Budget: 10^8 FEs

We find that the **FFA-based** randomized local search **FRLS does not just find better solutions than the objective-guided RLS algorithm on the vast majority of the QAPLIB instances**, it also keeps improving its current best solution for the complete computational budget of 10^8 FEs that we assigned to the runs. With this budget, **it can discover the optimal solutions of over 58% of the QAPLIB instances**. Had we assigned a larger budget – (Liang et al., 2022; Liang et al., 2024; Weise et al., 2021b; Weise et al., 2023) use 10^{10} FEs – we would likely have seen even more instances solved.

We furthermore confirm the remarkable ability of FFA to discover very diverse solutions (at least from the perspective of the objective function). It is known that on the QAP, many solutions tend to have the same objective values (Tayarani-N. and Prügel-Bennett, 2015). Yet, on some of the instances, more than half of the objective values discovered by FRLS were unique.

Job Shop Scheduling Problem



- FFA on the Job Shop Scheduling Problem (JSSP) Problem: [20, 99, 108].

Job Shop Scheduling Problem



- FFA on the Job Shop Scheduling Problem (JSSP) Problem: [20, 99, 108].

The **end result quality delivered by (1+1)-FEA is better in average on the abz*, ft*, la*, orb*, and yn4* instance sets, both in terms of *best* and *mean*.** On swv*, the average for *mean* is better for (1+1)-FEA, while (1+1)-EA has a slight lead in *best*. The (1+1)-EA performs better on the dmu* and ta* instances. Since these two sets are larger (holding 160 out of the 242 instances), the (1+1)-EA comes out ahead in the overall averages, but with no more than a 1.5% advantage.

- Snippet of page 10 of [99] (copyright ACM).
- Budget: $2^{30} \approx 10^9$ FEs

Algorithm Synthesis and Genetic Programming



- FFA for algorithm synthesis and Genetic Programming (GP): [105, 106].

Algorithm Synthesis and Genetic Programming



- FFA for algorithm synthesis and Genetic Programming (GP): [105, 106].

Fourth, we were able to confirm with great significance that **FFA** has a tremendous positive impact on the performance of GP, as it **can increase the success rate by 40%**.

- Snippet of page 7 of [105] (copyright IEEE).
- Budget: $\approx 100\,000$ FEs

Where FFA did not work



- FFA cannot work well on problems where local search already gives good results.

Where FFA did not work



- FFA cannot work well on problems where local search already gives good results.
- This includes, surprisingly, also some computationally hard problems.

Where FFA did not work



- FFA cannot work well on problems where local search already gives good results.
- This includes, surprisingly, also some computationally hard problems:
 - the Traveling Tournament Problem (TTP), where RLS can find surprisingly good results: [112].

Where FFA did not work



- FFA cannot work well on problems where local search already gives good results.
- This includes, surprisingly, also some computationally hard problems:
 - the Traveling Tournament Problem (TTP), where RLS can find surprisingly good results: [112].
 - the two-dimensional bin packing problem, where RLS can actually **outperform** several complex state-of-the-art metaheuristics [117, 118].

Where FFA did not work



- FFA cannot work well on problems where local search already gives good results.
- This includes, surprisingly, also some computationally hard problems:
 - the Traveling Tournament Problem (TTP), where RLS can find surprisingly good results: [112].
 - the two-dimensional bin packing problem, where RLS can actually **outperform** several complex state-of-the-art metaheuristics [117, 118].
 - FFA can get trapped too!

Where FFA did not work



- FFA cannot work well on problems where local search already gives good results.
- This includes, surprisingly, also some computationally hard problems:
 - the Traveling Tournament Problem (TTP), where RLS can find surprisingly good results: [112].
 - the two-dimensional bin packing problem, where RLS can actually **outperform** several complex state-of-the-art metaheuristics [117, 118].
 - FFA can get trapped too: If all neighbors that can be reached with one search step around one solution all have the same objective value, then FFA-guided search may not be able to escape from that solution¹¹⁶.

Where FFA did not work



- FFA cannot work well on problems where local search already gives good results.
- This includes, surprisingly, also some computationally hard problems:
 - the Traveling Tournament Problem (TTP), where RLS can find surprisingly good results: [112].
 - the two-dimensional bin packing problem, where RLS can actually **outperform** several complex state-of-the-art metaheuristics [117, 118].
 - FFA can get trapped too: If all neighbors that can be reached with one search step around one solution all have the same objective value, then FFA-guided search may not be able to escape from that solution¹¹⁶. This can happen with 1-bit flips on OneMax¹¹⁶.



Summary



Summary



- Frequency Fitness Assignment (FFA) is an algorithm module that can be plugged into existing algorithms.

Summary



- Frequency Fitness Assignment (FFA) is an algorithm module that can be plugged into existing algorithms.
- It renders algorithms invariant under all injective transformations of the objective function value.

Summary



- Frequency Fitness Assignment (FFA) is an algorithm module that can be plugged into existing algorithms.
- It renders algorithms invariant under all injective transformations of the objective function value.
- It makes them optimize without bias for good solutions.

Summary



- Frequency Fitness Assignment (FFA) is an algorithm module that can be plugged into existing algorithms.
- It renders algorithms invariant under all injective transformations of the objective function value.
- It makes them optimize without bias for good solutions.
- It slows them down on easy problems.

Summary



- Frequency Fitness Assignment (FFA) is an algorithm module that can be plugged into existing algorithms.
- It renders algorithms invariant under all injective transformations of the objective function value.
- It makes them optimize without bias for good solutions.
- It slows them down on easy problems.
- It can speed them up on hard problems or lead them to solution qualities they otherwise cannot reach.

Summary



- Frequency Fitness Assignment (FFA) is an algorithm module that can be plugged into existing algorithms.
- It renders algorithms invariant under all injective transformations of the objective function value.
- It makes them optimize without bias for good solutions.
- It slows them down on easy problems.
- It can speed them up on hard problems or lead them to solution qualities they otherwise cannot reach.
- It is limited to objective functions that cannot take on too many different objective values.



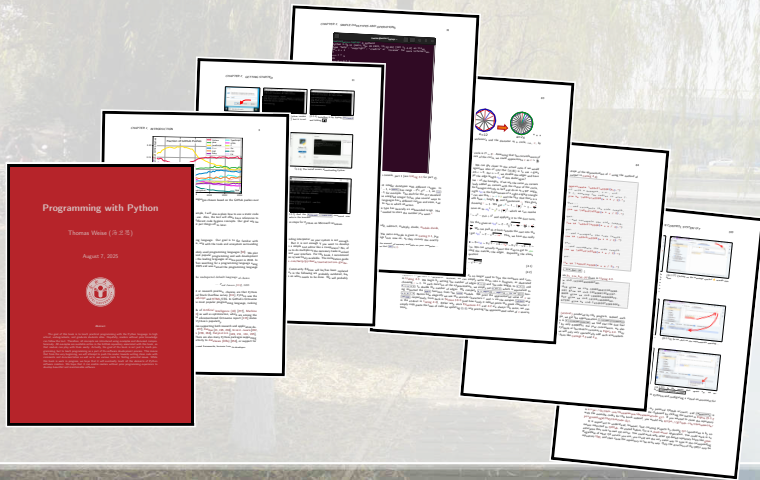
Advertisement



Programming with Python



We have a freely available course book on *Programming with Python* at <https://thomasweise.github.io/programmingWithPython>, with focus on practical software development using the Python ecosystem of tools⁹⁷.



Databases



We have a freely available course book on *Databases* at <https://thomasweise.github.io/databases>, with actual practical examples using a real database management system (DBMS)⁹⁴.



ization,

The image is a collage of screenshots from a web browser, primarily showing the moptipy GitHub repository and its documentation. The main screenshot displays the 'Table of Contents' for 'moptipy: Metaheuristic Optimization in Python'. The table lists sections such as '1. Introduction', '2. Installation', and '3. How-To'. Below the table, there is a section titled '1. Introduction' which describes the library's purpose and its educational/research focus. Other smaller screenshots show code snippets for running experiments, setting up the environment, and a progress plot showing fitness values over iterations.

Table of Contents

- moptipy: Metaheuristic Optimization in Python
- 1. Introduction
- 2. Installation
- 3. How-To
- 3.1. How to Apply 1 Optimization Algorithm Once to 1 Problem Instance
- 3.2. How to Run a Series of Experiments
- 3.3. How to Solve an Optimization Problem
- 3.3.1. Define a New Problem Type
- 3.3.2. Define a New Algorithm
- 3.3.3. Applying an Own Algorithm to an Own Problem
- 4. Implemented Algorithms, Search Spaces, and Problems
- 4.1. Implemented Algorithms
- 4.1.1. Single-Transfering data from thomawise.github.io

1. Introduction

moptipy is a library with implementations of metaheuristic optimization methods in Python 3.12 that also offers an environment for replicable experiments. The framework, *algorithms*, and the library design are accompanied by the book *Optimization Algorithms*. The library is structured with performance, ease-of-use, and generality in mind, but also based on an educational and research perspective. It is therefore hopefully suitable for practical industrial applications, scientific research, and for students who are just entering the field of metaheuristic optimization.

Metaheuristic optimization algorithms are methods for solving hard problems. *moptipy* provides an API, several *algorithm implementations*, as well as *experiment execution* and *evaluation* facilities for metaheuristics.

A metaheuristic algorithm can be a black-box method, which can solve problems without deeper knowledge about their nature. Such a black-box algorithm only requires methods to *create* and *modify* points in the search space and to *evaluate* their quality. With these operations, it will try to discover better solutions step-by-step. Black-box metaheuristics are very *general* and can be *adapted* to *almost any optimization problem*. They *allow us to plan* in al-

Code Snippets:

```

# Create a new problem instance
problem = Problem(...)

# Create a new algorithm instance
algorithm = Algorithm(...)

# Run the algorithm
result = algorithm.run(problem)

# Evaluate the result
fitness = result.fitness
  
```

Progress Plot:

Progress plots are implemented in the module `moptipy.evaluation.plot_progress`.

The plot shows fitness values (Y-axis, ranging from 0.0 to 10.0) versus iterations (X-axis, ranging from 0 to 200). The plot displays the progress of the optimization process, with a blue line representing the fitness value and a red line representing the best fitness value. The fitness value starts at approximately 10.0 and decreases to about 2.0, while the best fitness value starts at approximately 8.0 and decreases to about 1.0.





谢谢你们！
Thank you!
Vielen Dank!



References I

- [1] *Advanced Encryption Standard (AES)*. Federal Information Processing Standards Publication (FIPS PUB) 197-upd1. Gaithersburg, MD, USA: U.S. Department of Commerce, National Institute of Standards and Technology (NIST), May 9, 2023. doi:10.6028/NIST.FIPS.197-upd1. URL: <https://csrc.nist.gov/pubs/fips/197/final> (visited on 2025-08-17) (cit. on pp. 229–238, 278).
- [2] Denis Antipov and Benjamin Doerr. "Precise Runtime Analysis for Plateaus". In: *15th International Conference on Parallel Problem Solving from Nature (PPSN XV)*. Vol. 2. Sept. 8–12, 2018, Coimbra, Portugal. Ed. by Anne Auger, Carlos M. Fonseca, Nuno Lourenço, Penousal Machado, Luís Paquete, and L. Darrell Whitley. Vol. 11102 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2018, pp. 117–128. ISSN: 0302-9743. ISBN: 978-3-319-99258-7. doi:10.1007/978-3-319-99259-4_10 (cit. on pp. 210–213, 295).
- [3] David Lee Applegate, Robert E. Bixby, Vašek Chvátal, and William John Cook. *The Traveling Salesman Problem: A Computational Study*. 2nd ed. Vol. 17 of Princeton Series in Applied Mathematics. Princeton, NJ, USA: Princeton University Press, 2007. ISBN: 978-0-691-12993-8. This book also discusses the Concorde TSP solver. (Cit. on pp. 13–16, 292, 297).
- [4] Paul Gustav Heinrich Bachmann. *Die Analytische Zahlentheorie / Dargestellt von Paul Bachmann*. Vol. Zweiter Theil of Zahlentheorie: Versuch einer Gesamtdarstellung dieser Wissenschaft in ihren Haupttheilen. Leipzig, Sachsen, Germany: B. G. Teubner, 1894. ISBN: 978-1-4181-6963-3. URL: <http://gallica.bnf.fr/ark:/12148/bpt6k994750> (visited on 2023-12-13) (cit. on p. 300).
- [5] Thomas Bäck, David B. Fogel, and Zbigniew "Zbyszek" Michalewicz, eds. *Handbook of Evolutionary Computation*. Bristol, England, UK: IOP Publishing Ltd and Oxford, Oxfordshire, England, UK: Oxford University Press, 1997. ISBN: 978-0-7503-0392-7 (cit. on pp. 68–82, 293, 296).
- [6] Thomas Bartz-Beielstein, Carola Doerr, Daan van den Berg, Jakob Bossek, Sowmya Chandrasekaran, Tome Eftimov, Andreas Fischbach, Pascal Kerschke, William La Cava, Manuel López-Ibáñez, Katherine Mary Malan, Jason Hall Moore, Boris Naujoks, Patryk Orzechowski, Vanessa Volz, Markus Wagner, and Thomas Weise (汤卫思). "Benchmarking in Optimization: Best Practice and Open Issues". (abs/2007.03488), Dec. 18, 2020. doi:10.48550/arXiv.2007.03488. URL: <https://arxiv.org/abs/2007.03488> (visited on 2025-07-25). arXiv:2007.03488v2 [cs.NE] 16 Dec 2020 (cit. on p. 293).
- [7] Tim Berners-Lee. *Re: Qualifiers on Hypertext links...*. Geneva, Switzerland: World Wide Web project, European Organization for Nuclear Research (CERN) and Newsgroups: alt.hypertext, Aug. 6, 1991. URL: <https://www.w3.org/People/Berners-Lee/1991/08/art-6484.txt> (visited on 2025-02-05) (cit. on p. 298).

References II



- [8] Tim Berners-Lee, Larry Masinter, and Mark P. McCahill. *Uniform Resource Locators (URL)*. Request for Comments (RFC) 1738. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Dec. 1994. URL: <https://www.ietf.org/rfc/rfc1738.txt> (visited on 2025-02-05) (cit. on p. 298).
- [9] Jacek Błażewicz, Wolfgang Domschke, and Erwin Pesch. "The Job Shop Scheduling Problem: Conventional and New Solution Techniques". *European Journal of Operational Research* 93(1):1–33, Aug. 1996. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(95)00362-2 (cit. on p. 293).
- [10] Joshua Bloch. *Effective Java*. Reading, MA, USA: Addison-Wesley Professional, May 2008. ISBN: 978-0-321-35668-0 (cit. on p. 293).
- [11] Rainer E. Burkard, Eranda Çela, Panos Miltiades Pardalos, and Leonidas S. Pitsoulis. "The Quadratic Assignment Problem". In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1713–1809. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_27 (cit. on p. 296).
- [12] Eduardo Carvalho Pinto and Carola Doerr. *Towards a More Practice-Aware Runtime Analysis of Evolutionary Algorithms*. arXiv.org: Computing Research Repository (CoRR) abs/1812.00493. Ithaca, NY, USA: Cornell University Library, Dec. 3, 2018. doi:10.48550/arXiv.1812.00493. URL: <https://arxiv.org/abs/1812.00493> (visited on 2025-08-08). arXiv:1812.00493v1 [cs.NE] 3 Dec 2018 (cit. on pp. 44–54, 292).
- [13] Vladimír Černý. "Thermodynamical Approach to the Traveling Salesman Problem: An Efficient Simulation Algorithm". *Journal of Optimization Theory and Applications* 45(1):41–51, Jan. 1985. New York, NY, USA: Springer Science+Business Media, LLC. ISSN: 0022-3239. doi:10.1007/BF00940812 (cit. on p. 296).
- [14] Bo Chen, Chris N. Potts, and Gerhard J. Woeginger. "A Review of Machine Scheduling: Complexity, Algorithms and Approximability". In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1493–1641. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_25. See also pages 21–169 in volume 3/3 by Norwell, MA, USA: Kluwer Academic Publishers. (Cit. on pp. 293, 299).

References III



- [15] Jiayang Chen (陈嘉阳), Zhize Wu (吴志泽), Sarah Louise Thomson, and Thomas Weise (汤卫恩). "Frequency Fitness Assignment: Optimization Without Bias for Good Solution Outperforms Randomized Local Search on the Quadratic Assignment Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 27–37. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888600003837 (cit. on pp. 250, 251, 293, 295, 296).
- [16] Philippe Chrétienne, Edward G. Coffman, Jan Karel Lenstra, and Zhen Liu, eds. *Scheduling Theory and Its Applications*. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., Sept. 1995. ISBN: 978-0-471-94059-3 (cit. on p. 293).
- [17] Stephen Arthur Cook. "The Complexity of Theorem-Proving Procedures". In: *Third Annual ACM Symposium on Theory of Computing (STOC'1971)*. May 3–5, 1971, Shaker Heights, OH, USA. Ed. by Michael A. Harrison, Ranajit B. Banerji, and Jeffrey D. Ullman. New York, NY, USA: Association for Computing Machinery (ACM), 1971, pp. 151–158. ISBN: 978-1-4503-7464-4. doi:10.1145/800157.805047 (cit. on pp. 17–24, 214–218, 294, 299).
- [18] William John Cook. *Concorde TSP Solver*. Waterloo, ON, Canada: University of Waterloo, 1997. URL: <https://math.uwaterloo.ca/tsp/concorde> (visited on 2026-09-26) (cit. on p. 292).
- [19] William John Cook. *In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation*. Princeton, NJ, USA: Princeton University Press, 2012–Nov. 9, 2014. ISBN: 978-0-691-16352-9 (cit. on p. 292).
- [20] Ege de Bruin, Sarah Louise Thomson, and Daan van den Berg. "Frequency Fitness Assignment on JSSP: A Critical Review". In: *26th European Conference on Applications of Evolutionary Computation (EvoApplications'2023), Held as Part of EvoStar'2023*. Apr. 12–14, 2023, Brno, Czech Republic. Ed. by João Correia, Stephen Smith, and Raneem Qaddoura. Vol. 13989 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2023, pp. 351–363. ISSN: 0302-9743. ISBN: 978-3-031-30228-2. doi:10.1007/978-3-031-30229-9_23. Independent reproduction of ⁹⁹ (cit. on pp. 252, 253).
- [21] Kenneth Alan De Jong. *Evolutionary Computation: A Unified Approach*. Vol. 4 of Complex Adaptive Systems. Cambridge, MA, USA: MIT Press, 2006. ISBN: 978-0-262-04194-2. URL: <https://www.researchgate.net/publication/220740669> (visited on 2025-08-08) (cit. on pp. 68–82, 296).

References IV



- [22] Paul Deitel, Harvey Deitel, and Abbey Deitel. *Internet & World Wide Web: How to Program*. 5th ed. Hoboken, NJ, USA: Pearson Education, Inc., Nov. 2011. ISBN: 978-0-13-299045-5 (cit. on p. 298).
- [23] Slobodan Dimitrović. *Modern C for Absolute Beginners: A Friendly Introduction to the C Programming Language*. New York, NY, USA: Apress Media, LLC, Mar. 2024. ISBN: 979-8-8688-0224-9 (cit. on p. 292).
- [24] Benjamin Doerr, Carola Doerr, and Franziska Ebel. "From Black-Box Complexity to Designing New Genetic Algorithms". *Theoretical Computer Science* 567:87–104, Feb. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/J.TCS.2014.11.028 (cit. on pp. 193–195, 295).
- [25] Stefan Droste, Thomas Jansen, and Ingo Wegener. "On the Analysis of the (1 + 1) Evolutionary Algorithm". *Theoretical Computer Science* 276(1-2):51–81, Apr. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/S0304-3975(01)00182-7 (cit. on pp. 44–54, 149–159, 196–198, 202–209, 292, 294, 297).
- [26] Stefan Droste, Thomas Jansen, and Ingo Wegener. "Upper and Lower Bounds for Randomized Search Heuristics in Black-Box Optimization". *Theoretical Computer Science* 39(4):525–544, July 2006. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1007/S00224-004-1177-Z (cit. on pp. 105–110, 295).
- [27] Morris J. Dworkin, Elaine Barker, James R. Nechvatal, James Foti, Lawrence E. Bassham, E. Roback, and James F. Dray Jr. *Advanced Encryption Standard (AES)*. Federal Information Processing Standards Publication (FIPS PUB) 197. Gaithersburg, MD, USA: U.S. Department of Commerce, National Institute of Standards and Technology (NIST), Nov. 26, 2001. doi:10.6028/NIST.FIPS.197. URL: <https://www.nist.gov/publications/advanced-encryption-standard-aes> (visited on 2025-08-17). Updated/replaced by ¹ (cit. on pp. 229–238).
- [28] Harald Dyckhoff and Ute Finke. *Cutting and Packing in Production and Distribution: A Typology and Bibliography*. Contributions to Management Science. Heidelberg, Baden-Württemberg, Germany: Physica-Verlag, 1992. ISSN: 1431-1941. ISBN: 978-3-642-63487-1. doi:10.1007/978-3-642-58165-6 (cit. on pp. 4–9).
- [29] Kelly Easton, George L. Nemhauser, and Michael A. Trick. "The Traveling Tournament Problem Description and Benchmarks". In: *7th International Conference on Principles and Practice of Constraint Programming (CP'01)*. Nov. 26–Dec. 1, 2001, Paphos, Cyprus. Ed. by Toby Walsh. Vol. 2239 of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2001, pp. 580–584. ISSN: 0302-9743. ISBN: 978-3-540-42863-3. doi:10.1007/3-540-45578-7_43 (cit. on p. 298).

References V



- [30] Luca Ferrari and Enrico Pirozzi. *Learn PostgreSQL*. 2nd ed. Birmingham, England, UK: Packt Publishing Ltd, Oct. 2023. ISBN: **978-1-83763-564-1** (cit. on p. **295**).
- [31] Tobias Friedrich, Pietro Simone Oliveto, Dirk Sudholt, and Carsten Witt. "Analysis of Diversity-Preserving Mechanisms for Global Exploration". *Evolutionary Computation* 17(4):455–476, Win. 2009. Cambridge, MA, USA: MIT Press. ISSN: **1063-6560**. doi:**10.1162/evco.2009.17.4.17401** (cit. on pp. **199–201, 298**).
- [32] Tobias Friedrich, Francesco Quinzan, and Markus Wagner. "Escaping Large Deceptive Basins of Attraction with Heavy-Tailed Mutation Operators". In: *20th Genetic and Evolutionary Computation Conference (GECCO'2018)*. July 15–19, 2018, Kyoto, Japan. Ed. by Hernán E. Aguirre and Keiki Takadama. New York, NY, USA: Association for Computing Machinery (ACM), 2018, pp. 293–300. ISBN: **978-1-4503-5618-3**. doi:**10.1145/3205455.3205515** (cit. on pp. **199–201, 206–209, 294, 298**).
- [33] Michael R. Garey and David Stifler Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York, NY, USA: W. H. Freeman and Company, 1979. ISBN: **978-0-7167-1045-5** (cit. on p. **297**).
- [34] Fred Glover. "Tabu Search – Part I". *ORSA Journal on Computing* 1(3):190–206, Sum. 1989. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: **0899-1499**. doi:**10.1287/ijoc.1.3.190** (cit. on p. **297**).
- [35] Fred Glover. "Tabu Search – Part II". *ORSA Journal on Computing* 2(1):4–32, Win. 1990. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: **0899-1499**. doi:**10.1287/ijoc.2.1.4** (cit. on p. **297**).
- [36] Fred Glover and Gary A. Kochenberger, eds. *Handbook of Metaheuristics*. Vol. 57 of International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2003. ISSN: **0884-8289**. ISBN: **978-0-306-48056-0**. doi:**10.1007/b101874** (cit. on p. **294**).
- [37] Fred Glover, Éric D. Taillard, and Dominique de Werra. "A User's Guide to Tabu Search". *Annals of Operations Research* 41(1):3–28, Mar. 1993. London, England, UK: Springer Nature Limited. ISSN: **0254-5330**. doi:**10.1007/BF02078647**. Special issue on Tabu Search (TS). (Cit. on p. **297**).
- [38] David Edward Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Redwood City, CA, USA: Addison Wesley Longman Publishing Co., Inc., 1989. ISBN: **978-0-201-15767-3** (cit. on pp. **68–82, 296**).

References VI



- [39] Michael T. Goodrich. *A Gentle Introduction to NP-Completeness*. Irvine, CA, USA: University of California, Irvine, Apr. 2022. URL: <https://ics.uci.edu/~goodrich/teach/cs165/notes/NPComplete.pdf> (visited on 2025-08-01) (cit. on p. 299).
- [40] Gregory Z. Gutin and Abraham P. Punnen, eds. *The Traveling Salesman Problem and its Variations*. Vol. 12 of Combinatorial Optimization (COOP). New York, NY, USA: Springer New York, May 2002. ISSN: 1388-3011. doi:10.1007/b101971 (cit. on pp. 13–16, 297).
- [41] Nikolaus Hansen and Anne Auger. "Principled Design of Continuous Stochastic Search: From Theory to Practice". In: *Theory and Principled Methods for the Design of Metaheuristics*. Ed. by Yossi Borenstein and Alberto Moraglio. Natural Computing Series. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2014, pp. 145–180. ISSN: 1619-7127. ISBN: 978-3-642-33205-0. doi:10.1007/978-3-642-33206-7_8. URL: <https://inria.hal.science/hal-00808450/file/hansen2013principled.pdf> (visited on 2025-08-09) (cit. on pp. 90–95).
- [42] Holger H. Hoos and Thomas Stützle. *Stochastic Local Search: Foundations & Applications*. The Morgan Kaufmann Series in Artificial Intelligence. Amsterdam, The Netherlands: Elsevier B.V., 2005. ISBN: 978-1-55860-872-6. doi:10.1016/B978-1-55860-872-6.X5016-1 (cit. on pp. 17–24, 214–218, 294, 297).
- [43] Holger H. Hoos and Thomas Stützle. *SATLIB – The Satisfiability Library*. 1.4.4. Vancouver, BC, Canada: The University of British Columbia (UBC), May 11, 2001. URL: <https://www.cs.ubc.ca/~hoos/SATLIB/index-ubc.html> (visited on 2026-09-20). See paper ⁴⁴ (cit. on pp. 280, 296).
- [44] Holger H. Hoos and Thomas Stützle. "SATLIB: An Online Resource for Research on SAT". In: *SAT2000 – Highlights of Satisfiability Research in the Year 2000*. Ed. by Ian Philip Gent, Toby Walsh, and Hans van Maaren. Vol. 63 of Frontiers in Artificial Intelligence and Applications. Amsterdam, The Netherlands: IOS Press BV, Jan. 2000, pp. 283–292. ISSN: 0922-6389. ISBN: 978-1-58603-061-2. See the website ⁴³ (cit. on pp. 214–218, 280, 296).
- [45] John Hunt. *A Beginners Guide to Python 3 Programming*. 2nd ed. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2023. ISSN: 1863-7310. ISBN: 978-3-031-35121-1. doi:10.1007/978-3-031-35122-8 (cit. on p. 295).
- [46] *Information Technology – Universal Coded Character Set (UCS)*. International Standard ISO/IEC 10646:2020. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Dec. 2020 (cit. on p. 290).

References VII



- [47] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". In: *15th Annual Workshop on Microprogramming (MICRO 15)*. Oct. 5–7, 1982. Ed. by Joseph Allen Fisher, William J. Tracz, and Bill Hopkins. Palo Alto, CA, USA: Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and New York, NY, USA: Association for Computing Machinery (ACM), Oct. 1982, pp. 143–148. doi:10.5555/800036.800944. See ⁴⁸ (cit. on p. 281).
- [48] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". *ACM SIGMICRO Newsletter* 13(4):143–148, Dec. 1982. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 1050-916X. doi:10.1145/1014194.800944. See ⁴⁷ (cit. on pp. 281, 296).
- [49] Mohamed Jebalia and Anne Auger. "Log-Linear Convergence of the Scale-Invariant $(\mu/\mu_w, \lambda)$ -ES and Optimal μ for Intermediate Recombination for Large Population Sizes". In: *11th International Conference on Parallel Problem Solving from Nature (PPSN XI)*. Vol. 1. Sept. 11–15, 2010, Kraków, Poland. Ed. by Robert Schaefer, Carlos Cotta, Joanna Kolodziej, and Günter Rudolph. Vol. 6238 of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2010, pp. 52–62. ISSN: 0302-9743. ISBN: 978-3-642-15843-8. doi:10.1007/978-3-642-15844-5_6 (cit. on pp. 90–95).
- [50] Scott Kirkpatrick, C. Daniel Gelatt, Jr., and Mario P. Vecchi. "Optimization by Simulated Annealing". *Science Magazine* 220(4598):671–680, May 13, 1983. Washington, D.C., USA: American Association for the Advancement of Science (AAAS). ISSN: 0036-8075. doi:10.1126/science.220.4598.671. URL: <https://www.researchgate.net/publication/6026283> (visited on 2025-08-08) (cit. on pp. 55–67, 296).
- [51] Donald Ervin Knuth. "Big Omicron and Big Omega and Big Theta". *ACM SIGACT News* 8(2):18–24, Apr.–June 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0163-5700. doi:10.1145/1008328.1008329 (cit. on pp. 299, 300).
- [52] Donald Ervin Knuth. *Fundamental Algorithms*. 3rd ed. Vol. 1 of The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, 1997. ISBN: 978-0-201-89683-1 (cit. on pp. 299, 300).
- [53] Tjalling Charles Koopmans and Martin Beckmann. "Assignment Problems and the Location of Economic Activities". *Econometrica* 25(1):53–76, 1957. New Haven, CT, USA: The Econometric Society and Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0012-9682. doi:10.2307/1907742 (cit. on p. 296).
- [54] John R. Koza. *Genetic Programming – On the Programming of Computers by Means of Natural Selection*. Complex Adaptive Systems. Cambridge, MA, USA: MIT Press, 1993. ISBN: 978-0-262-11170-6 (cit. on p. 293).

References VIII



- [55] Edmund Landau. *Handbuch der Lehre von der Verteilung der Primzahlen*. Leipzig, Sachsen, Germany: B. G. Teubner, 1909. ISBN: 978-0-8218-2650-8 (cit. on p. 300).
- [56] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. "Sequencing and Scheduling: Algorithms and Complexity". In: *Production Planning and Inventory*. Ed. by Stephen C. Graves, Alexander Hendrik George Rinnooy Kan, and Paul H. Zipkin. Vol. IV of Handbooks of Operations Research and Management Science. Amsterdam, The Netherlands: Elsevier B.V., 1993. Chap. 9, pp. 445–522. ISSN: 0927-0507. ISBN: 978-0-444-87472-6. doi:10.1016/S0927-0507(05)80189-6. URL: <http://alexandria.tue.nl/repository/books/339776.pdf> (visited on 2023-12-06) (cit. on pp. 293, 299).
- [57] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Estimation, Simulation, and Control – Wiley-Interscience Series in Discrete Mathematics and Optimization. Chichester, West Sussex, England, UK: Wiley Interscience, Sept. 1985. ISSN: 0277-2698. ISBN: 978-0-471-90413-7 (cit. on pp. 13–16, 297).
- [58] Kent D. Lee and Steve Hubbard. *Data Structures and Algorithms with Python*. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2015. ISSN: 1863-7310. ISBN: 978-3-319-13071-2. doi:10.1007/978-3-319-13072-9 (cit. on p. 295).
- [59] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, Sarah Louise Thomson, and Thomas Weise (汤卫思). "Addressing the Traveling Salesperson Problem with Frequency Fitness Assignment and Hybrid Algorithms". *Soft Computing* 28(17-18):9495–9508, July 2024. London, England, UK: Springer Nature Limited. ISSN: 1432-7643. doi:10.1007/S00500-024-09718-8 (cit. on pp. 13–16, 248, 249, 293, 295, 297).
- [60] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, and Thomas Weise (汤卫思). "Solving the Traveling Salesperson Problem using Frequency Fitness Assignment". In: *IEEE Symposium Series on Computational Intelligence (SSCI'2022)*. Dec. 4–7, 2022, Singapore. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022. ISBN: 978-1-6654-8769-6. doi:10.1109/SSCI51031.2022.10022296 (cit. on pp. 248, 249, 293, 295).

References IX



- [61] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Matthias Thüerer, Markus Wagner, and Thomas Weise (汤卫恩). "Generating Small Instances with Interesting Features for the Traveling Salesperson Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 173–180. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888800003837 (cit. on pp. 248, 249, 295).
- [62] Eliane Maria Loiola, Nair Maria Maia de Abreu, Paulo Oswaldo Boaventura-Netto, Peter M. Hahn, and Tania Querido. "A Survey for the Quadratic Assignment Problem". *European Journal of Operational Research* 176(2):657–690, 2007. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2005.09.032 (cit. on p. 296).
- [63] Marc Loy, Patrick Niemeyer, and Daniel Leuck. *Learning Java*. 5th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2020. ISBN: 978-1-4920-5627-0 (cit. on p. 293).
- [64] Mark Lutz. *Learning Python*. 6th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2025. ISBN: 978-1-0981-7130-8 (cit. on p. 295).
- [65] Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe, eds. *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0000195000003837.
- [66] Zbigniew "Zbyszek" Michalewicz. *Genetic Algorithms + Data Structures = Evolution Programs*. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 1996. ISBN: 978-3-540-58090-4. doi:10.1007/978-3-662-03315-9 (cit. on pp. 68–82, 296).
- [67] Melanie Mitchell. *An Introduction to Genetic Algorithms*. Complex Adaptive Systems. Cambridge, MA, USA: MIT Press, Feb. 1998. ISBN: 978-0-262-13316-6. URL: <http://boente.eti.br/fuzzy/ebook-fuzzy-mitchell.pdf> (visited on 2025-08-08) (cit. on pp. 68–82, 296).
- [68] Heinz Mühlenbein. "How Genetic Algorithms Really Work: Mutation and Hillclimbing". In: *Parallel Problem Solving from Nature 2 (PPSN-II)*. Sept. 28–30, 1992, Brussels, Belgium. Ed. by Reinhard Männer and Bernard Manderick. Amsterdam, The Netherlands: Elsevier B.V., 1992, pp. 15–26. URL: <https://www.researchgate.net/publication/284805798> (visited on 2025-08-15) (cit. on pp. 105–110, 193–195, 295).

References X



- [69] Siegfried Nijssen and Thomas Bäck. "An Analysis of the Behavior of Simplified Evolutionary Algorithms on Trap Functions". *IEEE Transactions on Evolutionary Computation* 7(1):11–22, Feb. 2003. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2002.806169 (cit. on pp. 149–154, 202–205, 297).
- [70] Regina O. Obe and Leo S. Hsu. *PostgreSQL: Up and Running*. 3rd ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Oct. 2017. ISBN: 978-1-4919-6336-4 (cit. on p. 295).
- [71] Yann Ollivier, Ludovic Arnold, Anne Auger, and Nikolaus Hansen. "Information-Geometric Optimization Algorithms: A Unifying Picture via Invariance Principles". *Journal of Machine Learning Research (JMLR)* 18(18):1–65, Apr. 2017. Cambridge, MA, USA: MIT Press. ISSN: 1532-4435. URL: <https://jmlr.org/papers/volume18/14-467/14-467.pdf> (visited on 2025-08-09) (cit. on pp. 90–95).
- [72] Richard E. Overill. "Codes and Ciphers: Julius Caesar, The Enigma, and the Internet". *Journal of Logic and Computation* 12(3):543, June 2002. Oxford, Oxfordshire, England, UK: Oxford University Press. ISSN: 0955-792X. doi:10.1093/LOGCOM/12.3.543 (cit. on pp. 229–238).
- [73] Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham, eds. *Handbook of Combinatorial Optimization*. 1st ed. Boston, MA, USA: Springer, 1998. ISBN: 978-1-4613-7987-4.
- [74] Martin Pincus. "Letter to the Editor – A Monte Carlo Method for the Approximate Solution of Certain Types of Constrained Optimization Problems". *Operations Research* 18(6):1225–1228, Nov.–Dec. 1970. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0030-364X. doi:10.1287/opre.18.6.1225 (cit. on p. 296).
- [75] Alexander Podlich, Thomas Weise (汤卫恩), Manfred Menze, and Christian Gorltdt. "Intelligente Wechselbrückensteuerung für die Logistik von Morgen". *Electronic Communications of the EASST (ECEASST)* 17(Communication in Distributed Systems), Feb. 27, 2009. The Netherlands: European Association of Software Science and Technology (EASST). ISSN: 1863-2122. doi:10.14279/tuj.eceasst.17.205. In the proceedings of *Workshops der Wissenschaftlichen Konferenz Kommunikation in Verteilten Systemen 2009* (WowKiVS 2009), 3 6, 2009, Kassel, Hessen, Germany (cit. on pp. 4–9).
- [76] Riccardo Poli, William B. Langdon, and Nicholas Freitag McPhee. *A Field Guide to Genetic Programming*. lulu.com, 2008. ISBN: 978-1-4092-0073-4. URL: <https://www.researchgate.net/publication/216301261> (visited on 2025-08-18) (cit. on p. 293).
- [77] *PostgreSQL Essentials: Leveling Up Your Data Work*. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2024 (cit. on p. 295).

References XI



- [78] *Programming Languages – C, Working Document of SC22/WG14*. International Standard ISO/3IEC9899:2017 C17 Ballot N2176. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Nov. 2017. URL: <https://files.lhmouse.com/standards/ISO%20C%20N2176.pdf> (visited on 2024-06-29) (cit. on p. 292).
- [79] Sanatan Rai and George Vairaktarakis. “NP-Complete Problems and Proof Methodology”. In: *Encyclopedia of Optimization*. Ed. by Christodoulos A. Floudas and Panos Miltiades Pardalos. 2nd ed. Boston, MA, USA: Springer, Sept. 2008, pp. 2675–2682. ISBN: 978-0-387-74758-3. doi:10.1007/978-0-387-74759-0_462 (cit. on p. 299).
- [80] Soraya B. Rana and L. Darrell Whitley. “Genetic Algorithm Behavior in the MAXSAT Domain”. In: *5th International Conference on Parallel Problem Solving from Nature (PPSN V)*. Sept. 27–30, 1998, Amsterdam, The Netherlands. Ed. by Ágoston Endre Eiben, Thomas Bäck, Marc Schoenauer, and Hans-Paul Schwefel. Lecture Notes in Computer Science (LNCS). 1998, pp. 785–794. ISSN: 0302-9743. ISBN: 978-3-540-65078-2. doi:10.1007/BFb0056920 (cit. on p. 294).
- [81] Gerhard Reinelt. “TSPLIB – A Traveling Salesman Problem Library”. *ORSA Journal on Computing* 3(4):376–384, Aut. 1991. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.3.4.376 (cit. on p. 297).
- [82] Gerhard Reinelt. *TSPLIB95*. Tech. rep. Heidelberg, Baden-Württemberg, Germany: Universität Heidelberg, Institut für Angewandte Mathematik, 1995–Apr. 28, 2016. URL: <https://github.com/shredderzwj/TSPLIB> (visited on 2026-09-25). The original Uniform Resource Locator (URL) of the repository has become defunct in 2026. Therefore, we provide the repository listed as main URL which contains the data. Additionally, you may also check <https://github.com/thomasWeise/tspSuite> of our own project *TSPSuite*, which contains all the TSPLib instances as well as Java code to load them. (Cit. on p. 297).
- [83] Ronald Linn Rivest, Adi Shamir, and Leonard M. Adleman. “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems”. *Communications of the ACM (CACM)* 21(2):120–126, Feb. 1978. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/359340.359342 (cit. on pp. 229–238, 296).
- [84] Günter Rudolph. *Convergence Properties of Evolutionary Algorithms*. Vol. 35 of Forschungsergebnisse zur Informatik. Hamburg, Germany: Verlag Dr. Kovač, 1997. ISSN: 1435-6260. ISBN: 978-3-86064-554-3 (cit. on pp. 196–198, 294).

References XII



- [85] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach (AIMA)*. 4th ed. Hoboken, NJ, USA: Pearson Education, Inc. ISBN: 978-1-292-40113-3. URL: <https://aima.cs.berkeley.edu> (visited on 2024-06-27) (cit. on p. 292).
- [86] Sartaj Sahni and Teofilo Francisco Gonzalez Arce. "P-complete Approximation Problems". *Journal of the ACM (JACM)* 23(3):555–565, July 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0004-5411. doi:10.1145/321958.321975. URL: <https://sites.cs.ucsb.edu/~teo/papers/JACM-PCOM.pdf> (visited on 2026-02-21) (cit. on p. 296).
- [87] Guntram Scheithauer. *Introduction to Cutting and Packing Optimization: Problems, Modeling Approaches, Solution Methods*. Vol. 263 of International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2018. ISSN: 0884-8289. ISBN: 978-3-319-64402-8. doi:10.1007/978-3-319-64403-5 (cit. on pp. 4–9).
- [88] Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge, England, UK: Cambridge University Press (CUP), July 2014. ISBN: 978-1-107-05713-5. URL: <http://www.cs.huji.ac.il/~shais/UnderstandingMachineLearning> (visited on 2024-06-27) (cit. on p. 295).
- [89] Éric D. Taillard. "Benchmarks for Basic Scheduling Problems". *European Journal of Operational Research* 64(2):278–285, Jan. 1993. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(93)90182-M. URL: <https://arodes.hes-so.ch/record/8224/files/Preprint.pdf> (visited on 2026-02-20) (cit. on p. 293).
- [90] Alkin Tezuysal and Ibrar Ahmed. *Database Design and Modeling with PostgreSQL and MySQL*. Birmingham, England, UK: Packt Publishing Ltd, July 2024. ISBN: 978-1-80323-347-5 (cit. on p. 295).
- [91] Sarah Louise Thomson, Gabriela Ochoa, Daan van den Berg, Tianyu Liang (梁天宇), and Thomas Weise (汤卫恩). "Entropy, Search Trajectories, and Explainability for Frequency Fitness Assignment". In: *Parallel Problem Solving from Nature (PPSN XVIII)*. Vol. 1. Sept. 14–18, 2024, Hagenberg, Mühlkreis, Austria. Ed. by Michael Affenzeller, Stephan M. Winkler, Anna V. Kononova, Heike Trautmann, Tea Tušar, Penousal Machado, and Thomas Bäck. Vol. 15148 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer. ISSN: 0302-9743. ISBN: 978-3-031-70054-5. doi:10.1007/978-3-031-70055-2_23 (cit. on pp. 250, 251, 293, 295).

References XIII



- [92] Clarissa Van Hoyweghen, David Edward Goldberg, and Bart Naudts. "From Twomax To The Ising Model: Easy And Hard Symmetrical Problems". In: *4th Genetic and Evolutionary Computation Conference (GECCO'2002)*. July 9–13, 2002, New York, NY, USA. Ed. by William B. Langdon, Erick Cantú-Paz, Keith E. Mathias, Rajkumar Roy, David Davis, Riccardo Poli, Karthik Balakrishnan, Vasant G. Honavar, Günter Rudolph, Joachim Wegener, Larry Bull, Mitchell A. Potter, Alan C. Schultz, Julian F. Miller, Edmund K. Burke, and Natasa Jonoska. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 2002, pp. 626–633. ISBN: 978-1-55860-878-8 (cit. on pp. 199–201, 298).
- [93] Kristian Verduin, Sarah Louise Thomson, and Daan van den Berg. "Too Constrained for Genetic Algorithms. Too Hard for Evolutionary Computing. The Traveling Tournament Problem". In: *15th International Joint Conference on Computational Intelligence (IJCCI'23)*. Nov. 13–15, 2023, Rome, Italy. Ed. by Niki van Stein, Francesco Marcelloni, H. K. Lam, Marie Cottrell, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2023, pp. 246–257. ISSN: 2184-3236. ISBN: 978-989-758-674-3. doi:10.5220/0012192100003595 (cit. on p. 298).
- [94] Thomas Weise (汤卫思). *Databases*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2025. URL: <https://thomasweise.github.io/databases> (visited on 2025-01-05) (cit. on pp. 272, 292).
- [95] Thomas Weise (汤卫思). *Global Optimization Algorithms – Theory and Application*. self-published, 2009. URL: <https://www.researchgate.net/publication/200622167> (visited on 2025-07-25) (cit. on pp. 68–82, 293, 294, 296).
- [96] Thomas Weise (汤卫思). *Programming with C*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2026. URL: <https://thomasweise.github.io/programmingWithC> (visited on 2026-05-10) (cit. on p. 292).
- [97] Thomas Weise (汤卫思). *Programming with Python*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2024–2026. URL: <https://thomasweise.github.io/programmingWithPython> (visited on 2025-01-05) (cit. on pp. 271, 295).

References XIV

- [98] Thomas Weise (汤卫恩), Raymond Chiong, Jörg Lässig, Ke Tang (唐珂), Shigeyoshi Tsutsui, Wenxiang Chen (陈文祥), Zbigniew "Zbyszek" Michalewicz, and Xin Yao (姚新). "Benchmarking Optimization Algorithms: An Open Source Framework for the Traveling Salesman Problem". *IEEE Computational Intelligence Magazine (CIM)* 9(3):40–52, Aug. 2014. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1556-603X. doi:10.1109/MCI.2014.2326101 (cit. on pp. 13–16, 293, 297).
- [99] Thomas Weise (汤卫恩), Xinlu Li (李新路), Yan Chen (陈岩), and Zhize Wu (吴志泽). "Solving Job Shop Scheduling Problems without using a Bias for Good Solutions". In: *23rd Genetic and Evolutionary Computation Conference (GECCO'2021), Companion Volume*. July 10–14, 2021, Lille, France. Ed. by Krzysztof Krawiec. New York, NY, USA: Association for Computing Machinery (ACM), 2021, pp. 1459–1466. ISBN: 978-1-4503-8351-6. doi:10.1145/3449726.3463124 (cit. on pp. 252, 253, 277).
- [100] Thomas Weise (汤卫恩), Li Niu (牛力), and Ke Tang (唐珂). "AOAB – Automated Optimization Algorithm Benchmarking". In: *12th Annual Conference Companion on Genetic and Evolutionary Computation (GECCO'2010)*. July 7–11, 2010, Portland, OR, USA. Ed. by Martin Pelikan and Jürgen Branke. New York, NY, USA: Association for Computing Machinery (ACM), 2010, pp. 1479–1486. ISBN: 978-1-4503-0073-5. doi:10.1145/1830761.1830763 (cit. on p. 293).
- [101] Thomas Weise (汤卫恩), Alexander Podlich, and Christian Gorltd. "Solving Real-World Vehicle Routing Problems with Evolutionary Algorithms". In: *Natural Intelligence for Scheduling, Planning and Packing Problems*. Ed. by Raymond Chiong and Sandeep Dhakal. Vol. 250 of *Studies in Computational Intelligence (SCI)*. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, Sept. 2009. Chap. 2, pp. 29–53. ISSN: 1860-949X. ISBN: 978-3-642-04038-2. doi:10.1007/978-3-642-04039-9_2 (cit. on pp. 4–9).
- [102] Thomas Weise (汤卫恩), Alexander Podlich, Manfred Menze, and Christian Gorltd. "Optimierte Güterverkehrsplanung mit Evolutionären Algorithmen". *Industrie Management – Zeitschrift für industrielle Geschäftsprozesse* 10(3):37–40, June 2009. Berlin, Germany: GITO mbH Verlag (cit. on pp. 4–9).
- [103] Thomas Weise (汤卫恩), Alexander Podlich, Kai Reinhard, Christian Gorltd, and Kurt Geihs. "Evolutionary Freight Transportation Planning". In: *Applications of Evolutionary Computing (EvoWorkshops 2009): Proceedings of EvoCOMNET, EvoENVIRONMENT, EvoFIN, EvoGAMES, EvoHOT, EvoIASP, EvoINTERACTION, EvoMUSART, EvoNUM, EvoSTOC, EvoTRANSLOG*. Apr. 15–17, 2009, Tübingen, Baden-Württemberg, Germany. Ed. by Mario Giacobini, Anthony Brabazon, Stefano Cagnoni, Gianni A. Caro, Anikó Ekárt, Anna Isabel Esparcia-Alcázar, Muddassar Farooq, Andreas Fink, and Penousal Machado. Vol. 5484 of *Theoretical Computer Science and General Issues (LNTCS)*, sub-series of *Lecture Notes in Computer Science (LNCS)*. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, Apr. 2009, pp. 768–777. ISSN: 2512-2010. doi:10.1007/978-3-642-01129-0_87 (cit. on pp. 4–9).

References XV



- [104] Thomas Weise (汤卫恩) and Ke Tang (唐珂). "Evolving Distributed Algorithms with Genetic Programming". *IEEE Transactions on Evolutionary Computation* 16(2):242–265, Apr. 2012. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2011.2112666 (cit. on p. 293).
- [105] Thomas Weise (汤卫恩), Mingxu Wan (万明绪), Ke Tang (唐珂), and Xin Yao (姚新). "Evolving Exact Integer Algorithms with Genetic Programming". In: *IEEE Congress on Evolutionary Computation (CEC'2014)*. July 6–11, 2014, China, Beijing (中国北京市). Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2014, pp. 1816–1823. ISBN: 978-1-4799-1488-3. doi:10.1109/CEC.2014.6900292 (cit. on pp. 254, 255).
- [106] Thomas Weise (汤卫恩), Mingxu Wan (万明绪), Pu Wang (王璞), Ke Tang (唐珂), Alexandre Devert, and Xin Yao (姚新). "Frequency Fitness Assignment". *IEEE Transactions on Evolutionary Computation* 18(2):226–243, Apr. 2014. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2013.2251885 (cit. on pp. 246, 247, 254, 255, 293).
- [107] Thomas Weise (汤卫恩) and Zhize Wu (吴志泽). "Replicable Self-Documenting Experiments with Arbitrary Search Spaces and Algorithms". In: *25th Conference on Genetic and Evolutionary Computation (GECCO'2023), Companion Volume*. July 15–19, 2023, Lisbon, Portugal. Ed. by Sara Silva and Luís Paquete. New York, NY, USA: Association for Computing Machinery (ACM), 2023, pp. 1891–1899. ISBN: 979-8-4007-0120-7. doi:10.1145/3583133.3596306 (cit. on pp. 273, 293, 295).
- [108] Thomas Weise (汤卫恩), Zhize Wu (吴志泽), Xinlu Li (李新路), and Yan Chen (陈岩). "Frequency Fitness Assignment: Making Optimization Algorithms Invariant under Bijective Transformations of the Objective Function Value". *IEEE Transactions on Evolutionary Computation* 25(2):307–319, Apr. 2021. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2020.3032090 (cit. on pp. 166, 193–209, 220, 246, 247, 252, 253, 292–295, 297, 298).
- [109] Thomas Weise (汤卫恩), Zhize Wu (吴志泽), Xinlu Li (李新路), Yan Chen (陈岩), and Jörg Lässig. "Frequency Fitness Assignment: Optimization without Bias for Good Solutions can be Efficient". *IEEE Transactions on Evolutionary Computation* 27(4):980–992, 2023. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2022.3191698 (cit. on pp. 246, 247, 292–294, 297, 298).
- [110] Thomas Weise (汤卫恩), Yuezhong Wu (吴越钟), Raymond Chiong, Ke Tang (唐珂), and Jörg Lässig. "Global versus Local Search: The Impact of Population Sizes on Evolutionary Algorithm Performance". *Journal of Global Optimization* 66(3):511–534, Feb. 2016. London, England, UK: Springer Nature Limited. ISSN: 0925-5001. doi:10.1007/s10898-016-0417-5 (cit. on pp. 13–16, 297).

References XVI



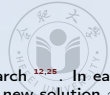
- [111] L. Darrell Whitley. "The GENITOR Algorithm and Selection Pressure: Why Rank-Based Allocation of Reproductive Trials is Best". In: *3rd International Conference on Genetic Algorithms (ICGA'1989)*. June 1989, Fairfax, VA, USA: George Mason University. Ed. by J. David Schaffer. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 1989, pp. 116–123. ISBN: 978-1-55860-066-9. URL: <https://www.researchgate.net/publication/2527551> (visited on 2025-08-08) (cit. on pp. 196–198, 294, 296).
- [112] CAO Xiang (曹翔), Zhize Wu (吴志泽), Daan van den Berg, and Thomas Weise (汤卫思). "Randomized Local Search vs. NSGA-II vs. Frequency Fitness Assignment on The Traveling Tournament Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 38–49. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012891500003837 (cit. on pp. 256–262, 295, 298).
- [113] Kinza Yasar and Craig S. Mullins. *Definition: Database Management System (DBMS)*. Newton, MA, USA: TechTarget, Inc., June 2024. URL: <https://www.techtarget.com/searchdatamanagement/definition/database-management-system> (visited on 2025-01-11) (cit. on p. 292).
- [114] François Yergeau. *UTF-8, A Transformation Format of ISO 10646*. Request for Comments (RFC) 3629. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Nov. 2003. URL: <https://www.ietf.org/rfc/rfc3629.txt> (visited on 2025-02-05). See Unicode and ⁴⁶ (cit. on p. 298).
- [115] Anne L. Young. *Mathematical Ciphers: From Caesar to RSA*. Vol. 25 of Mathematical World. Providence, RI, USA: American Mathematical Society (AMS), 2006. ISSN: 1055-9426. ISBN: 978-0-8218-3730-6 (cit. on pp. 229–238, 296).
- [116] Jiazheng Zeng (曾嘉政), Thomas Weise (汤卫思), Zhize Wu (吴志泽), and Markus Wagner. "When Frequency Fitness Assignment Fails: Trapped States in Frequency-Guided Local Search". In: *28th Genetic and Evolutionary Computation Conference (GECCO'2026)*. July 13–17, 2026, San José, Costa Rica. Ed. by Leonardo Trujillo and Ting Hu. New York, NY, USA: Association for Computing Machinery (ACM), 2026, pp. 233–242. ISBN: 979-8-4007-2487-9. doi:10.1145/3795095.3805098 (cit. on pp. 105–110, 199–201, 256–262, 292–298).

References XVII



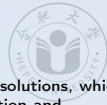
- [117] Rui Zhao (赵睿), Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thüerer, and Thomas Weise (汤卫思). "Randomized Local Search on the 2D Rectangular Bin Packing Problem with Item Rotation". In: *26th Genetic and Evolutionary Computation Conference (GECCO'2024)*. July 14–18, 2024, Melbourne, VIC, Australia. Ed. by Xiaodong Li and Julia Handl. New York, NY, USA: Association for Computing Machinery (ACM), 2024, pp. 235–238. ISBN: 979-8-4007-0494-9. doi:10.1145/3638530.3654139 (cit. on pp. 4–9, 25–28, 256–262, 295).
- [118] Rui Zhao (赵睿), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thüerer, Tianyu Liang (梁天宇), Ming Tan (檀明), and Thomas Weise (汤卫思). "Randomized Local Search for Two-Dimensional Bin Packing and a Negative Result for Frequency Fitness Assignment". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 15–26. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888500003837 (cit. on pp. 4–9, 25–28, 256–262, 295).

Glossary I



- (1 + 1) EA The (1 + 1) EA is a local search algorithm that retains the best solution x_c discovered so far during the search^{12,25}. In each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n . If the new solution x_n is *better or equally good* when compared with x_c , i.e., not worse, then it replaces it, i.e., is stored as the new x_c . If the search space are bit strings of length n , then the (1 + 1) EA uses a unary search operator that flips each bit independently with probability m/n , where usually $m = 1$. This operator is the main difference to randomized local search (RLS). The (1 + 1) EA is a special case of the $(\mu + \lambda)$ evolutionary algorithm $((\mu + \lambda)$ EA) where $\mu = \lambda = 1$.
- (1 + 1) FEA The (1 + 1) EA with FFA plugged in^{108,109,116}. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips each bit with probability $1/n$, where n is the dimension of the search space, distinguishing it from the unary operator in RLS with FFA (FRLS) which flips exactly one bit. On other domains, such as permutations, the term is used synonymously with FRLS.
- $(\mu + \lambda)$ EA The $(\mu + \lambda)$ EA is an evolutionary algorithm (EA) where, in each generation, λ offspring solutions are generated from the current population of μ parent solutions. The offspring and parent populations are merged, yielding $\mu + \lambda$ solutions, from which then the best μ solutions are retained to form the parent population of the next generation. If the search space is the bit strings of length n , then this algorithm usually applies a mutation operator flipping each bit independently with probability $1/n$.
- AI Artificial Intelligence, see, e.g.,⁸⁵
- C is a programming language, which is very successful in system programming situations^{23,78,96}.
- Concorde is an exact solver for the TSP^{3,18,19}, which could even solve the largest instance of TSPLib (with 85 900 cities) to optimality.
- DB A *database* is an organized collection of structured information or data, typically stored electronically in a computer system. Databases are discussed in our book *Databases*⁹⁴.
- DBMS A *database management system* is the software layer located between the user or application and the database (DB). The DBMS allows the user/application to create, read, write, update, delete, and otherwise manipulate the data in the DB¹¹³.

Glossary II



EA An *evolutionary algorithm* is a metaheuristic optimization method that maintains a population of candidate solutions, which undergo selection (where better solutions are chosen with higher probability) and reproduction (where mutation and recombination create a new candidate solution from one or two existing ones, respectively) ^{5,95}.

FE *Objective function evaluations* are an implementation-independent measure of runtime for optimization algorithms ^{6,98,100,107}. 1 FE equals to one evaluated candidate solution during the optimization process.

FFA *Frequency Fitness Assignment* (中文: 频率适应度分配) is a algorithm plugin for optimization methods applied to discrete or combinatorial problems with not-too-many different possible objective values ^{106,108,109}. It replaces the objective values in all comparisons with their absolute encounter frequency so far during the search. FFA has successfully been applied to the MaxSAT ^{108,109}, the TSP ^{59,60}, and the QAP ^{15,91}, amongst others. Its properties have been intensely researched on discrete benchmark functions such as OneMax, LeadingOnes, Trap, TwoMax, Jump, and Plateau ^{108,109,116}. The most common realizations of FFA so far are the $(1 + 1)$ FEA and FRLS.

GP Genetic Programming ^{54,76,95,104} is the application of metaheuristic optimization, usually in form of an EAs, to a search space comprised of tree datastructures. These tree datastructures often represent programs or mathematical expressions.

Java is another very successful programming language, with roots in the C family of languages ^{10,63}.

JSSP The *Job Shop Scheduling Problem* ^{9,10,56} is one of the most prominent and well-studied scheduling tasks. In a JSSP instance, there are k machines and m jobs ⁸⁹. Each job must be processed once by each machine in a job-specific sequence and has a job-specific processing time on each machine. The goal is to find an assignment of jobs to machines that results in an overall shortest makespan, i.e., the schedule which can complete all the jobs in the shortest time. The JSSP is $\mathcal{NP}$ -complete ^{14,56}.

Glossary III



Jump The Jump problem is a benchmark problem for bit-string based black-box search^{25,32,116}. Like the OneMax, it tries to maximize the number of 1 bits in a bit string of chosen length n . However, the solutions representing the last ω bits around the optimum to flip are deceptive, meaning that they are actually worse and getting worse than surrounding solutions. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and $\omega + \|x\|_1$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. This means that, the larger ω , the larger the deceptive basin around the optimum that an algorithm needs to cross. An $(1+1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^\omega + n \ln n)$ ²⁵. In experiments, the $(1+1)$ FEA achieves polynomial runtime on Jump, seemingly in $\mathcal{O}(n^2 \ln n)$ ^{108,109,116}.

LeadingOnes The LeadingOnes problem is a benchmark problem for bit-string based black-box search^{84,111,116}. It tries to maximize the number of *leading* 1 bits in the bit string of a chosen dimension n . The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^n \prod_{j=1}^i x_j$. Different from OneMax, the contribution to the objective value of the bit at index i thus depends on the state of all the bits *before* this index. The problem is unimodal, i.e., has only a single global optimum and no local optimum. Different from OneMax, it is not separable, i.e., we can optimize each variable separately. The expected runtime of $(1+1)$ EA on LeadingOnes is in $\Theta(n^2)$ ²⁵. The average runtime $(1+1)$ FEA on LeadingOnes seems to be slower by factor proportional to n , i.e., seems to be in $\mathcal{O}(n^3)$ ¹⁰⁸.

MaxSAT The goal of satisfiability problems is to find an assignment for n Boolean variables that make a given Boolean formula $F : \{0, 1\}^n \mapsto \{0, 1\}$ become true. In the *Maximum Satisfiability* problem (MaxSAT)⁴², F is given in conjunctive normal form, i.e., the variables appear either directly or negated in m "or" clauses, which are all combined into one "and." The objective function $f(x)$, subject to minimization, computes the number of clauses which are false under the variable setting x . If $f(x) = 0$, then all clauses of F are true, which solves the problem. The MaxSAT problem is $\mathcal{NP}$ -complete¹⁷ and exhibits low epistasis but deceptiveness⁸⁰, making it hard for local searches.

metaheuristic A *metaheuristic* is an optimization algorithm which performs a cycle of trial and error during which candidate solutions are sampled from the solution space $\mathbb{Y}$ and evaluated using an objective function $f : \mathbb{Y} \mapsto \mathbb{R}$ ^{36,95}. The results of the objective function are used to decide which solution is promising. The neighborhoods around promising solutions or similarly-structured solutions are then sampled in the next cycle. Prominent metaheuristics are SLSes, SA, TS, and EAs.

Glossary IV



ML Machine Learning, see, e.g., ⁸⁸

moptipy is the *Metaheuristic Optimization in Python* library ¹⁰⁷. It has been used in several different research works, including ^{15,59–61,91,112,117,118}. Algorithms implemented by moptipy include EAs, (1 + 1) EA, RLS, SA, and the (1 + 1) FEA, amongst others. Learn more at <https://thomasweise.github.io/moptipy> and <https://thomasweise.github.io/moptipyapps>.

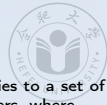
OneMax OneMax is the simplest possible benchmark problem for bit-string based black-box search ^{26,68,116}. The goal is to maximize the number of 1 bits in the bit string of a chosen dimension n . The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^n x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately. The expected runtime of (1 + 1) EA on OneMax is in $\Theta(n \ln n)$ ^{24,68}. The average runtime (1 + 1) FEA on OneMax seems to be slower by factor proportional to n , i.e., seems to be in $\mathcal{O}(n^2 \ln n)$ ¹⁰⁸.

Plateau The Plateau problem is a benchmark problem for bit-string based black-box search ^{2,116}. It is structured exactly like Jump, but instead of being deceptive, the last ω bits to flip around the optimum are neutral. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and ω otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. This means that, the larger ω , the larger the neutral plane around the optimum that an algorithm needs to cross. An (1 + 1) EA has an expected runtime of $\Theta(n^\omega)$ on plateaus ². The average runtime of the (1 + 1) FEA on Plateaus is slower by a factor seemingly proportional in n .

PostgreSQL An open source object-relational DBMS ^{30,70,77,90}. See <https://postgresql.org> for more information.

Python The Python programming language ^{45,58,64,97}, i.e., what you will learn about in our book ⁹⁷. Learn more at <https://python.org>.

Glossary V



QAP The *Quadratic Assignment Problem* is an optimization problem where the goal is to assign a set of n facilities to a set of n locations^{11,15,53,62}. Such an assignment can be represented as a permutation x of the first n natural numbers, where x_i specifies the location where facility i should be placed. For each QAP, a distance matrix D is given, where D_{pq} specifies the distance from location p to location q , as well as a flow matrix F , where F_{ij} is the amount of material flowing from facility i to facility j . The objective function $f(x)$ then rates a permutation x as $f(x) = \sum_{i=1}^n \sum_{j=1}^n D_{x_i x_j} F_{ij}$. The QAP is $\mathcal{NP}$ -complete⁸⁶.

RLS Randomized local search retains the best solution x_c discovered so far during the search and, in each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n . If the new solution x_n is *better or equally good* when compared with x_c , i.e., not worse, then it replaces it, i.e., is stored as the new x_c . If the search space are bit strings of length n , then RLS uses a unary search operator that flips exactly one bit. This operator is the main difference to $(1 + 1)$ EA.

FRLS The RLS with FFA plugged in¹¹⁶. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips exactly one bit, distinguishing it from the unary operator in the $(1 + 1)$ FEA, which flips each bit with probability $1/n$. On other domains, such as permutations, we use the term synonymously with $(1 + 1)$ FEA.

RSA The RSA algorithm^{83,115}, named after its inventors Rivest, Shamir, and Adleman, is a widely used asymmetric cryptographic technique for digital signatures, key exchange, and encryption. In the latter case, it uses a public key for encryption and a private key for decryption. It is based on the mathematical complexity of factorizing large integers.

SA Simulated Annealing is a SLS that sometimes accepts a worse solution^{13,48,50,74}. The probability to do so decreases over time and with the difference in objective values, i.e., is the lower the worse the new solution is.

SATLIB is a library of benchmark instances for the Maximum Satisfiability (MaxSAT) available at <https://www.cs.ubc.ca/~hoos/SATLIB/index-ubc.html>^{43,44}.

SGA The Standard Genetic Algorithm^{5,21,38,66,67,95} was the first EA. It maintains a population of solutions and applies mutation and crossover to generate offspring solutions. It uses fitness proportionate selection to choose which solutions should “survive” into the next generation, which today is considered a very bad design choice, see, e.g.,¹¹¹.

Glossary VI



SLS *Stochastic local search* or *local search* for short, is one of the simplest metaheuristic search paradigms⁴²: A single “current” candidate solution is maintained and new solutions are sampled from its neighborhood. If these solutions are deemed interesting based on their objective value, they replace the “current”. The cycle continues. Basic examples are RLS and $(1 + 1)$ EA. More complex examples that add mechanisms to escape local optima on top of the SLS paradigm are TS and SA.

Trap The Trap problem is a benchmark problem for bit-string based black-box search^{25,69,116}. It is structured exactly like OneMax, however the worst and the best solution are swapped. This makes the problem highly deceptive, as following the “gradient” in the search space leads the search away from the global optimum. For bit strings of length n , the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = 0$ and $n - \|x\|_1 - 1$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. An $(1 + 1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^n)$ ²⁵. In experiments, the $(1 + 1)$ FEA achieves polynomial runtime on Trap, seemingly in $\mathcal{O}(n^2 \ln n)$ ^{108,109,116}.

TS *Tabu Search* is one of the most successful SLSes^{34,35,37}. In order to avoid returning to previous solutions, it declares certain solutions or search moves as “tabu.” For example, if a bit in a discrete problem has been flipped by the search operator, it may be forbidden to flip the bit again for some time, thus preventing the algorithm from getting trapped in a cycle. This rule may be overridden if a new best-so-far solution is discovered that outperforms all previous solutions, in which case it would be accepted (this is the so-called aspiration criterion).

TSP In an instance of the *Traveling Salesperson Problem*, also known as *Traveling Salesman Problem*, a set of n cities or locations as well as the distances between them are defined^{3,40,57,59,98,110}. The goal is to find the shortest round-trip tour that starts at one city, visits all the other cities one time each, and returns to the origin. The TSP is one of the most well-known $\mathcal{NP}$ -hard combinatorial optimization problems^{33,40}.

TSPLib is a library of benchmark instances for the Traveling Salesperson Problem (TSP)^{81,82}. Its original URL was available until 2025 and then became defunct. The repository <https://github.com/shredderzjw/TSPLIB> contains the data. Additionally, you may also check <https://github.com/thomasWeise/tspSuite>, which contains all the TSPLib instances as well as Java code to load them. The TSPLib contains 110 instances of symmetric TSPs with scales n ranging from 14 to 85 900, but 93 instances have less than 2000 nodes. There also are 19 asymmetric TSPs with scales $n \in 17..443$.

Glossary VII



TTP The *Traveling Tournament Problem* is the combinatorial optimization problem of both efficiently and fairly organizing a tournament of n teams that play against each other in a pairwise fashion^{29,112}. The efficient part boils down to arranging the games such that the total travel length is short, which is somewhat similar to the classical TSP. Initially, each team is at its home location. On each day, a team needs to travel if its scheduled game is not at its present location. On the last day, each team may need to travel back home unless their last game is a home game. The total travel length sums up the lengths of all travels over all teams. The fair part is represented in several constraints, such as doubleRoundRobin, compactness, maxStreak, and noRepeat. The TTP is $\mathcal{NP}$ -hard⁹³.

TwoMax The TwoMax problem is a benchmark problem for bit-string based black-box search^{32,92,116}. It has a global and a local optimum, both of about the same size, at opposite ends of the search space. For bit strings of length n , the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = n$ and $1 + n - \max\{\|x\|_1, n - \|x\|_1\}$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. An $(1+1)$ EA has a 50/50 chance to converge to the local or global optimum and thus has an expected runtime in $\Theta(n^n)$ ^{31,32}. In experiments, the $(1+1)$ FEA achieves polynomial runtime seemingly in $\mathcal{O}(n^2 \ln n)$ on TwoMax^{108,109,116}.

URI A *Uniform Resource Identifier* is an identifier for an abstract or physical resource in the internet¹¹⁴. It can be a URL, a name, or both. URIs are supersets of URLs. The connection strings of the PostgreSQL DBMS are examples for URIs.

URL A *Uniform Resource Locator* identifies a resource in the World Wide Web (WWW) and a way to obtain it by describing a network access mechanism. The most notable example of URLs is the text you write into web browsers to visit websites⁸. URLs are subsets of Uniform Resource Identifiers (URIs).

WWW World Wide Web^{7,22}

$\circ^*$ the superscript $*$ symbol denotes that $\circ$ is a global optimum, i.e., there is no better solution than $\circ$.

$i..j$ with $i, j \in \mathbb{Z}$ and $i \leq j$ is the set that contains all integer numbers in the inclusive range from i to j . For example, $5..9$ is equivalent to $\{5, 6, 7, 8, 9\}$

Glossary VIII



$f(y)$ The *objective function* $f : \mathbb{Y} \mapsto \mathbb{R}$ can compute a cost or rating for each candidate solution y in the solution space $\mathbb{Y}$. The smaller this value, the better the solution, i.e., in the context of this work, objective functions are subject to minimization. Objective functions are not necessarily continuous and it is also quite common that they have integer or natural numbers as results, e.g., we often have $f : \mathbb{Y} \mapsto \mathbb{N}_0$ instead of $f : \mathbb{Y} \mapsto \mathbb{R}$.

$\gamma(x)$ The decoding function $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ translates point from the search space $\mathbb{X}$ to one point y in the solution space $\mathbb{Y}$. It is a very common case that $\mathbb{X} = \mathbb{Y}$ and that the decoding function γ is the identity mapping.

n We usually use n to denote the scale, i.e., the size of the problem to solve. For example, in a TSP, this may be the number of cities, in a discrete benchmark problem such as OneMax or Trap, this may be the number of bits over which the problem is defined, in a QAP, this may be the number of facilities and locations, and in a MaxSAT problem, this may be the number of variables that are input to the Boolean formula that we want to become true. There are also problems where the scale is not defined straightforwardly like this. For example, in a JSSP, we have a number of machines and a number of jobs, which together define the size of the problem. But even then, the scale n relates to the size of the solution space $\mathbb{Y}$, and, hence, also of the search space $\mathbb{X}$, which usually grow exponentially with n . And this is already the main issue: The larger n , the (exponentially) larger the number of possible solutions and thus, the larger the runtime of algorithms..

$\mathbb{N}_0$ the set of the natural numbers *including* 0, i.e., 0, 1, 2, 3, and so on. It holds that $\mathbb{N}_0 \subset \mathbb{Z}$.

$\mathcal{NP}$ is the class of computational problems that can be solved in polynomial time by a non-deterministic machine and can be verified in polynomial time by a deterministic machine (such as a normal computer) ³⁹.

$\mathcal{NP}$ -complete A decision problem is $\mathcal{NP}$ -complete if it is in $\mathcal{NP}$ and all problems in $\mathcal{NP}$ are reducible to it in polynomial time ^{39,79}. A problem is $\mathcal{NP}$ -complete if it is $\mathcal{NP}$ -hard and if it is in $\mathcal{NP}$.

$\mathcal{NP}$ -hard Algorithms that guarantee to find the correct solutions of $\mathcal{NP}$ -hard problems ^{14,17,56} have a worst-case runtime that grows exponentially with the problem scale. A problem is $\mathcal{NP}$ -hard if all problems in $\mathcal{NP}$ are reducible to it in polynomial time ³⁹.

$\Omega(g(x))$ If $f(x) = \Omega(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $f(x) \geq c * g(x) \geq 0 \forall x \geq x_0$ ^{51,52}. In other words, $\Omega(g(x))$ describes a lower bound for function growth.

Glossary IX



- $\mathcal{O}(g(x))$ If $f(x) = \mathcal{O}(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $0 \leq f(x) \leq c * g(x) \forall x \geq x_0$. In other words, $\mathcal{O}(g(x))$ describes an upper bound for function growth.
- $\Theta(g(x))$ If $f(x) = \Theta(g(x))$, then $f(x) = \mathcal{O}(g(x))$ and $f(x) = \Omega(g(x))$. In other words, $\Theta(g(x))$ describes an exact order of function growth.

$\mathbb{R}$ the set of the real numbers.

$\mathbb{R}^+$ the set of the positive real numbers, i.e., $\mathbb{R}^+ = \{x \in \mathbb{R} : x > 0\}$.

$\mathbb{X}$ The search space $\mathbb{X}$ is the space through which an optimization algorithm navigates in order to find good solutions. The search operators of metaheuristics are used to sample points $x \in \mathbb{X}$, for example. On many benchmark problems, the objective functions f directly evaluate the elements $x \in \mathbb{X}$, i.e., are defined as $f : \mathbb{X} \mapsto \mathbb{R}$. However, very often these elements x are not the same data structures that the user is interested in and/or that the objective functions evaluate: It is common that the search space $\mathbb{X}$ is a simple and plain data structure that can be tackled with standard operations or mathematical tools, such as real vectors $\mathbb{R}^n$, permutations, or bit strings, whereas the solution space $\mathbb{Y}$ can be something more complicated, like a Gantt chart. Then, the user only cares about the solutions $y \in \mathbb{Y}$ and not about the search space $\mathbb{X}$. A decoding $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ is employed to translate the $x \in \mathbb{X}$ to $y = \gamma(x)$ with $y \in \mathbb{Y}$.

x a point in the search space $\mathbb{X}$.

$\mathbb{Y}$ the set of candidate solutions of an optimization problem is called the *solution space* $\mathbb{Y}$. The solution space corresponds to the data structure holding the candidate solutions y . These are the elements that are passed to the user after the optimization process is completed. They are also the parameters of the objective function(s) $f : \mathbb{Y} \mapsto \mathbb{R}$.

y a candidate solution $y \in \mathbb{Y}$ is one possible solution to an optimization problem. After the optimization process is completed, one or multiple such solutions are passed to the user.

$\mathbb{Z}$ the set of the integers numbers including positive and negative numbers and 0, i.e., $\dots, -3, -2, -1, 0, 1, 2, 3, \dots$, and so on. It holds that $\mathbb{Z} \subset \mathbb{R}$.